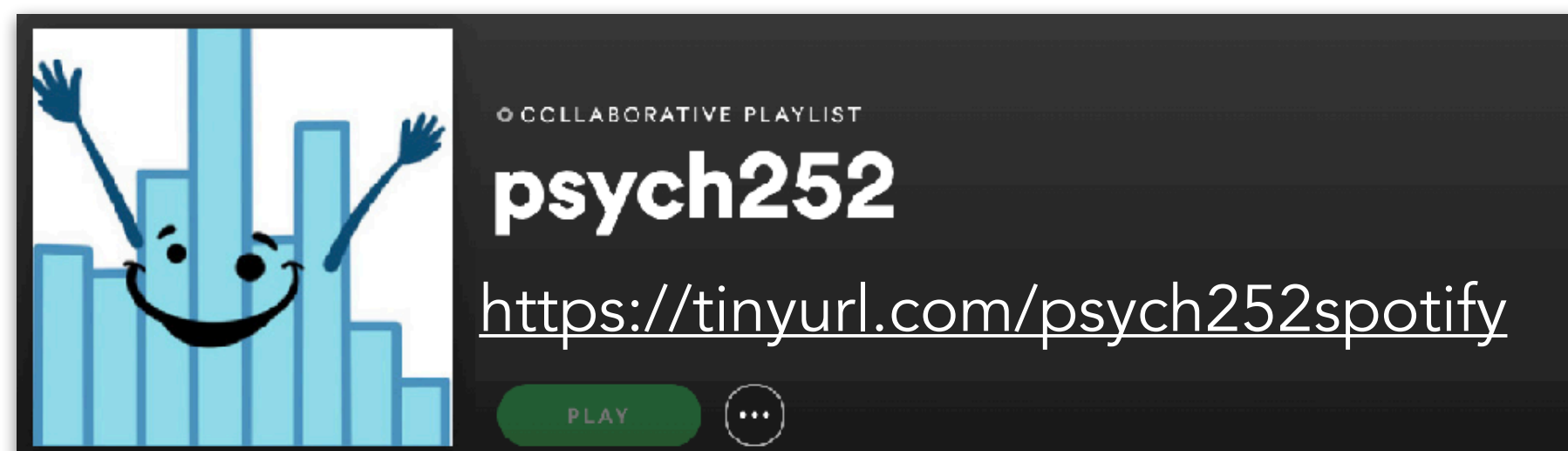
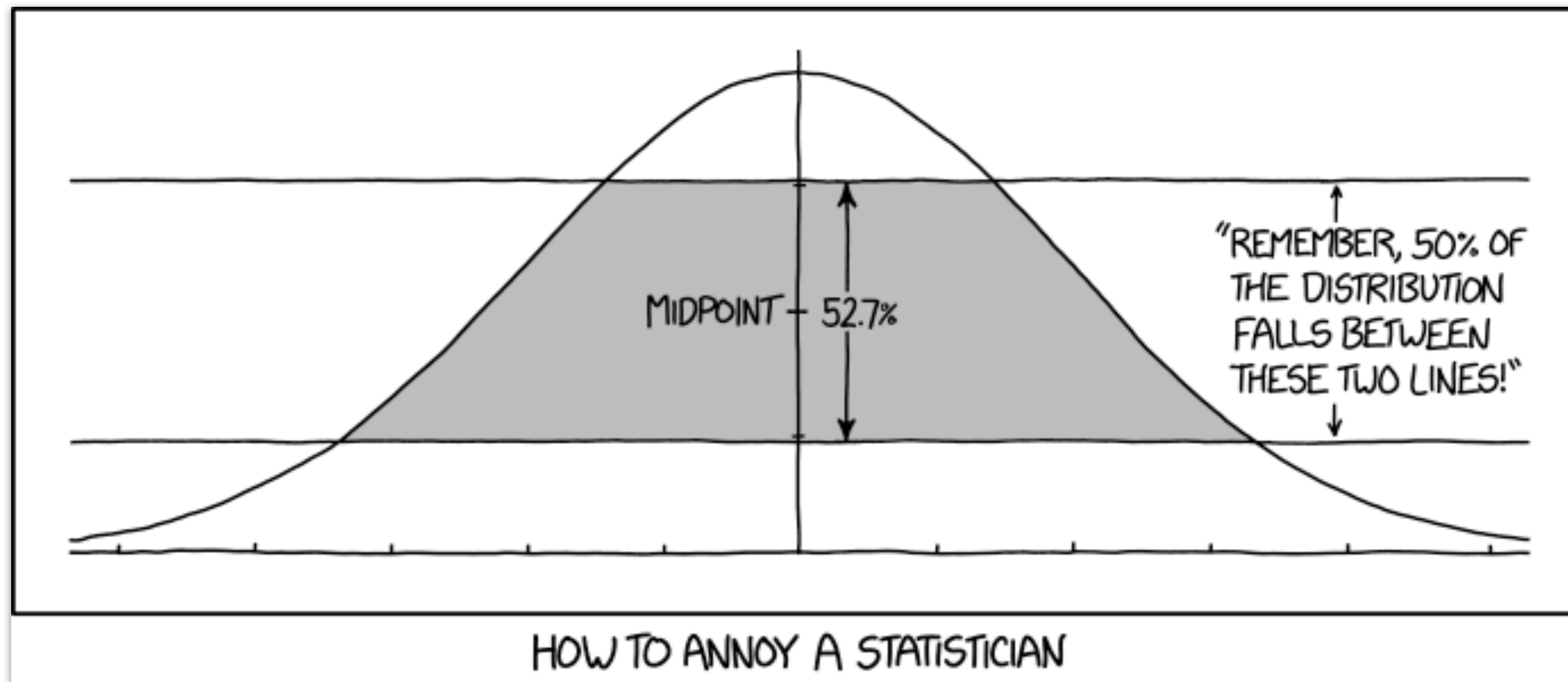


Bayesian data analysis 2



03/02/2026

Logistics

Final presentation

please fill out if you
haven't already

Questions Responses 20 Settings

Final presentation

Thanks for filling out this survey to help us with planning!

How are you planning to present? *

☐ In class (preferred option if possible)

☐ Remotely (live)

☐ I will record the presentation and submit a video before March 15th.

☐ Other...

What's your name (e.g. Tobias Gerstenberg)? *

Short answer text

What's the name of your team's github repository (e.g. final-project-tobi)? *

Short answer text

How many people are in your team (e.g. 1, 2, or 3)?

Short answer text

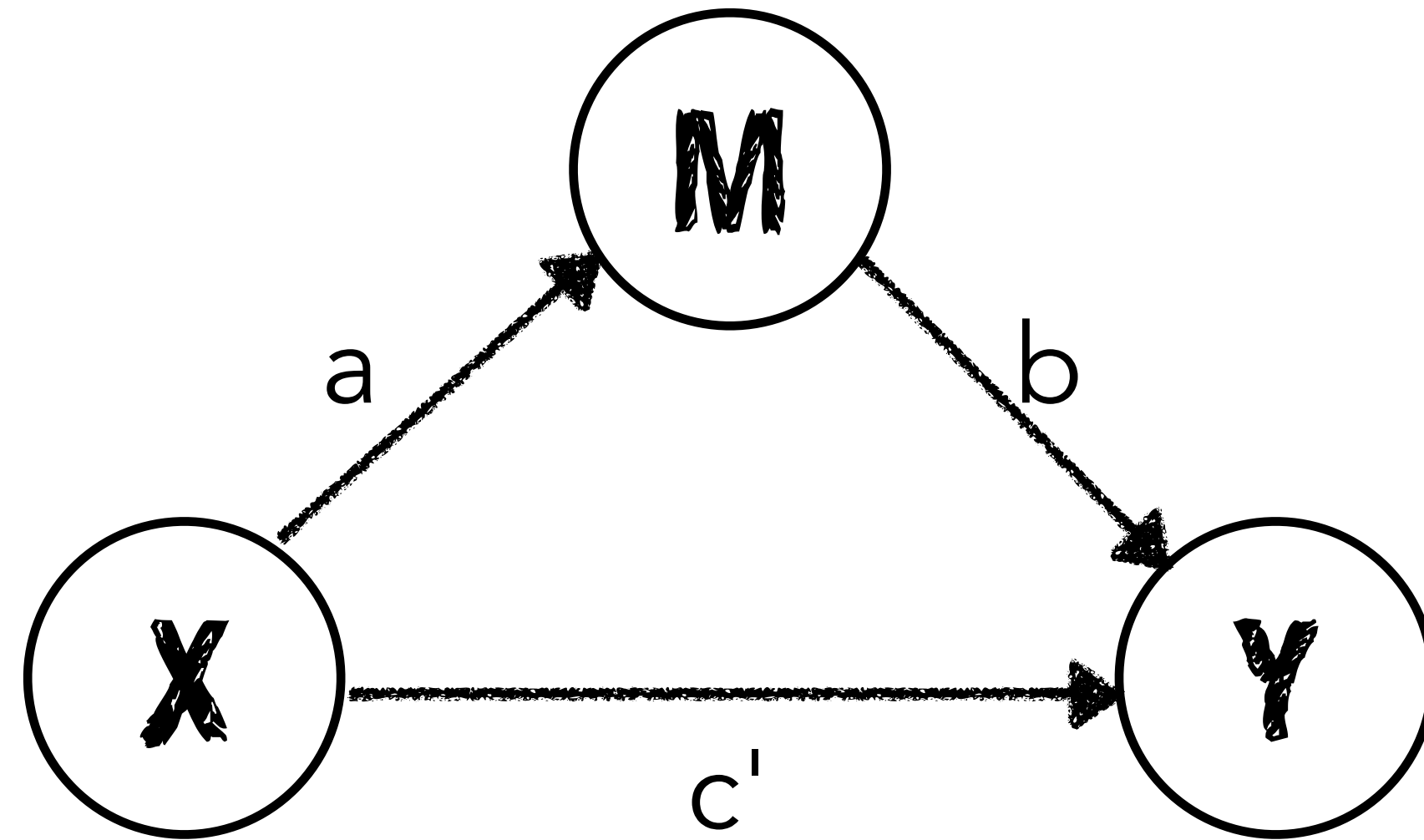
<https://tinyurl.com/psych252presentation26>

Plan for today

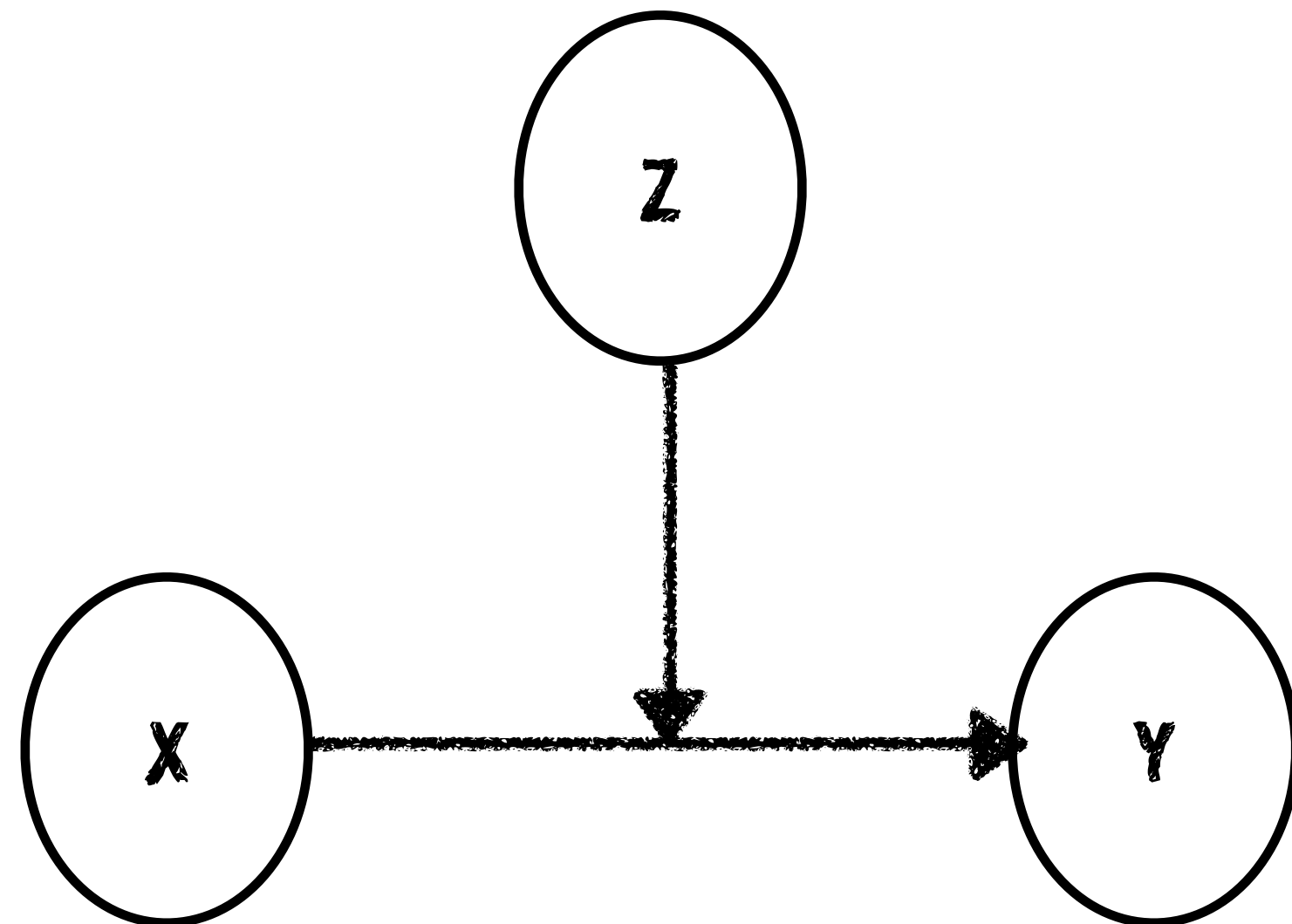
- Quick recap
- Bayesian data analysis
 - Ingredients: likelihood, prior, inference
 - Doing Bayesian data analysis
 - with `greta`
 - with BRMS

Quick recap

Quick recap: Mediation and Moderation



Mediation means that the **X** influences the mediator variable **M**, which in turn influences **Y**.



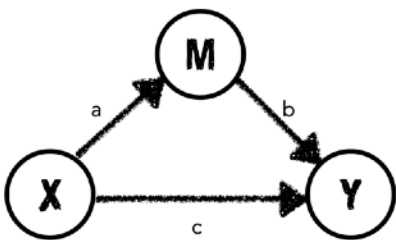
Moderation means that the effect of a predictor **X** on outcome **Y** depends on the value of another variable **Z**.

The nature of the relationship between **X** and **Y** depends on **Z**.

Quick recap: Mediation

Bootstrapping

```
1 library("mediation")
2
3 # bootstrapped mediation
4 fit.mediation = mediate(model.m = fit.m_x,
5                         model.y = fit.y_mx,
6                         treat = "x",
7                         mediator = "m",
8                         boot = T)
9
10 # summarize results
11 fit.mediation %>% summary()
```



$\hat{m} = b_0 + b_1 \cdot x$
 $\hat{y} = b_0 + b_1 \cdot m + b_2 \cdot x$

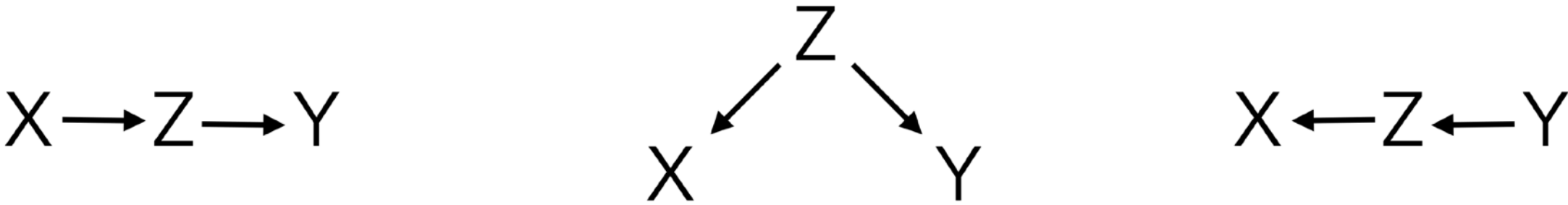
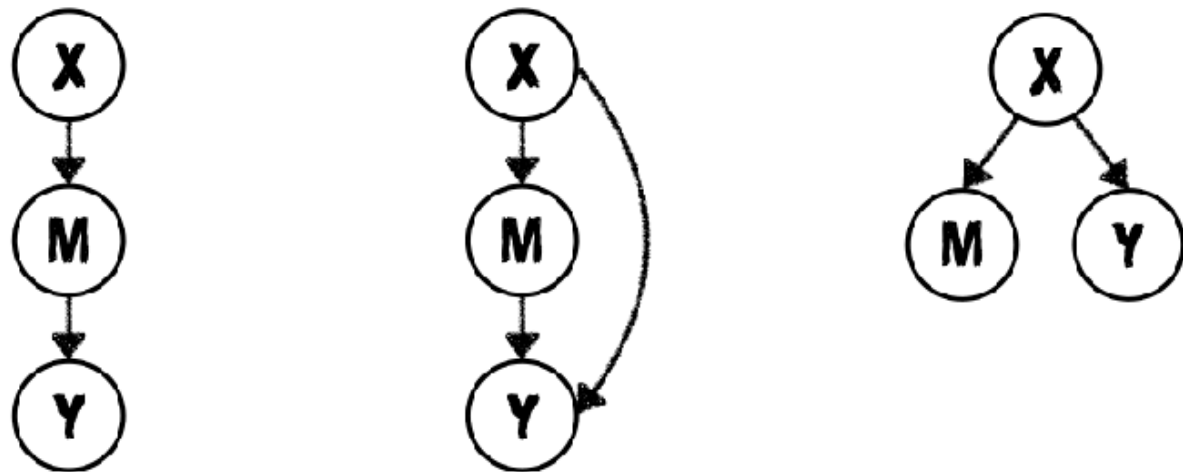
Causal Mediation Analysis					
Nonparametric Bootstrap Confidence Intervals with the Percentile Method					
	Estimate	95% CI Lower	95% CI Upper	p-value	
ACME	0.28078	0.14059	0.42	<2e-16 ***	
ADE	-0.11179	-0.29276	0.10	0.272	
Total Effect	0.16899	-0.00415	0.34	0.064 .	
Prop. Mediated	1.66151	-3.22476	11.46	0.064 .	

Signif. codes:	0 '***'	0.001 '**'	0.01 '*'	0.05 '.'	0.1 ' '
Sample Size Used: 100					
Simulations: 1000					



Underlying causal model

Full mediation Partial mediation No mediation

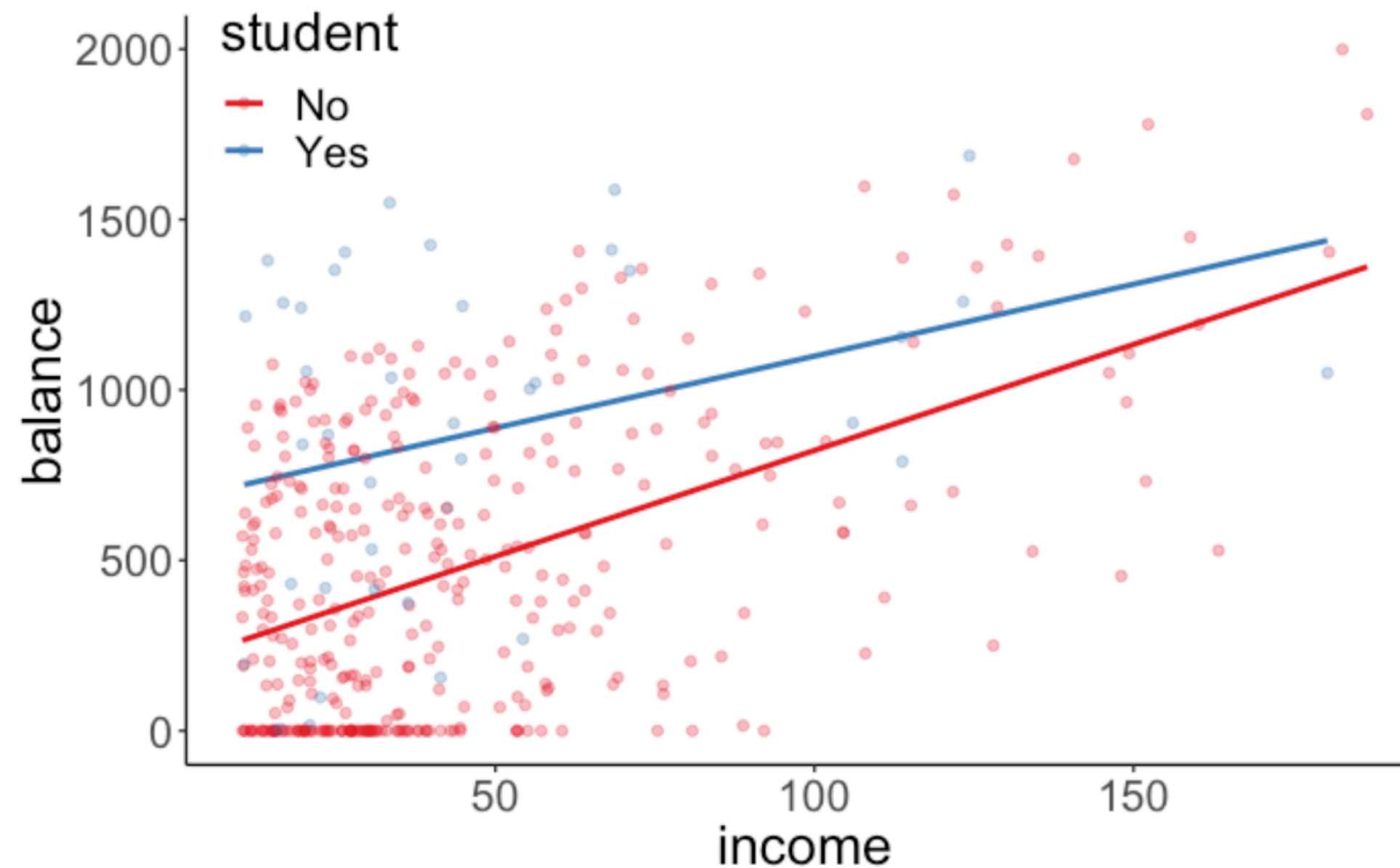


Full mediation: When the effect of **X** on **Y** completely disappears, **M** fully mediates between **X** and **Y**.

Partial mediation: When the effect of **X** on **Y** still exists, but in a smaller magnitude, **M** partially mediates between **X** and **Y**.

Quick recap: Moderation

Relationship between credit card balance, income, and whether the person is a student.



$$\widehat{\text{balance}}_i = 200.62 + 6.22 \cdot \text{income}_i + 476.68 \cdot \text{student}_i - 2.00 \cdot (\text{income}_i \times \text{student}_i)$$

if student = "No" $\widehat{\text{balance}}_i = 200.62 + 6.22 \cdot \text{income}_i$

if student = "Yes"

$$\begin{aligned}\widehat{\text{balance}}_i &= 200.62 + 6.22 \cdot \text{income}_i + 476.68 \cdot 1 - 2.00 \cdot (\text{income}_i \times 1) \\ &= 677.3 + 6.22 \cdot \text{income}_i - 2.00 \cdot \text{income}_i \\ &= 677.3 + 4.22 \cdot \text{income}_i\end{aligned}$$

Quick recap: Bayesian data analysis

Frequentists vs. Bayesian

Goal of data analysis: Inference about the world

Frequentist statistics

- generate a sampling distribution of the test statistic assuming H_0
- compare observed value of the test statistic with the sampling distribution
- reject the H_0 if probability of observed value (or more extreme values) is less than α

Bayesian statistics

- directly test hypotheses of interest
- define prior over hypotheses $p(H)$
- compute likelihood of the data for each hypothesis $p(D|H)$
- use Bayes' rule to infer the posterior over hypotheses given the data $p(H|D)$

- both "want" to evaluate the evidence for a hypothesis using a sample of data $p(H|D)$
- it's often easier to calculate the inverse: the probability of the data given a hypothesis $p(D|H)$
- frequentists use a rule of thumb (p-value) to make a decision
- Bayesians use Bayes' rule

Why don't more people use Bayesian Statistics?

- supposedly more difficult
 - relies on the logic of probability theory
- reliance on a *prior*
- reliance on computing and simulation
 - Were not able to just use SPSS
 - but emergence of JASP (Just Another Statistics Program) and other programs ...

**and we've already learned
how to simulate and
visualize data in this class!**



Quick recap: Bayesian data analysis

Clue guide to probability

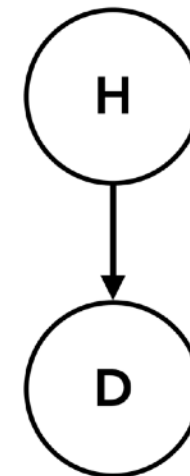
$$p(B|A) = \frac{p(A|B) \cdot p(B)}{p(A)}$$

we derived this using the definition of conditional probability

posterior

$$p(H|D) = \frac{p(D|H) \cdot p(H)}{p(D)}$$

subjective probability interpretation
 H = Hypothesis
 D = Data



formal framework for learning from data

updating our prior belief $p(H)$, to a posterior belief $p(H|D)$
 given some data

47

Summer camp

51

Clue guide to probability

what we know

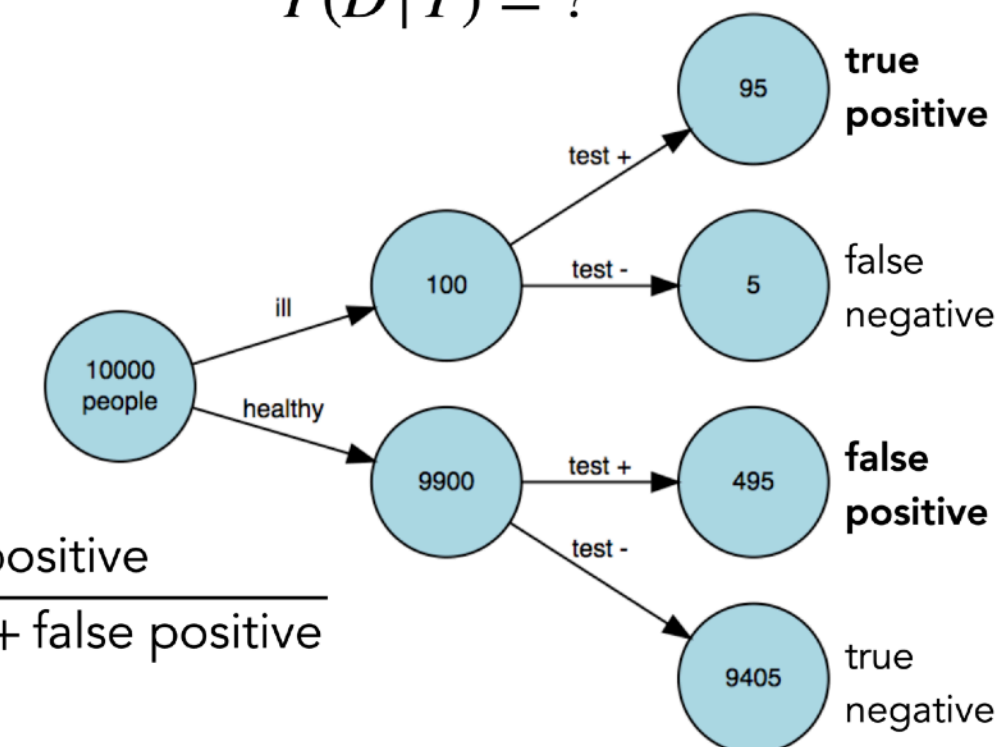
$$P(D) = 0.01$$

$$P(T|D) = 0.95$$

$$P(T|\neg D) = 0.05$$

what we want to know

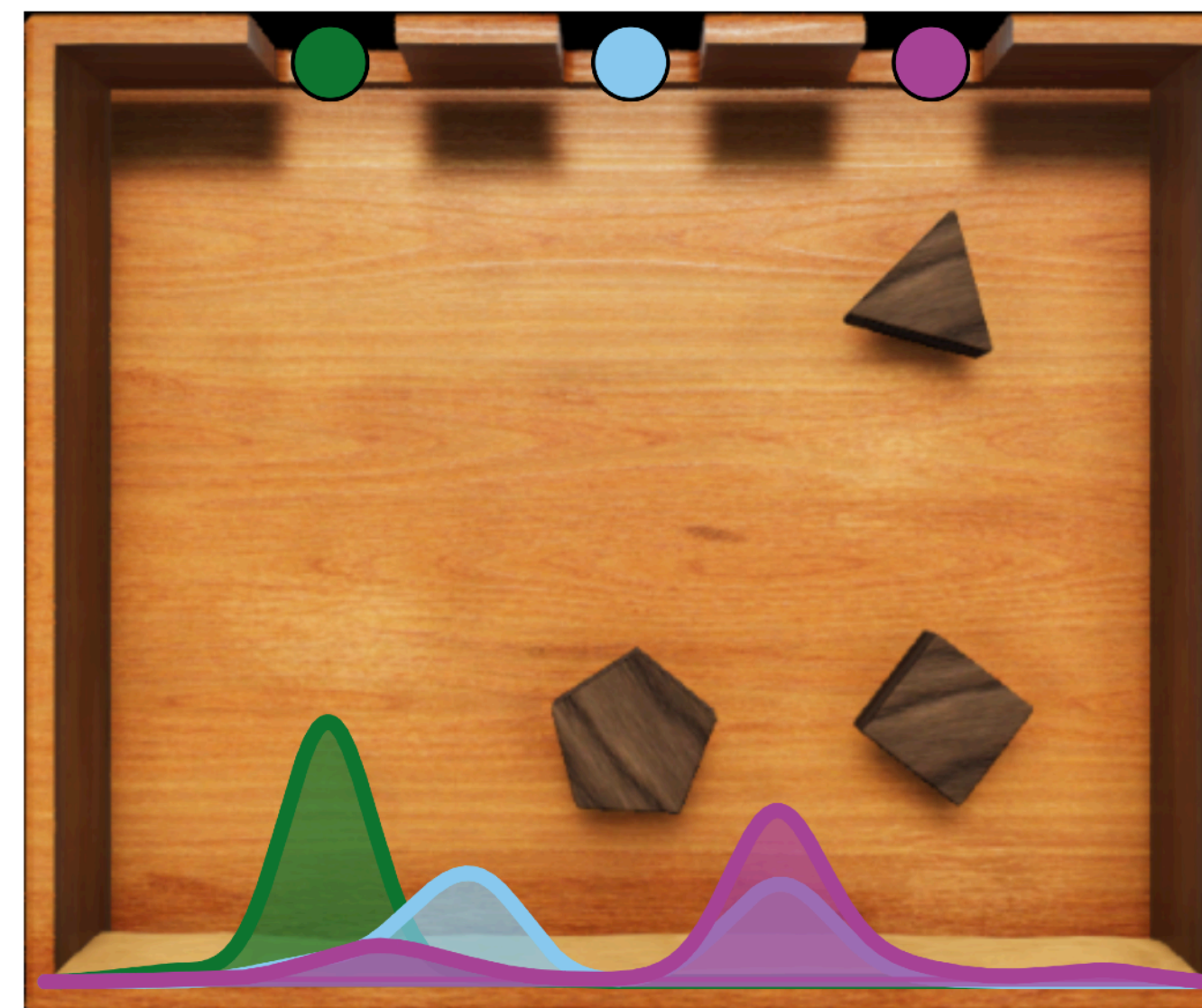
$$P(D|T) = ?$$



$$P(D|T) = \frac{\text{true positive}}{\text{true positive} + \text{false positive}}$$

$$= \frac{95}{95 + 495}$$

$$\approx 0.16$$



10

Quick recap: Bayesian data analysis

Coin flip example

```
1 # data
2 data = rep(0:1, c(8, 2)) ← 8 tails and 2 heads
3
4 # parameters
5 theta = c(0.1, 0.5, 0.9) ← hypotheses
6
7 # prior
8 prior = c(0.25, 0.5, 0.25) ← prior over the hypotheses
9
10 # likelihood
11 likelihood = dbinom(sum(data == 1), size = length(data), prob = theta)
12
13 # posterior
14 posterior = likelihood * prior / sum(likelihood * prior)
```

← binomial distribution

$$p(H|D) = \frac{p(D|H) \cdot p(H)}{p(D)}$$

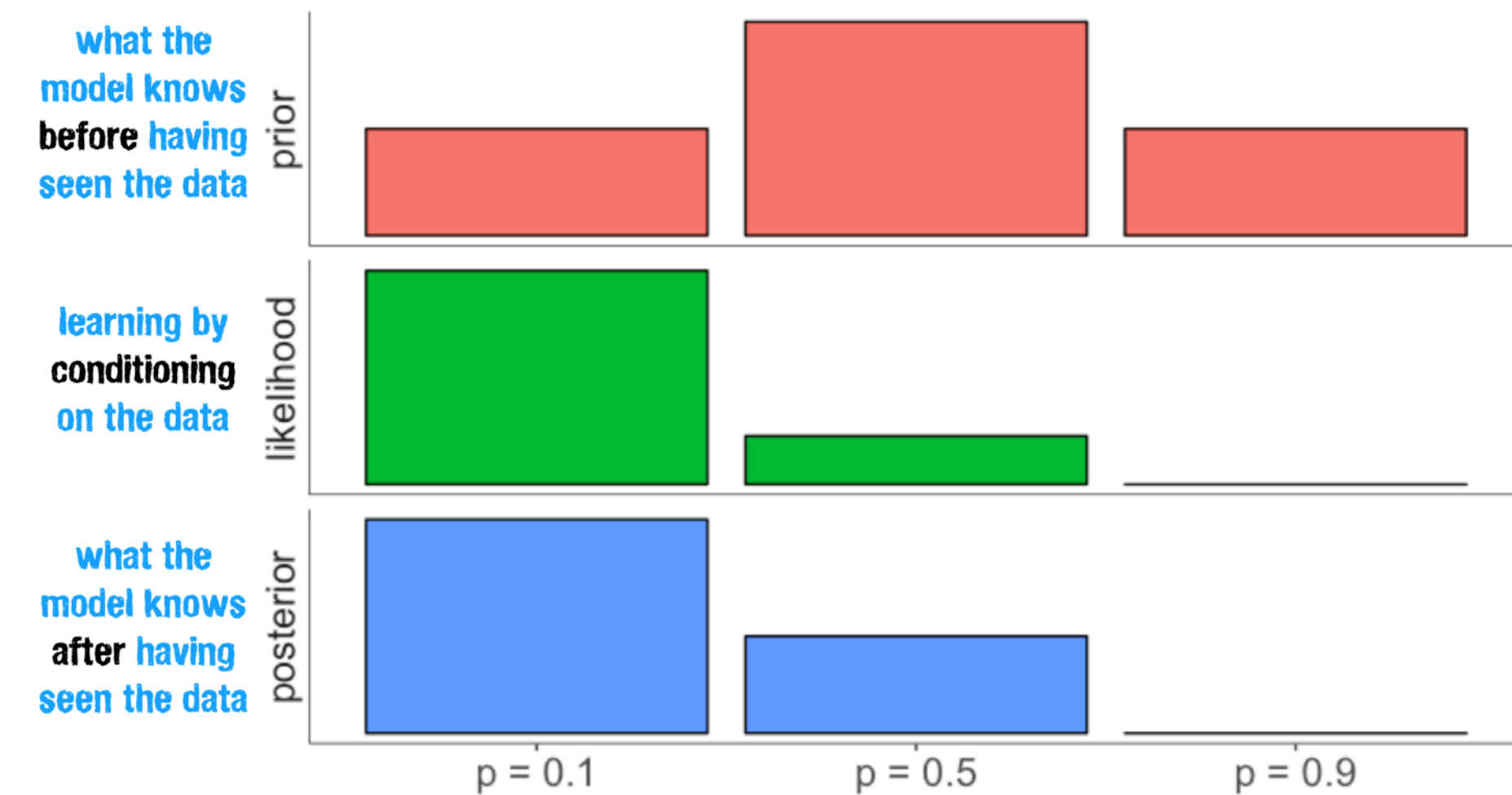
law of total probability:

$$p(D) = \sum_{i=1}^n p(D|H_i) \cdot p(H_i)$$

Coin flip example

data: #8 tails, #2 heads

Which coin was flipped?



posterior = multiplicative weighting of prior and likelihood

60

Quick recap: Bayesian data analysis

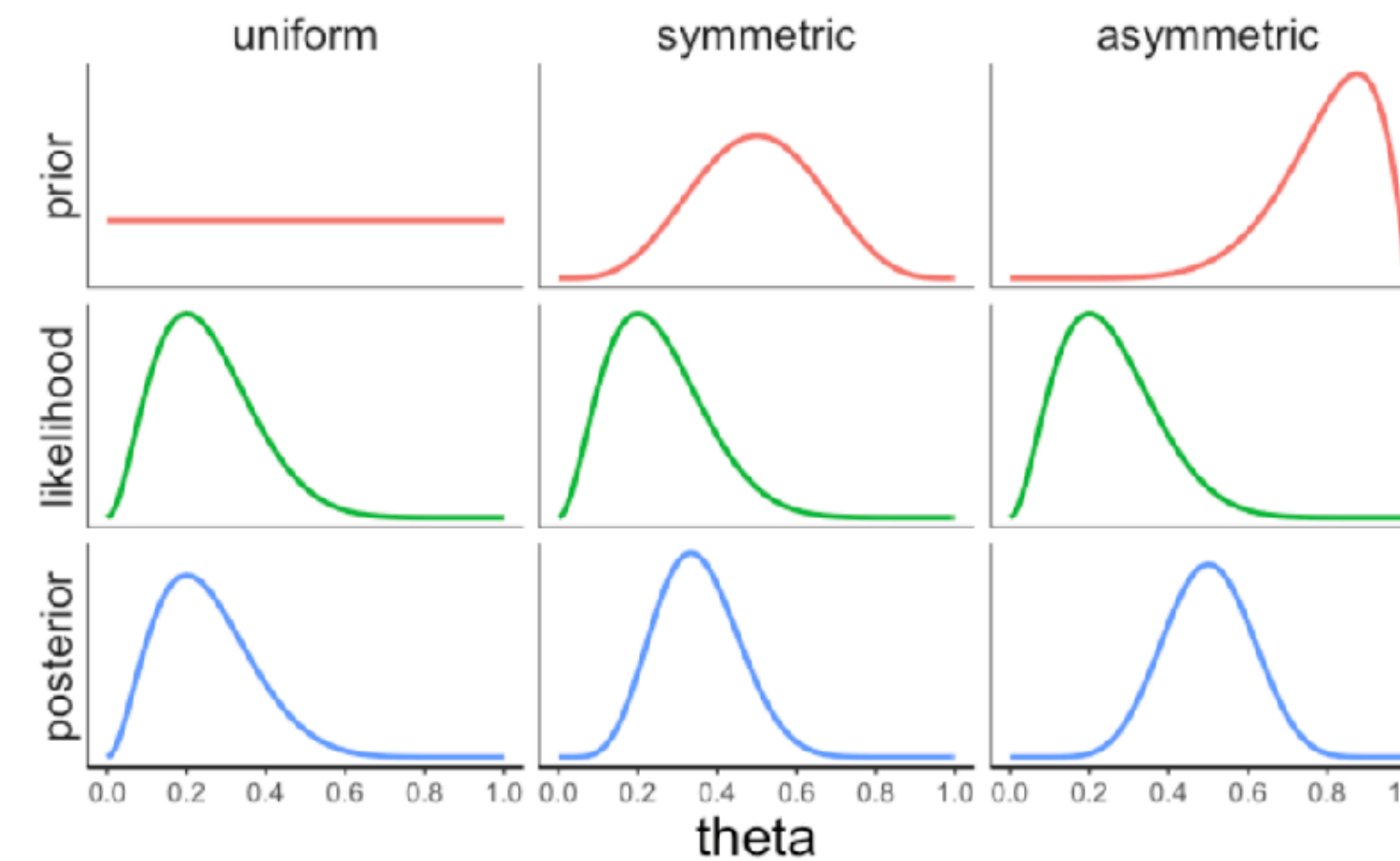
What affects the posterior?

1. the prior over hypotheses
2. the likelihood of the data given each hypothesis

$$\text{Posterior } p(H|D) = \frac{\text{Likelihood } p(D|H) \cdot \text{Prior } p(H)}{\text{Normalizing constant } p(D)}$$

The effect of the **prior**

same data, different priors

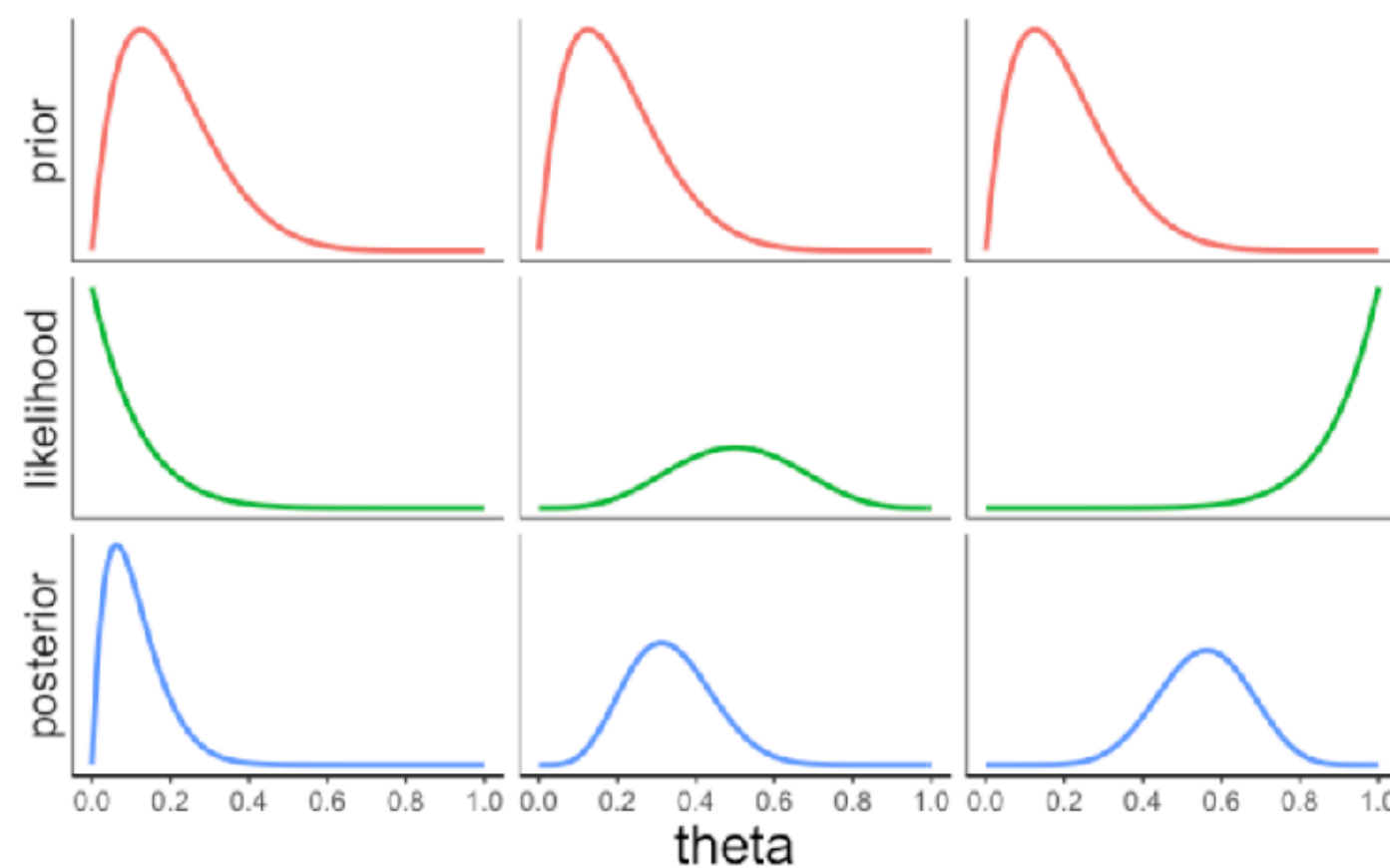


45

46

The effect of the **likelihood**

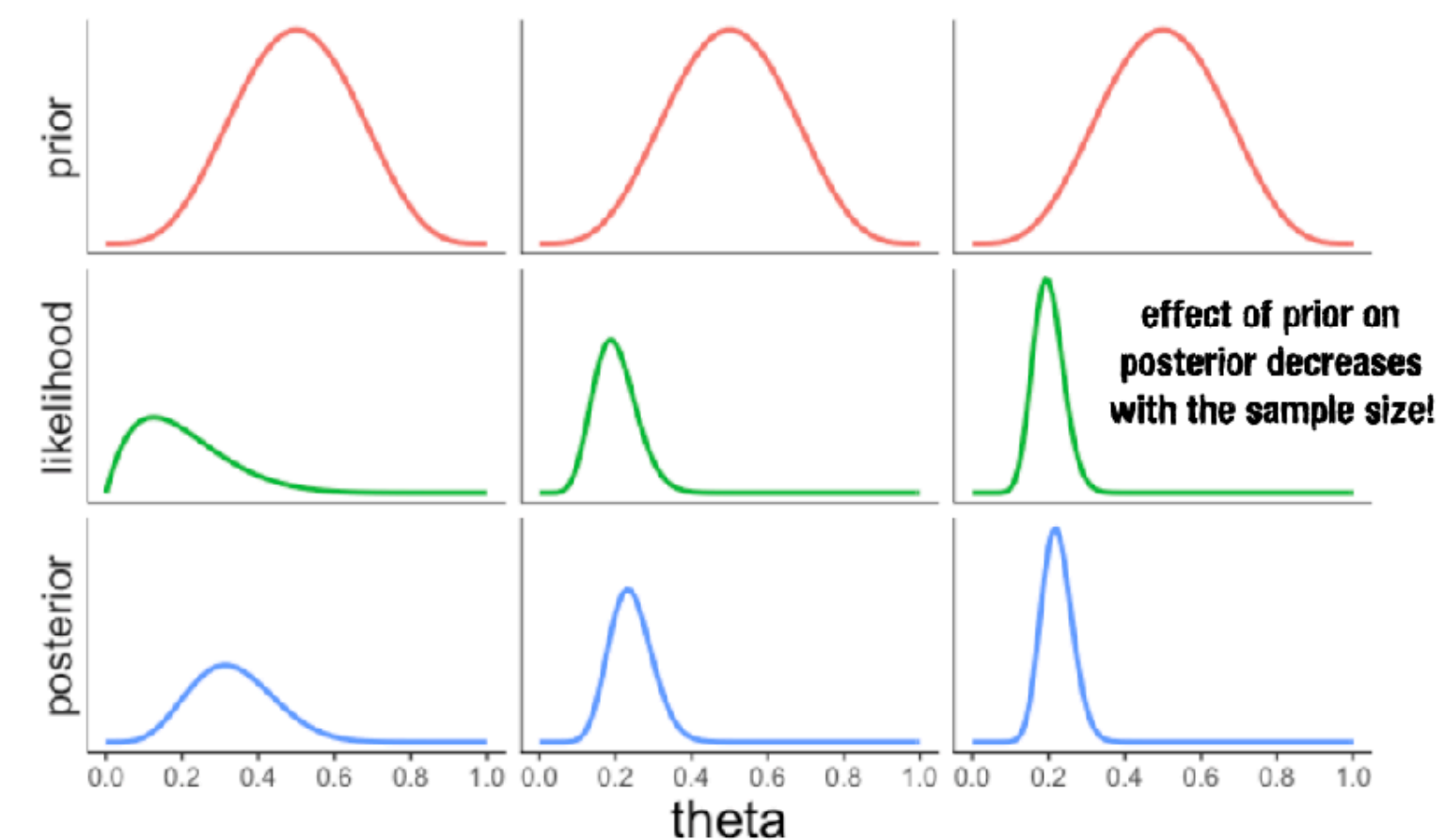
same prior, different data



47

The effect of **sample size**

n = low n = medium n = high



48

Ingredients: likelihood, prior, inference

Ingredients

$$\begin{array}{c} \text{Posterior} \\ p(H | D) \end{array} = \frac{\begin{array}{c} \text{Likelihood} \\ p(D | H) \end{array} \cdot \begin{array}{c} \text{Prior} \\ p(H) \end{array}}{\begin{array}{c} p(D) \\ \text{Normalizing constant} \end{array}}$$

$$\begin{array}{c} \text{Posterior} \\ p(H | D) \end{array} = \frac{\begin{array}{c} \text{Likelihood} \\ p(D | H) \end{array} \cdot \begin{array}{c} \text{Prior} \\ p(H) \end{array}}{\begin{array}{c} p(D) \\ \text{Normalizing constant} \end{array}}$$

Likelihood

- **What probabilistic model describes best how the data were generated?**
 - What assumptions can you make about the data?
 - What's the nature of your dependent variable (e.g. binary, ordered, continuous)?
 - Does the model re-create the behavior of interest?

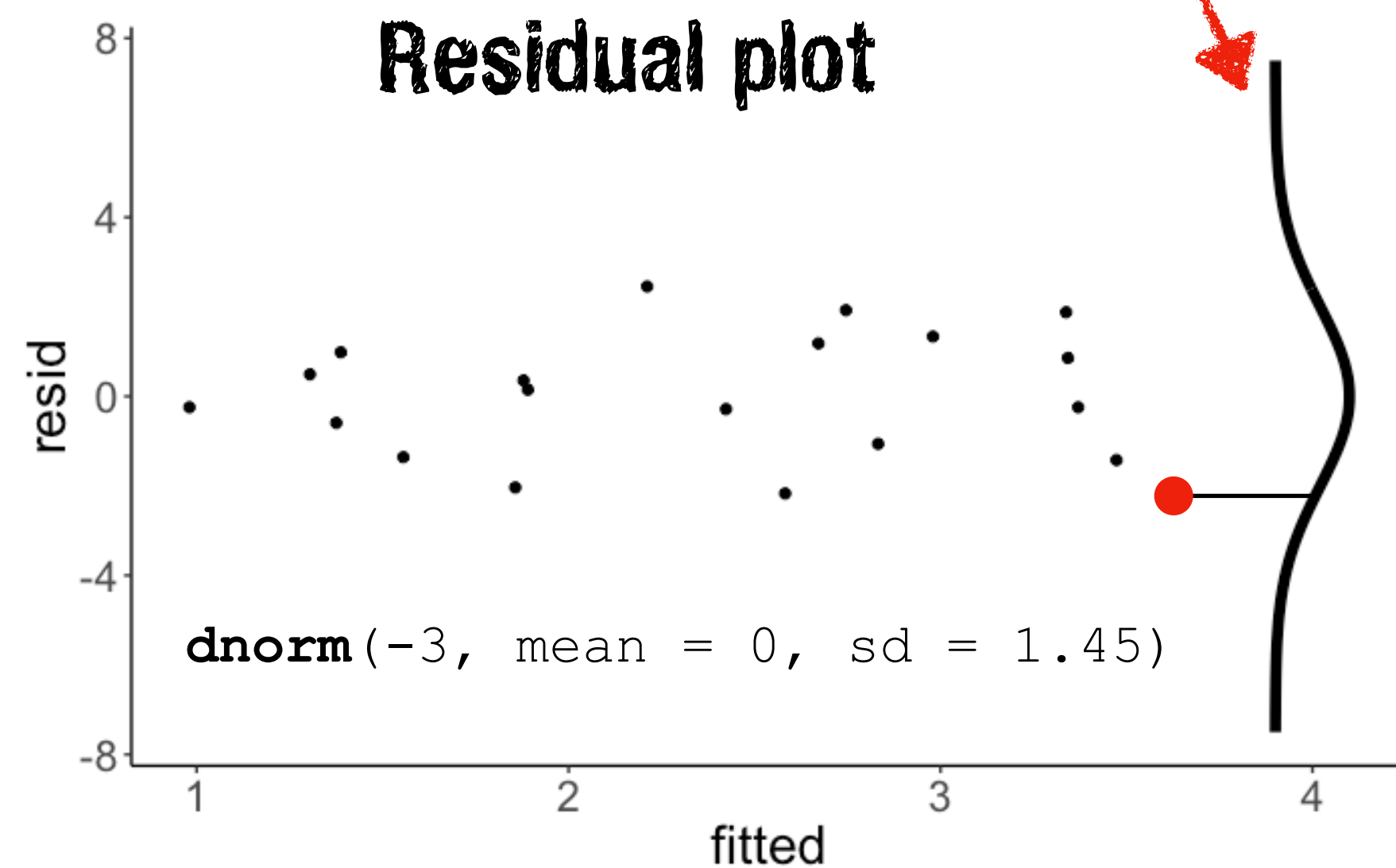


Likelihood

Gaussian distribution

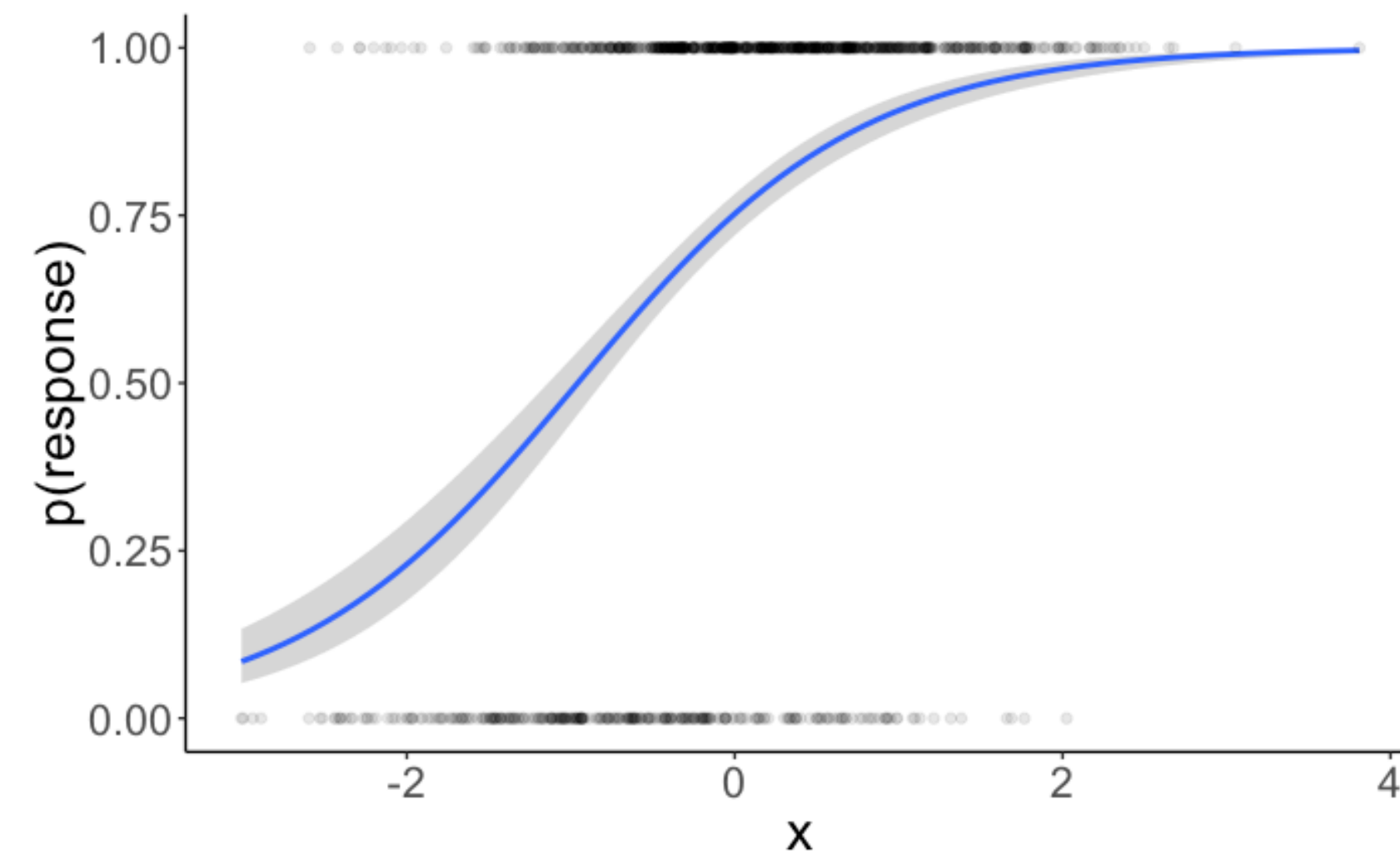
$$Y_i = b_0 + b_1 \cdot x_i + e_i$$

$$e_i \sim \mathcal{N}(0, \sigma)$$



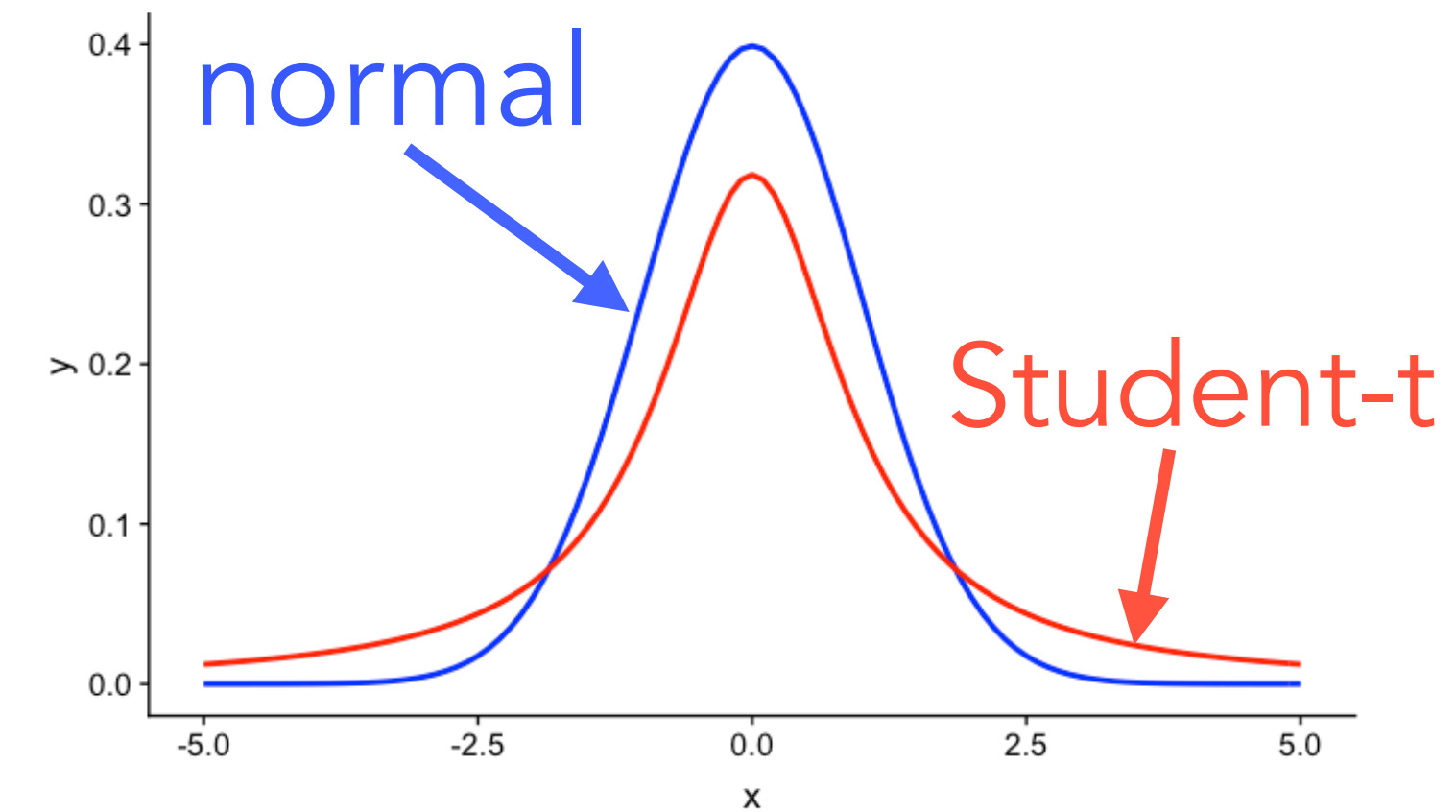
Binomial distribution

```
1 fit.glm = glm(formula = survived ~ 1 + fare,  
2               family = "binomial",  
3               data = df.titanic)
```



Likelihood

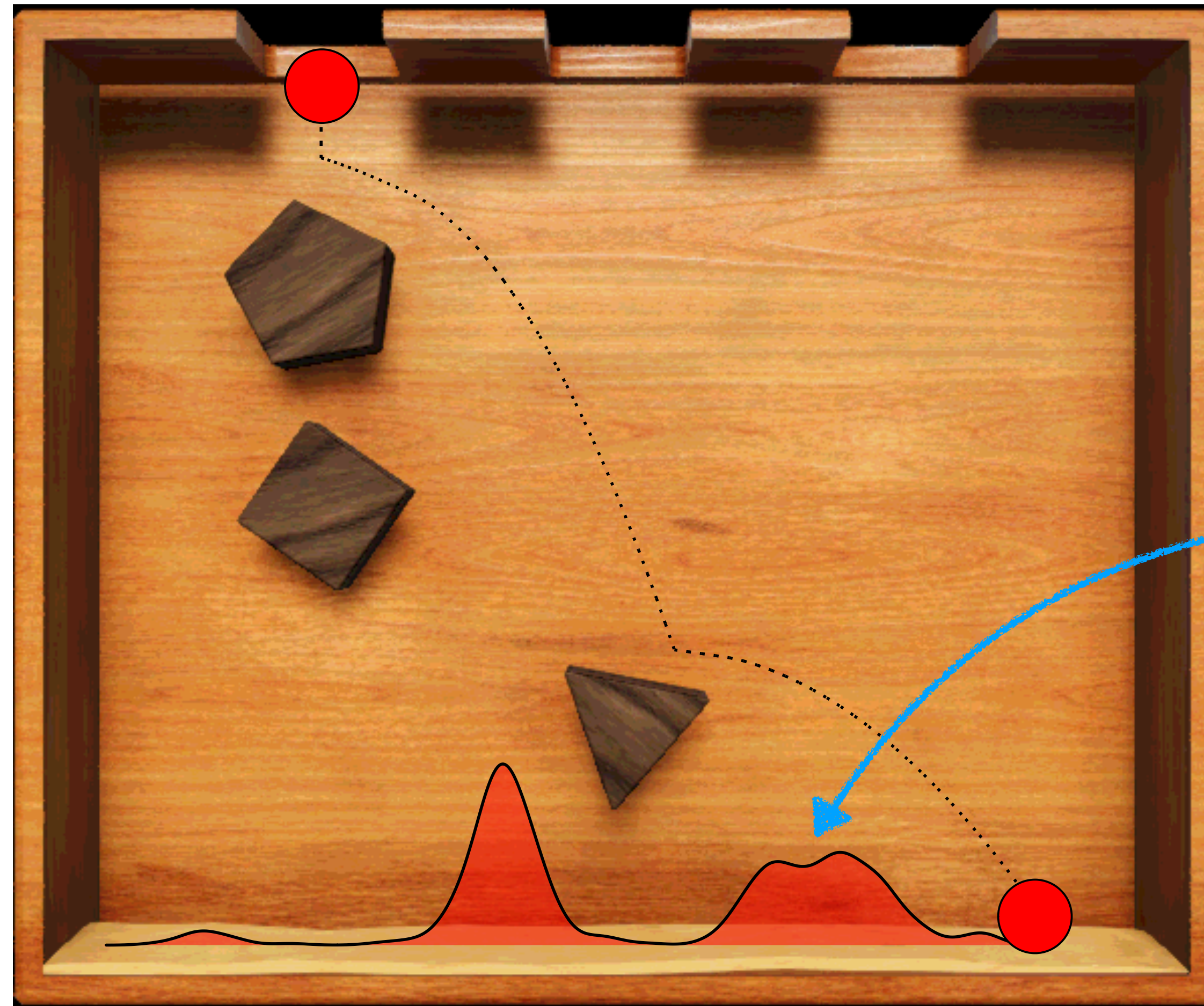
- **Bernoulli:**
 - binary data
 - a single trial
- **Poisson:** count of discrete events
- **Beta-binomial:** like binomial but probability of success may change across trials
- **Student-t:**
 - same as Normal
 - handles greater variability in the data (distribution has **fat tails**)
- ...







Prediction: Where will the ball land?



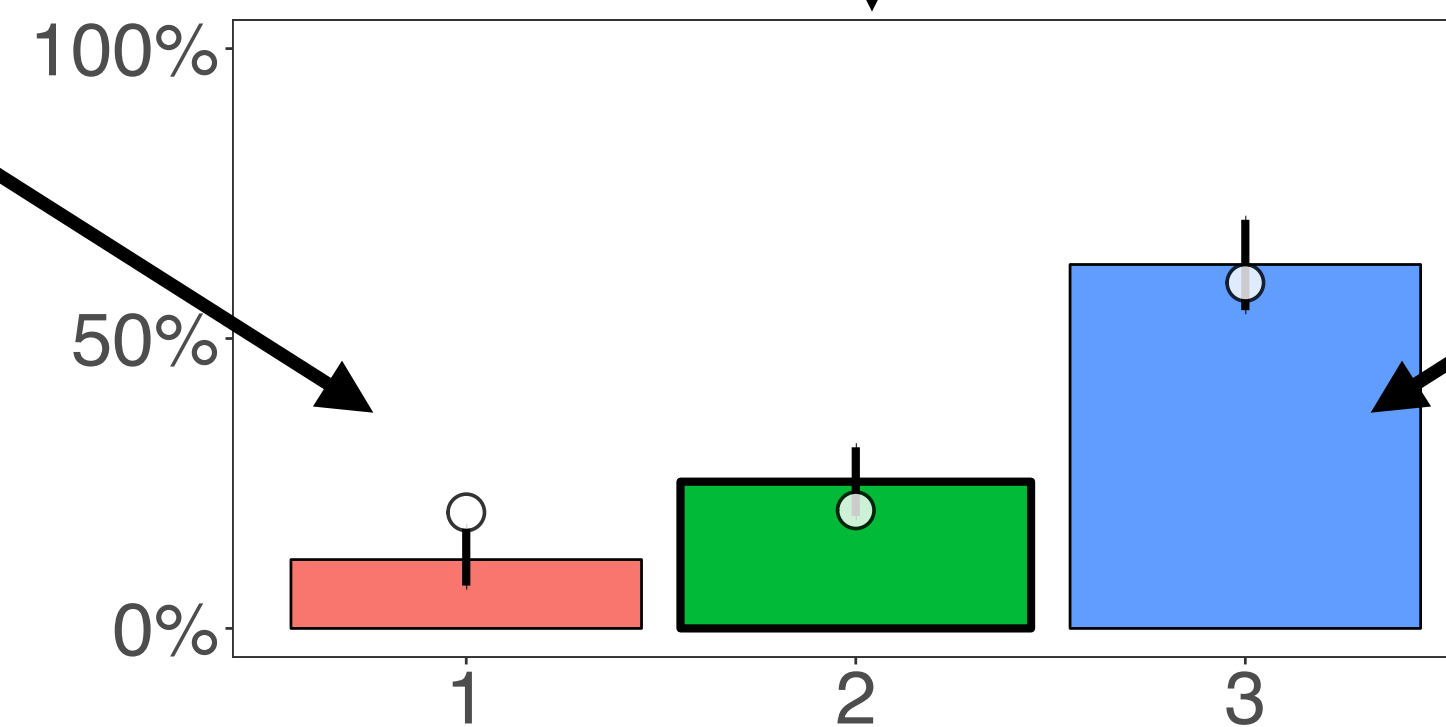
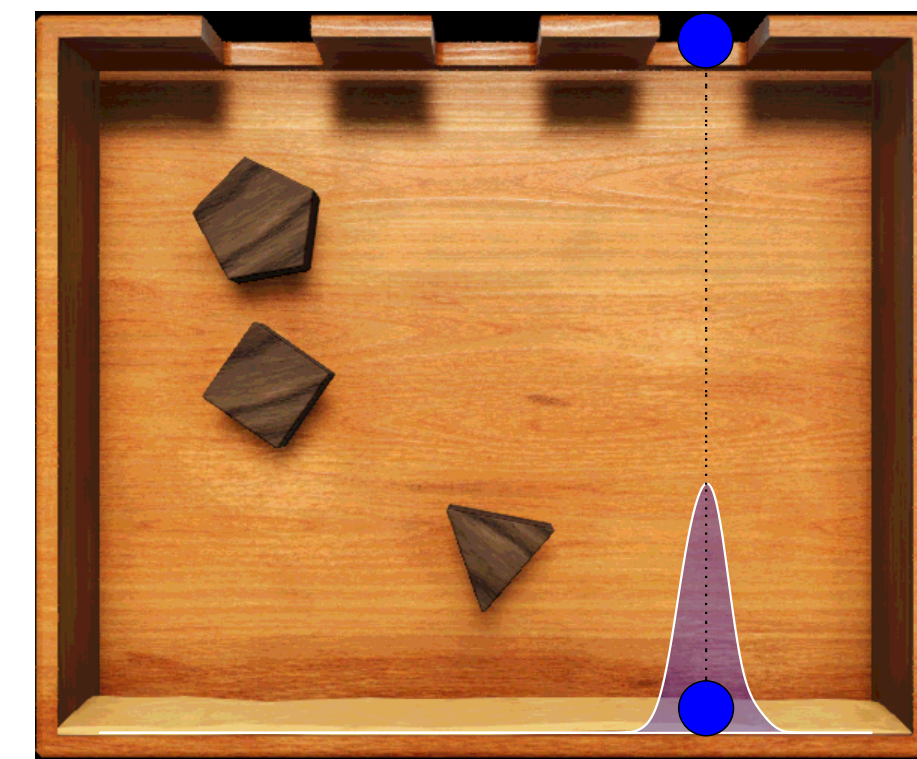
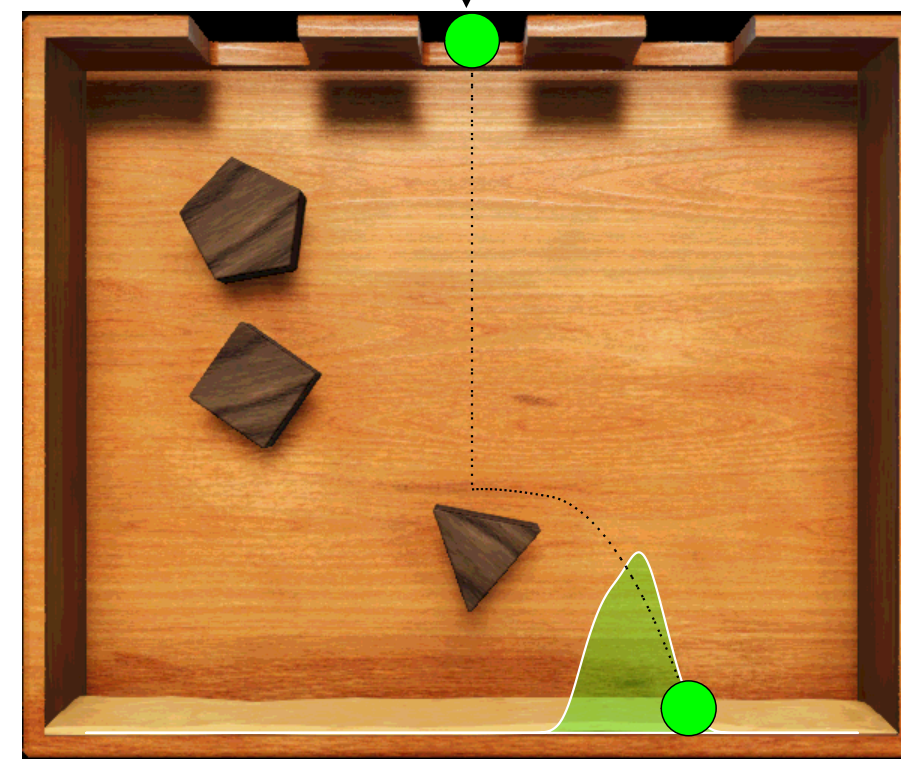
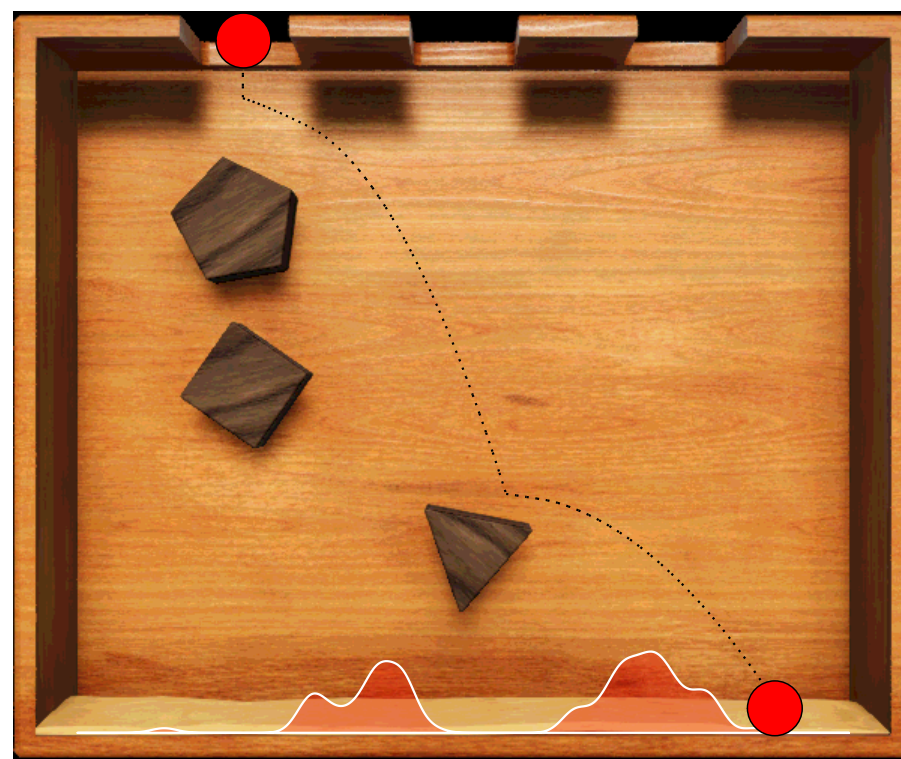
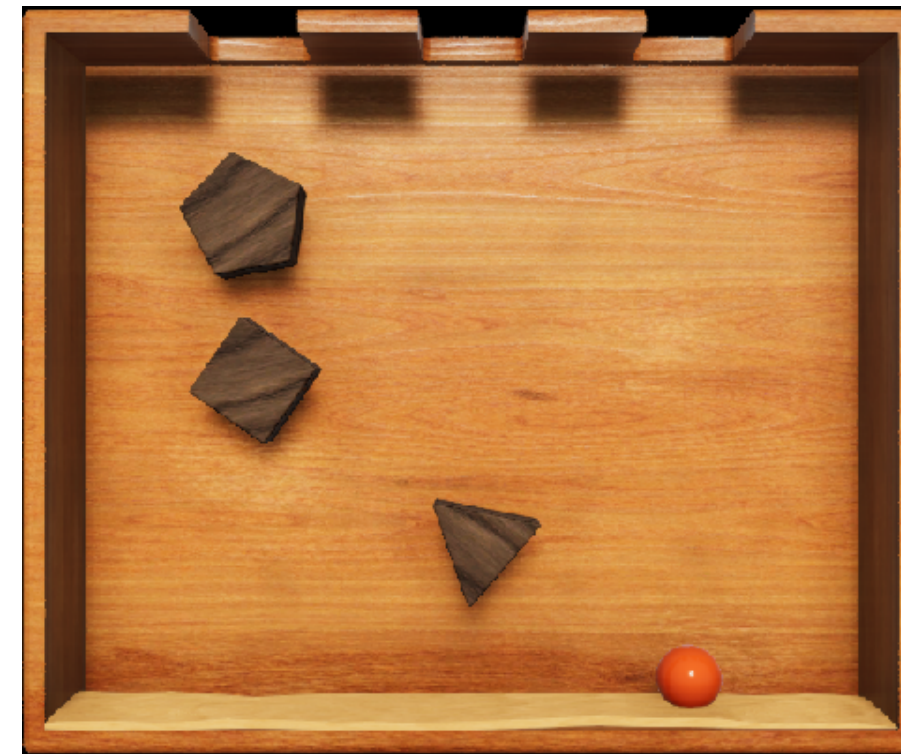
likelihood function

Aggregated responses

Inference: In which hole was the ball dropped?

distance between ball's true x position and x position in sample

$$\exp\left(-\frac{d(\text{ball_x}_{\text{final}}, \text{ball_x}_{\text{hole}})}{2\sigma^2}\right)$$



□ data

○ model prediction

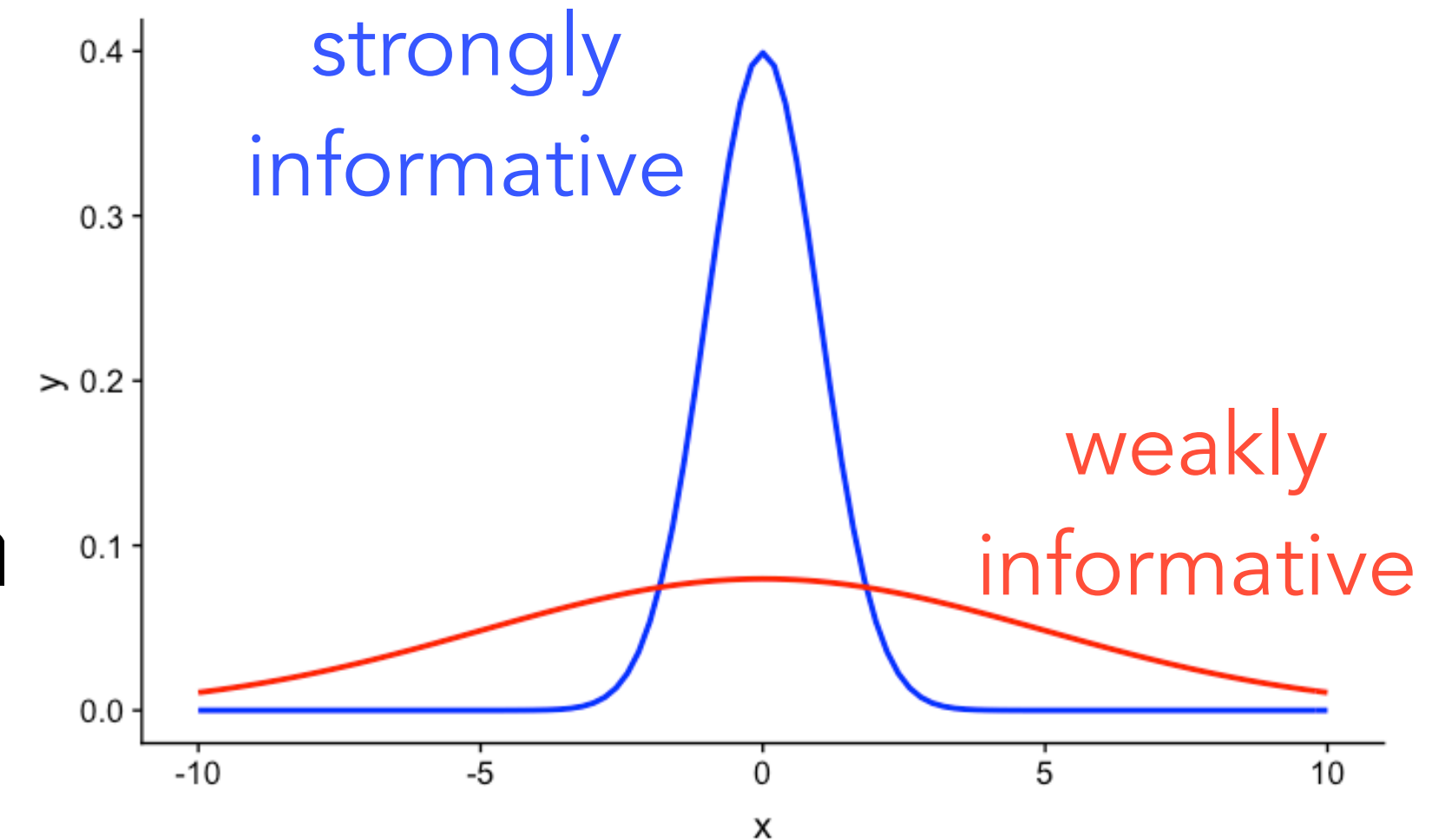
Prior

$$\begin{array}{c} \text{Posterior} \\ p(H | D) \end{array} = \frac{\begin{array}{c} \text{Likelihood} \\ p(D | H) \end{array} \cdot \begin{array}{c} \text{Prior} \\ p(H) \end{array}}{\begin{array}{c} p(D) \\ \text{Normalizing constant} \end{array}}$$

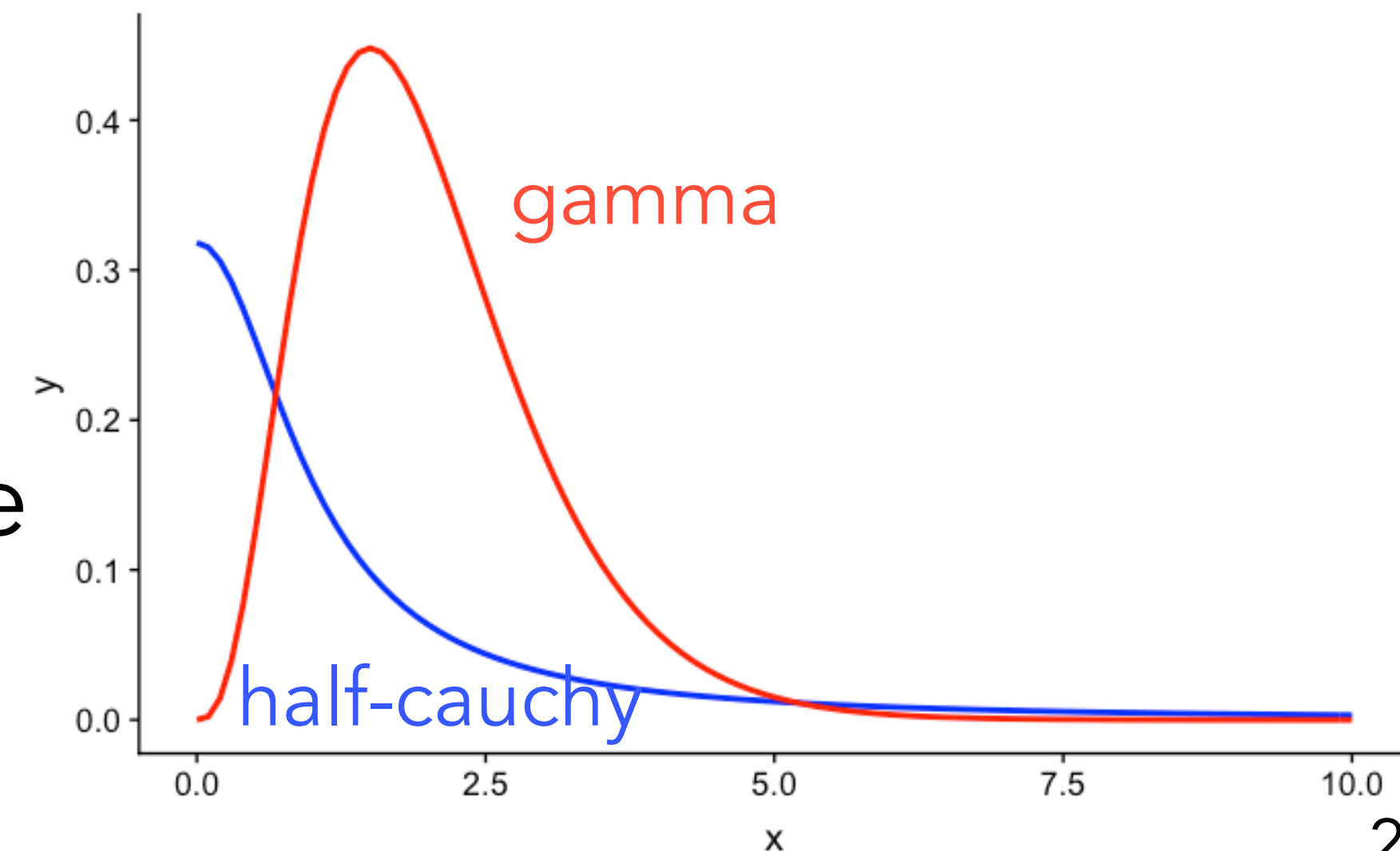
Prior

- **uniform:**
 - continuous or discrete
 - bounded between minimum and maximum
- **gaussian:**
 - sd determines how informative the prior is
- **gamma, half-cauchy:**
 - for parameters we know are positive

for **beta coefficients** in a regression

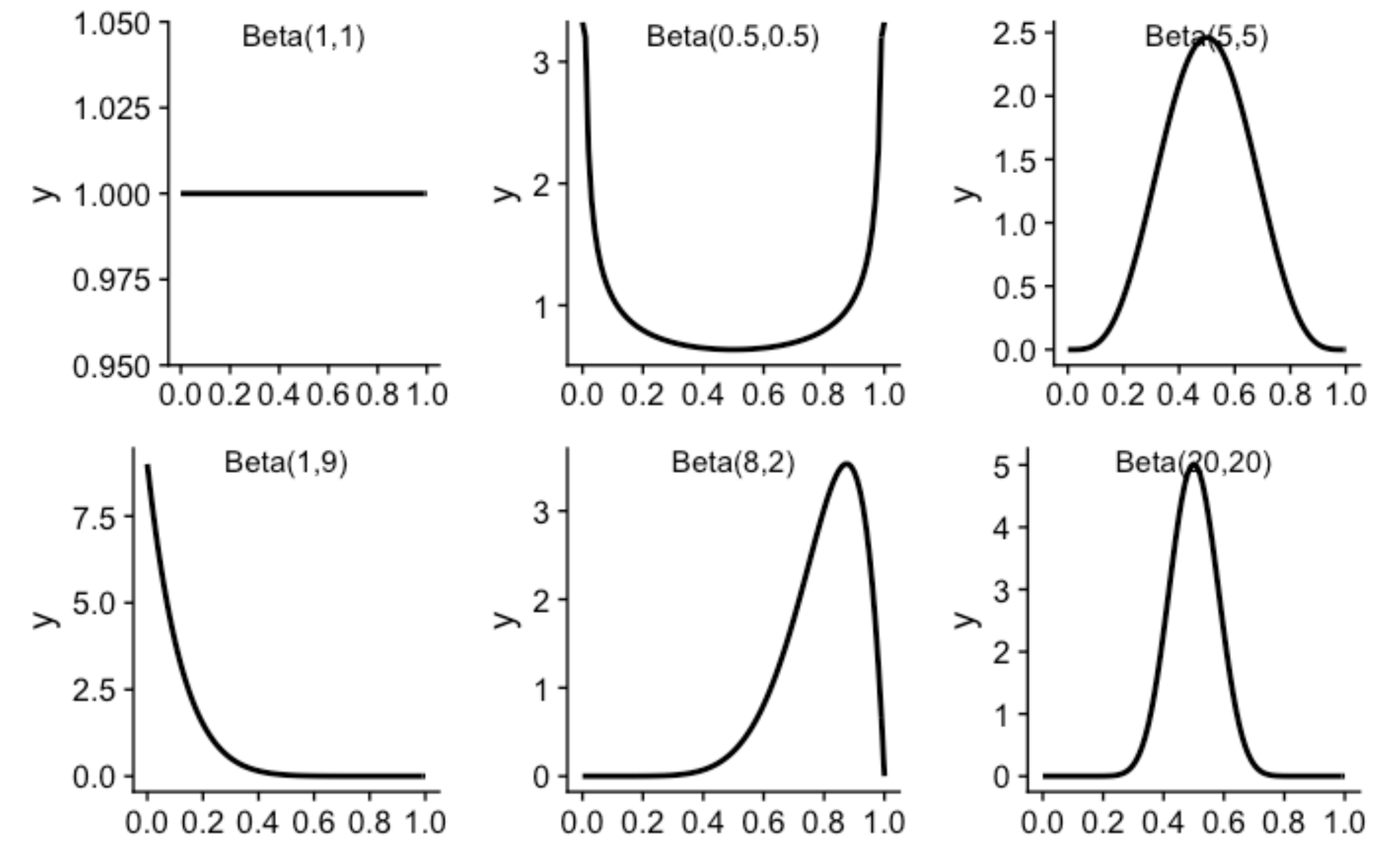


for **standard deviation** of the Gaussian



Prior

- **beta:**
 - continuous parameters
 - bounded between 0 and 1
 - can model a wide range of priors



Inference

$$\begin{array}{c} \text{Posterior} \\ p(H | D) \end{array} = \frac{\begin{array}{c} \text{Likelihood} \\ p(D | H) \end{array} \cdot \begin{array}{c} \text{Prior} \\ p(H) \end{array}}{p(D)}$$

Normalizing constant

the devil is in the denominator ...

Doing Bayesian inference

Discrete hypothesis space

$$p(H | D) = \frac{p(D | H) \cdot p(H)}{\sum_{i=1}^n p(D | H_i) \cdot p(H_i)}$$

sum over all possibilities

Continuous hypothesis space

$$p(H | D) = \frac{p(D | H) \cdot p(H)}{\int_{-\infty}^{\infty} p(D | H_i) \cdot p(H_i) dH_i}$$

integral over all possibilities

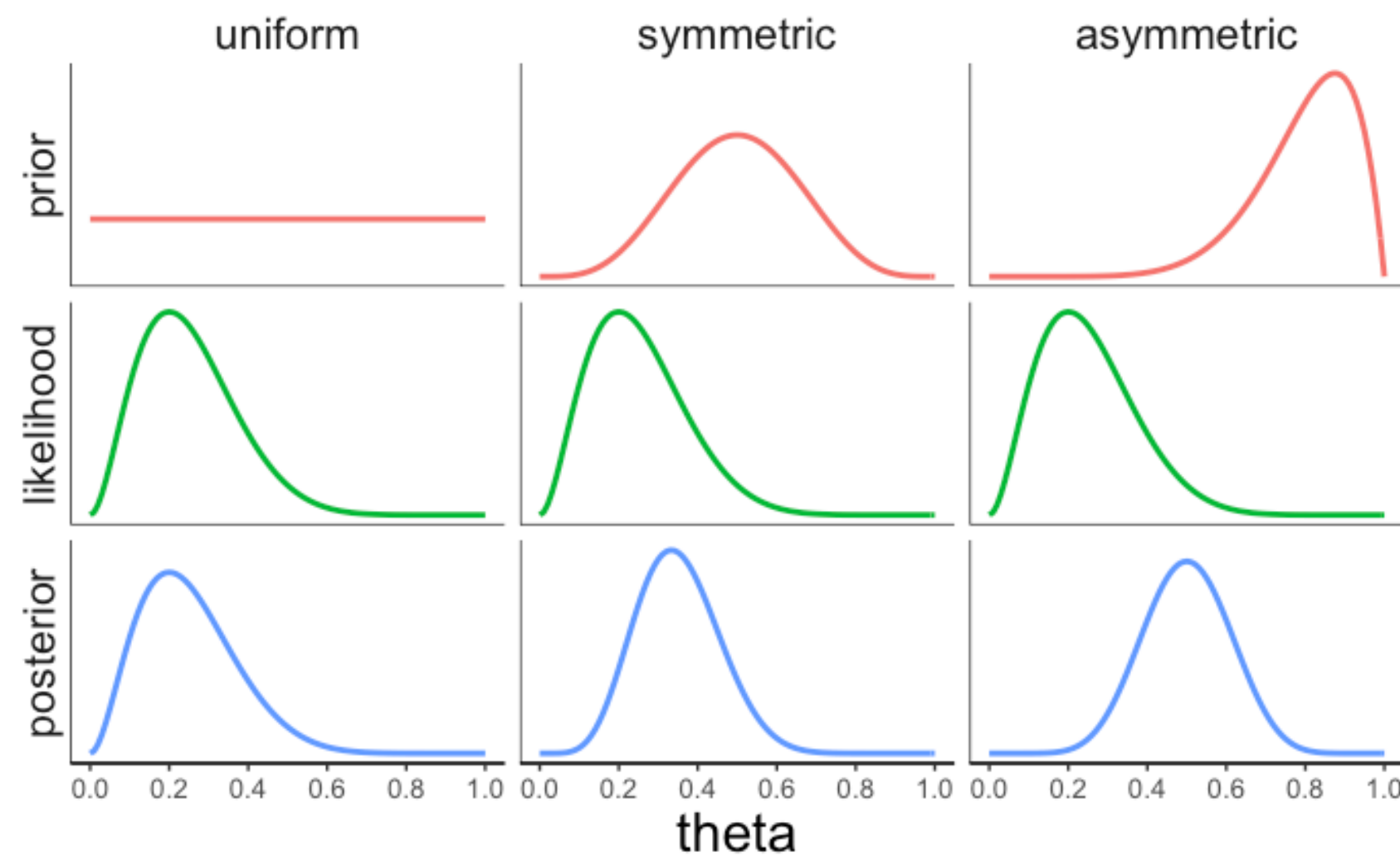
Discretizing the parameters

```
1 # grid
2 theta = seq(0, 1, 0.01)
3
4 # data
5 data = rep(0:1, c(8, 2))
6
7 # calculate posterior
8 df.prior = tibble(theta = theta,
9                   prior_uniform = dbeta(theta, shape1 = 1, shape2 = 1),
10                  prior_normal = dbeta(theta, shape1 = 5, shape2 = 5),
11                  prior_biased = dbeta(theta, shape1 = 8, shape2 = 2)) %>%
12   pivot_longer(cols = -theta,
13               names_to = "prior_index",
14               values_to = "prior") %>%
15   mutate(likelihood = dbinom(sum(data == 1),
16                             size = length(data),
17                             prob = theta)) %>%
18   group_by(prior_index) %>%
19   mutate(posterior = likelihood * prior / sum(likelihood * prior)) %>%
20   ungroup() %>%
21   pivot_longer(cols = -c(theta, prior_index),
22               names_to = "index",
23               values_to = "value")
```

← 100 discrete values

for 3 variables, we would already
need 1 Mio combinations

The CURSE of
dimensionality



Inference via sampling

- we cannot directly calculate the probability of the posterior (because it might have a pretty weird shape)
- **but:** we can draw random samples from the posterior
- we can then use our data wrangling and visualization skills to make inferences based on these samples

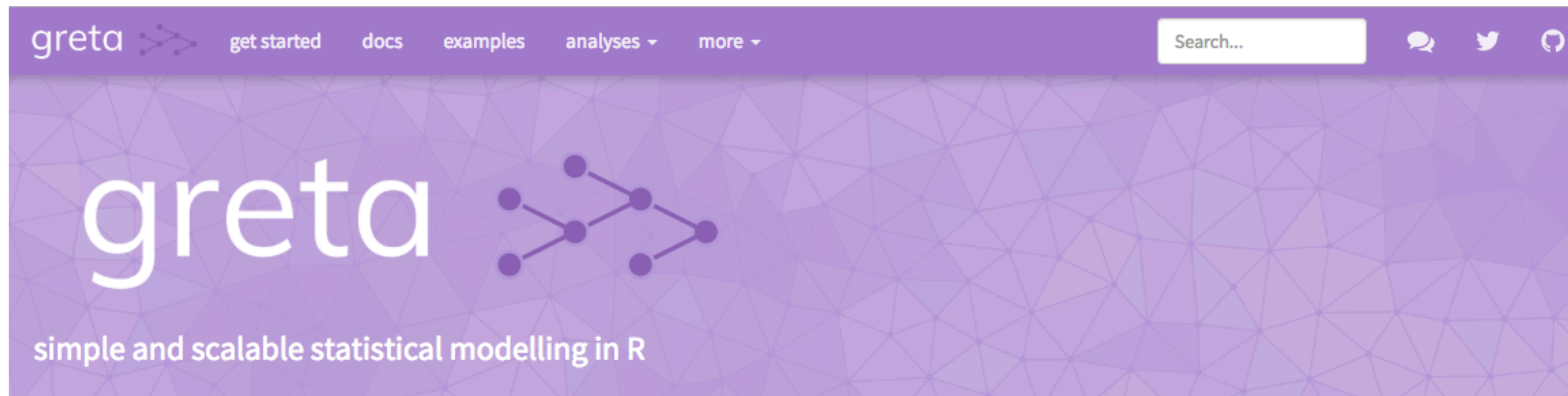
It's as if ...

we don't have **pnorm** () but we do have **rnorm** ()

Inference via sampling

- Bayesian data analysis is becoming more popular because:
 - computers are getting more powerful
 - inference techniques are getting better
 - software packages become easier to use

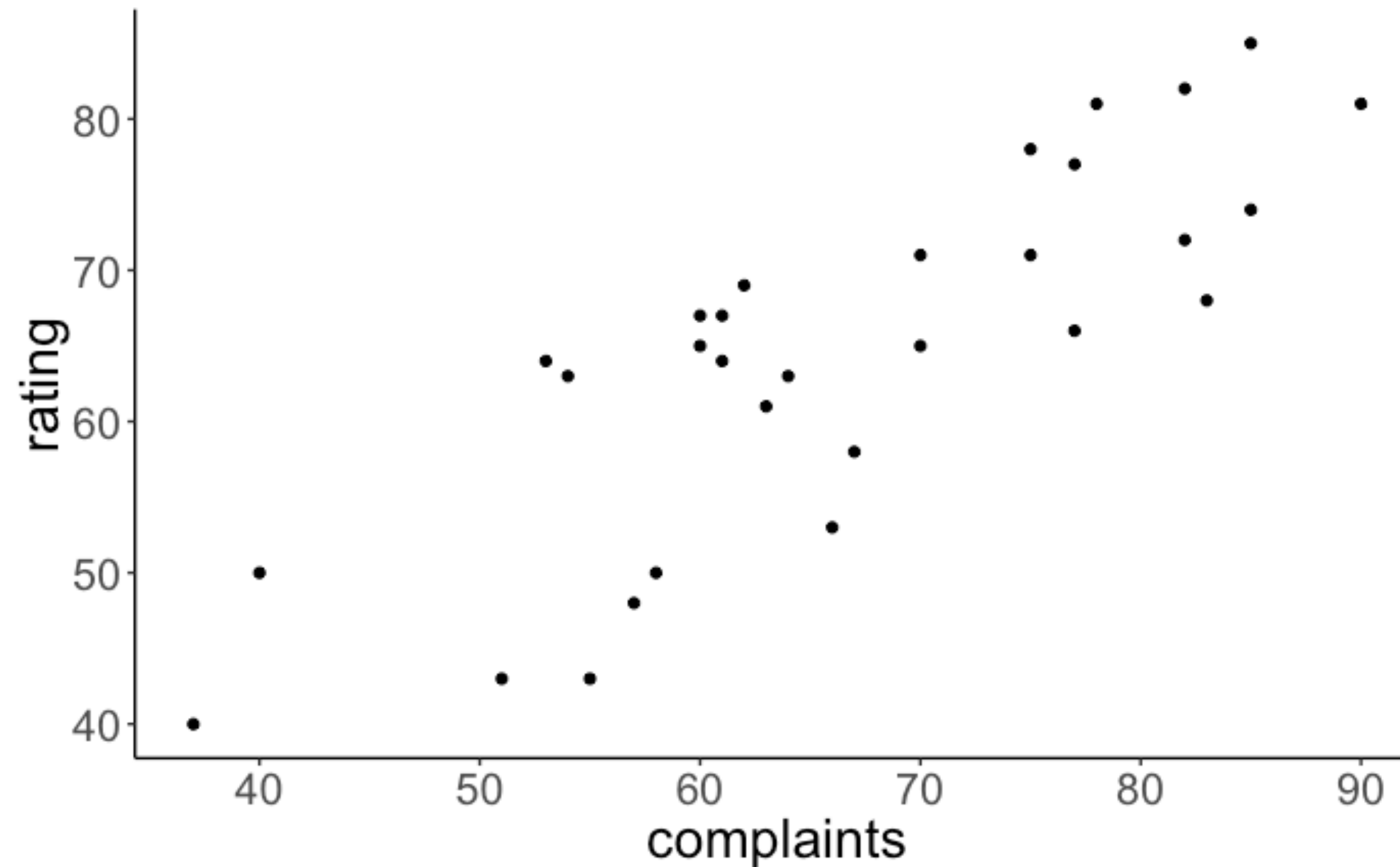
Doing Bayesian data analysis with `greta`



- let's us write Bayesian models directly in R with a simple syntax
- uses Tensorflow to implement Hamiltonian Monte Carlo sampling (a fast inference algorithm ...)

Attitude data set

What's the relationship between how well an employee handles complaints and their overall rating?



Frequentist analysis

Frequentist analysis

```
1 # fit model
2 fit = lm(formula = rating ~ 1 + complaints,
3           data = df.attitude)
4
5 # print summary
6 fit %>% summary()
```

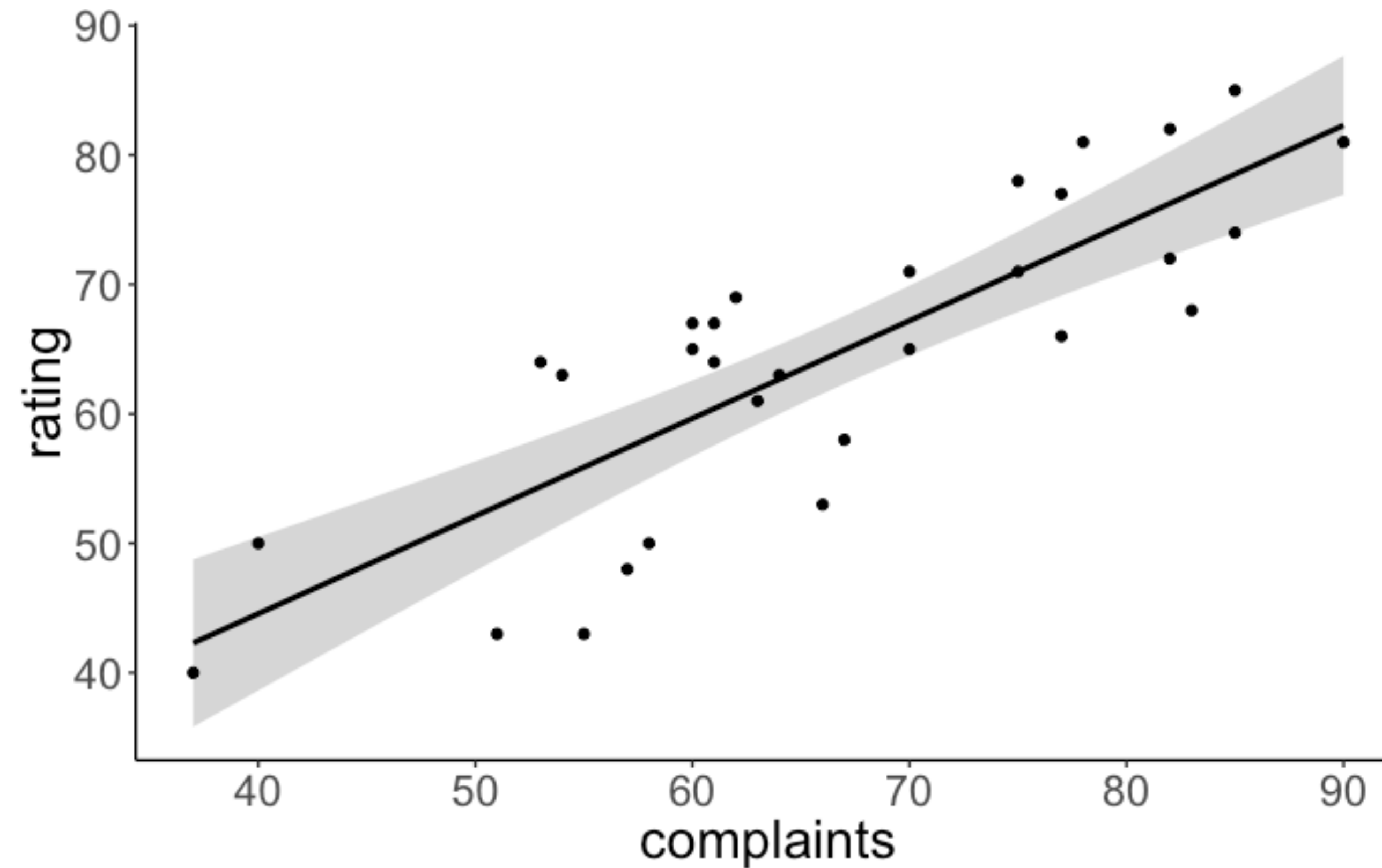
```
Call:
lm(formula = rating ~ 1 + complaints, data = df.attitude)

Residuals:
    Min       1Q   Median       3Q      Max
-12.8799  -5.9905   0.1783   6.2978   9.6294

Coefficients:
              Estimate Std. Error t value Pr(>|t|)
(Intercept)  14.37632    6.61999   2.172   0.0385 *
complaints    0.75461    0.09753   7.737 1.99e-08 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 6.993 on 28 degrees of freedom
Multiple R-squared:  0.6813,    Adjusted R-squared:  0.6699
F-statistic: 59.86 on 1 and 28 DF,  p-value: 1.988e-08
```


Visualize model predictions



Best-fitting regression line with confidence interval

Bayesian analysis

Model specification

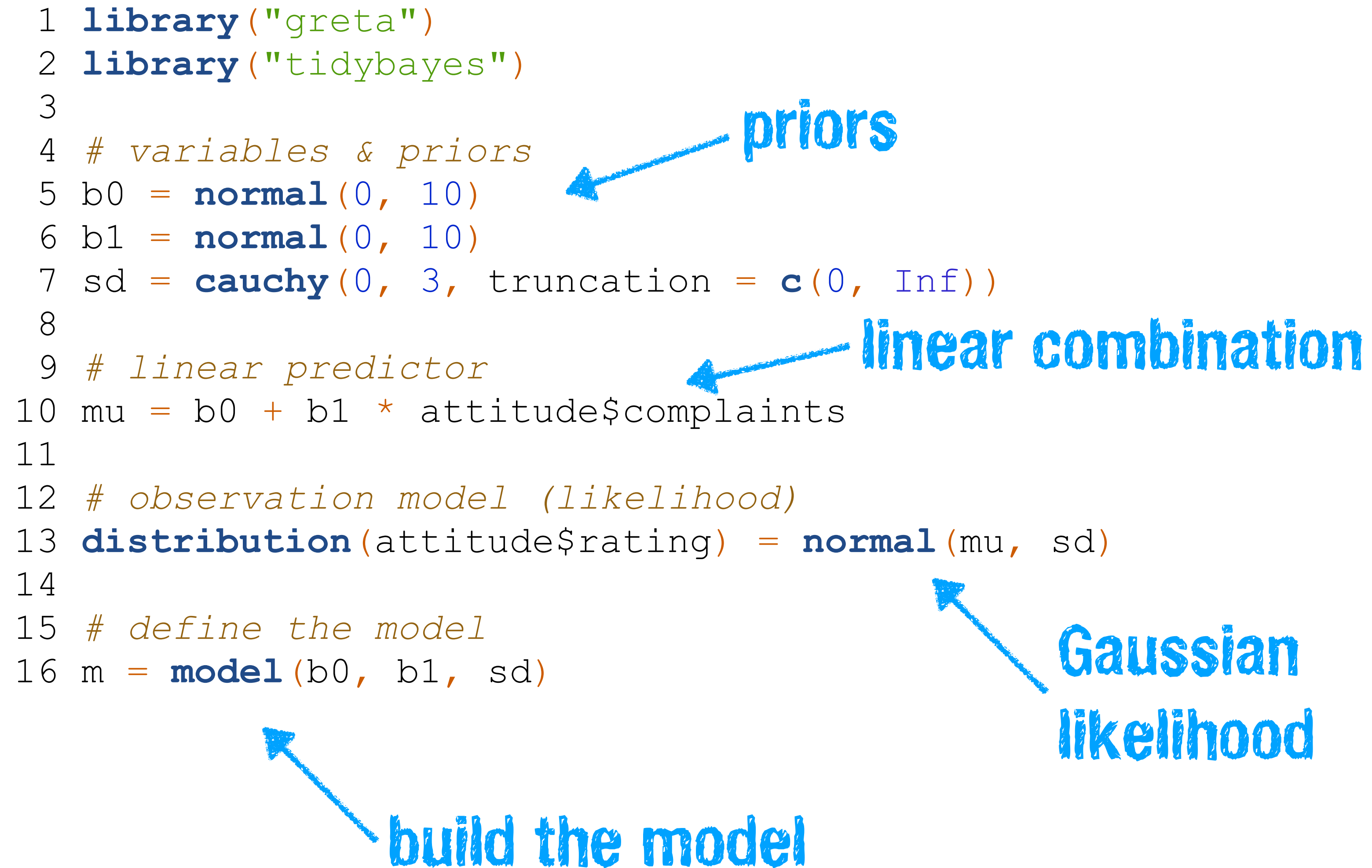
```
1 library("greta")
2 library("tidybayes")
3
4 # variables & priors
5 b0 = normal(0, 10)
6 b1 = normal(0, 10)
7 sd = cauchy(0, 3, truncation = c(0, Inf))
8
9 # linear predictor
10 mu = b0 + b1 * attitude$complaints
11
12 # observation model (likelihood)
13 distribution(attitude$rating) = normal(mu, sd)
14
15 # define the model
16 m = model(b0, b1, sd)
```

priors

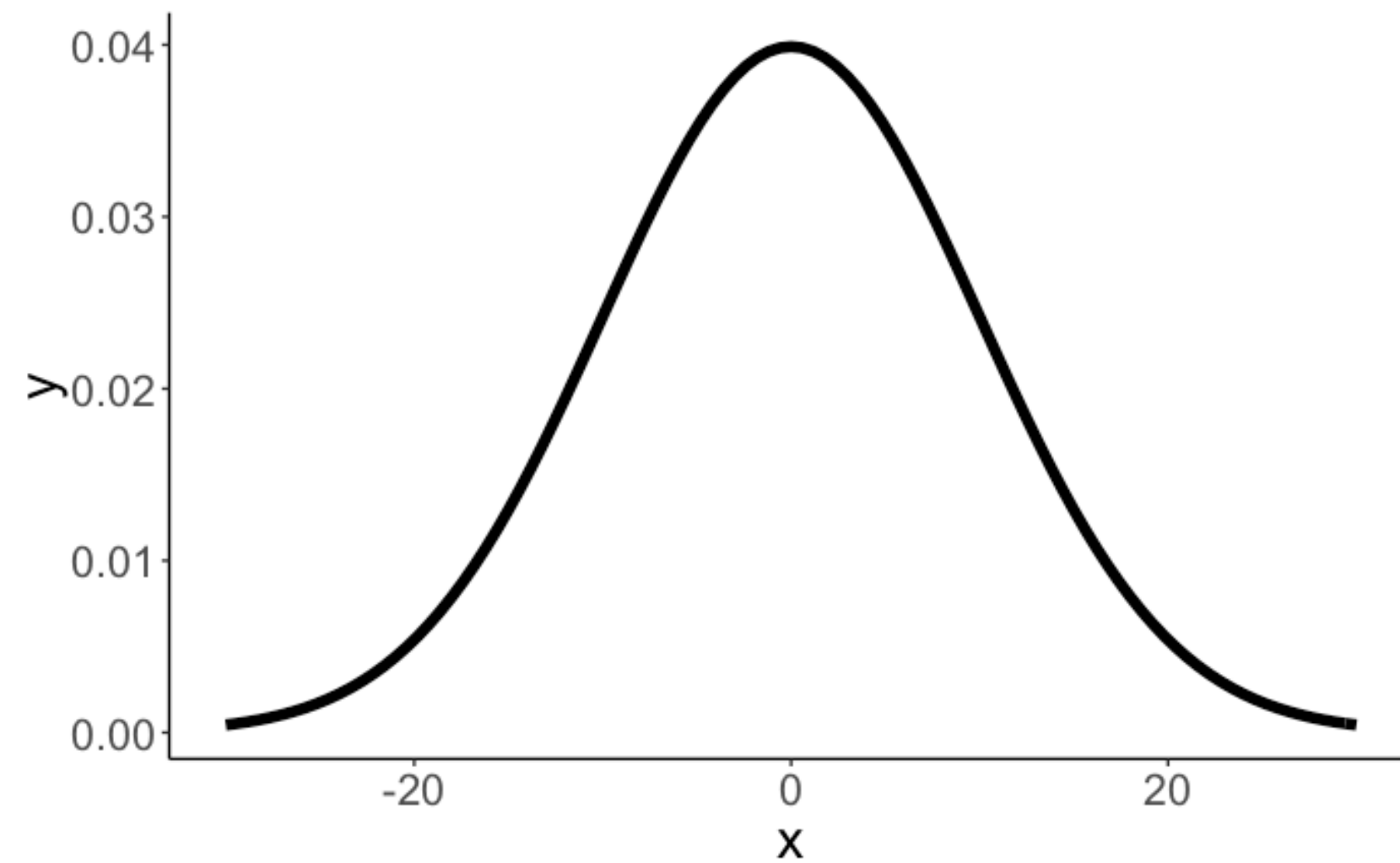
linear combination

Gaussian likelihood

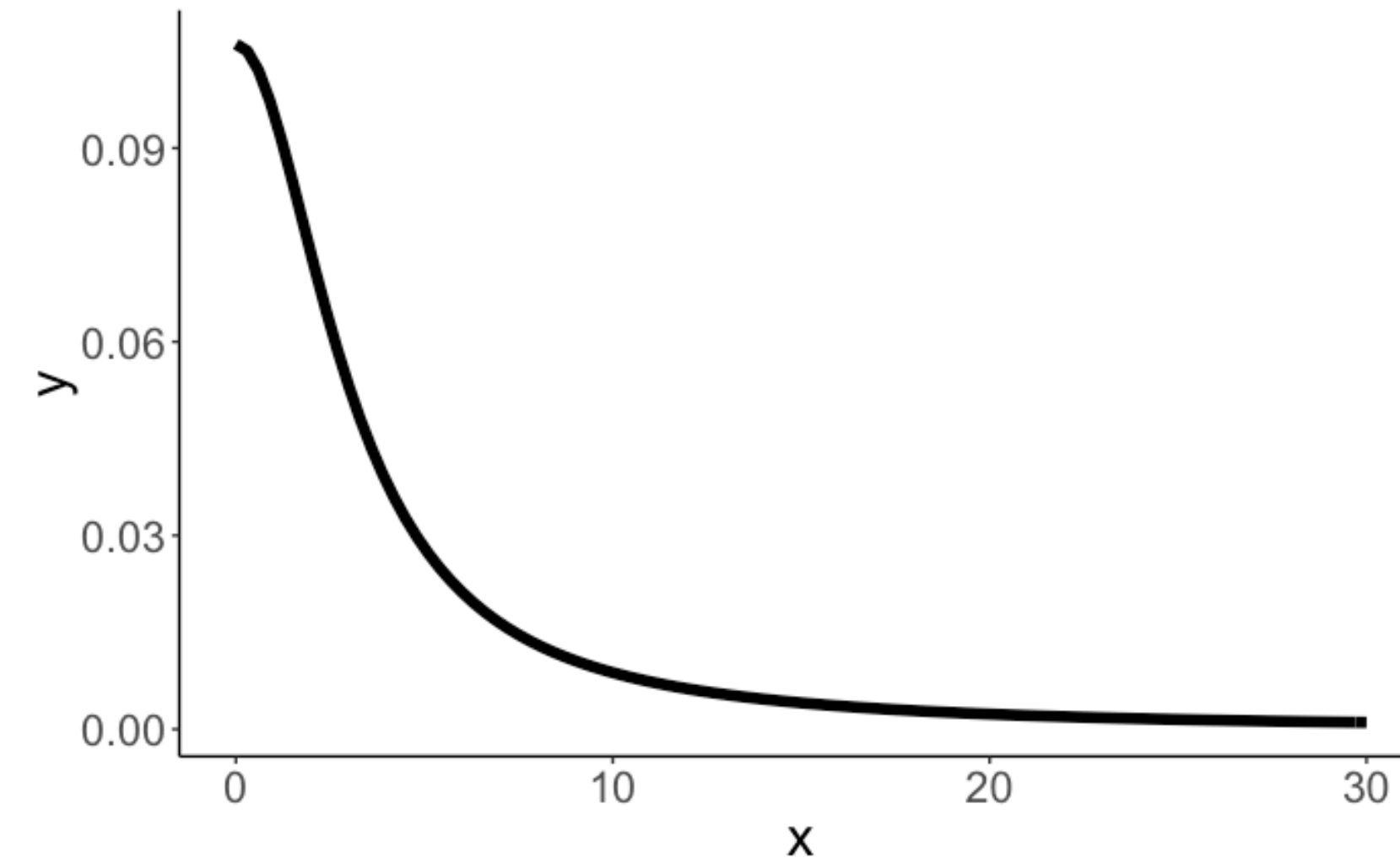
build the model



Priors



**Gaussian prior on intercept
and coefficient**

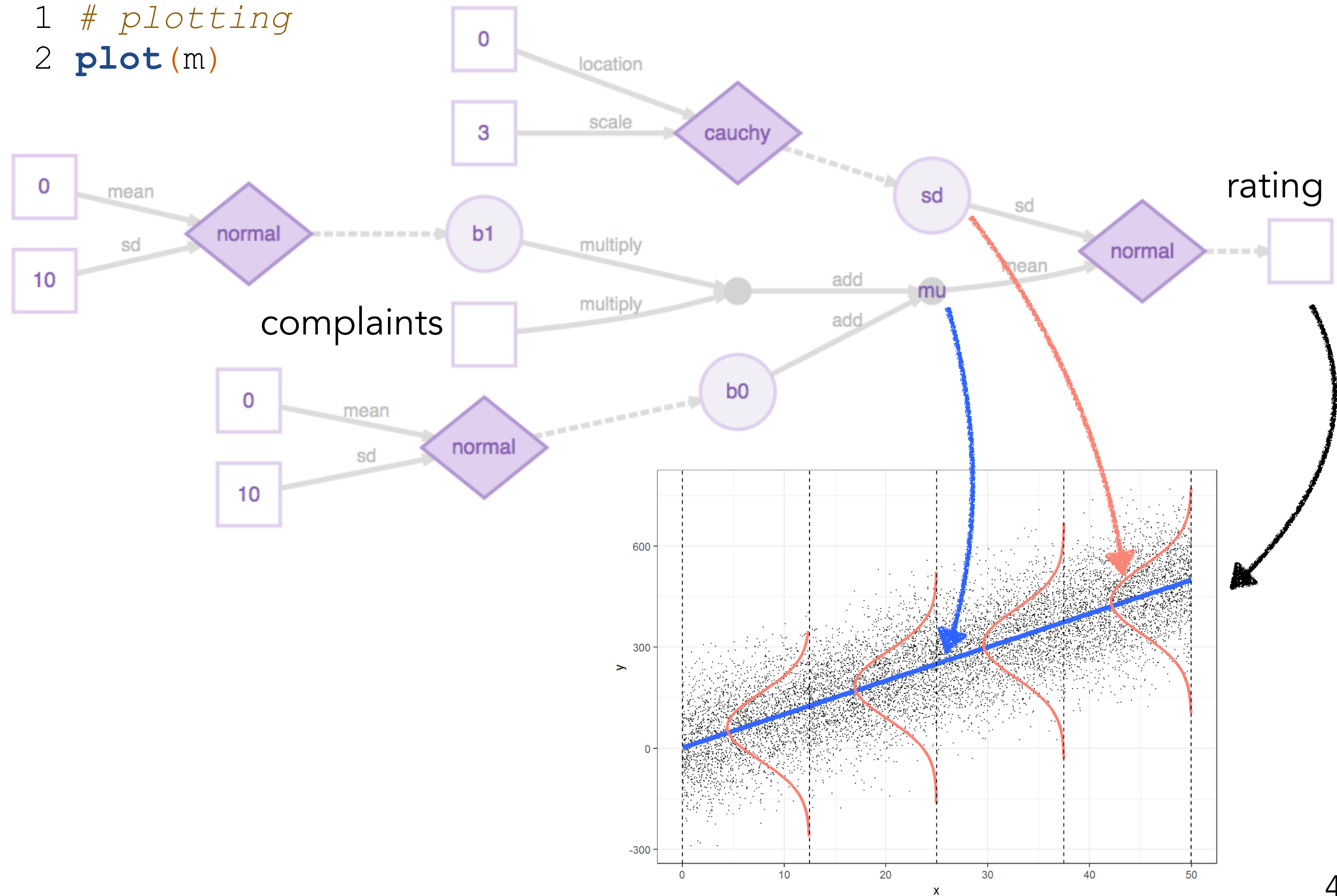


**Truncated Cauchy prior on
the standard deviation**

weakly informative priors (allow for a wide range of possible values)

Graphical representation of the model

```
1 # plotting  
2 plot(m)
```



Inference via sampling

Markov Chain Monte Carlo inference

```
1 # sampling
2 draws = mcmc(m, n_samples = 1000)
3
4 # tidy up the draws
5 df.draws = tidy_draws(draws) %>%
6   clean_names()
```

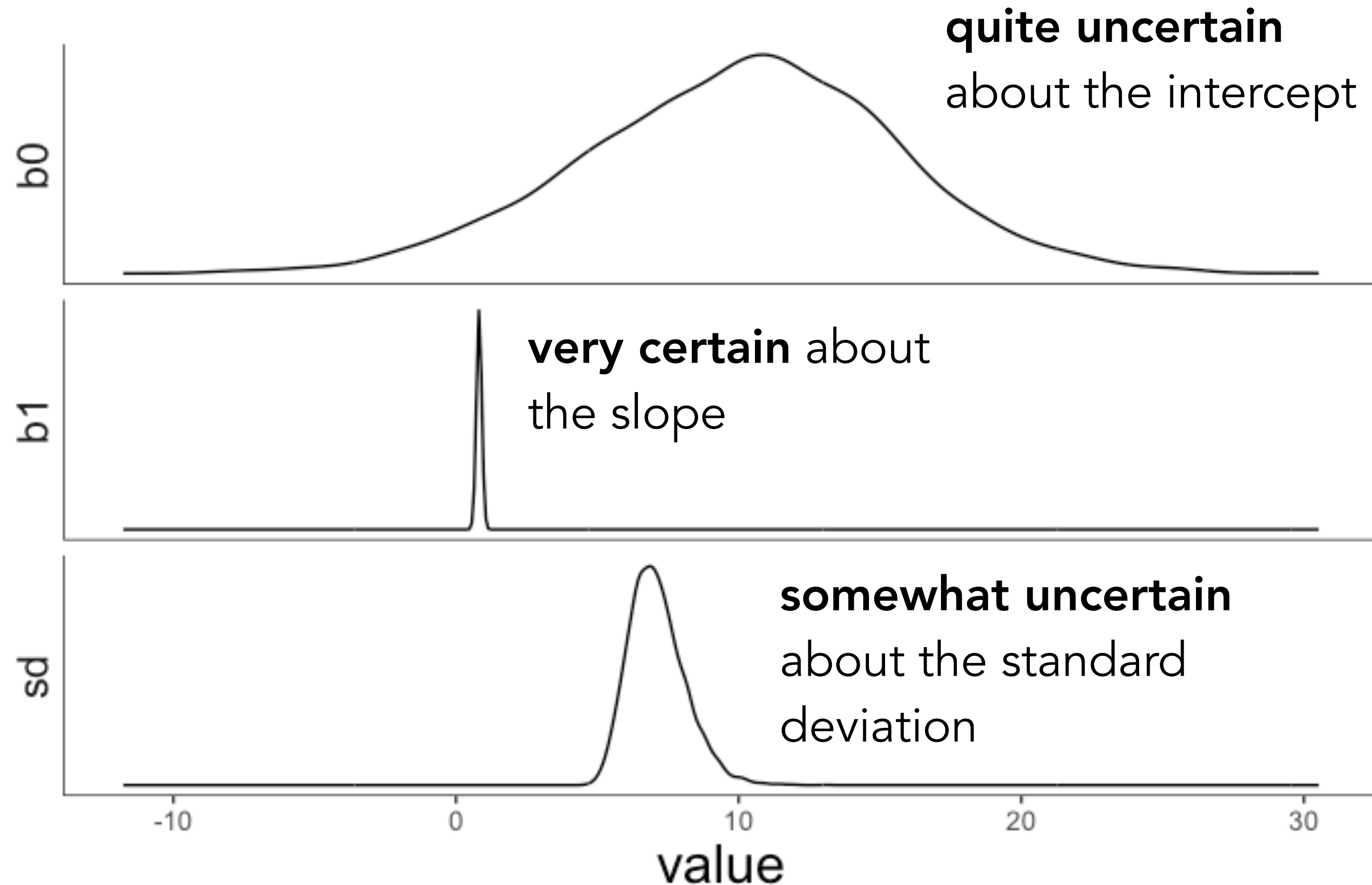
chain	iteration	draw	b0	b1	sd
1	1	1	6.08	0.87	7.60
1	2	2	1.12	0.95	7.66
1	3	3	-1.83	0.99	7.01
1	4	4	-4.23	1.02	6.64
1	5	5	3.26	0.87	7.96
1	6	6	-1.04	0.98	7.67
1	7	7	-0.83	0.97	10.12
1	8	8	-1.41	0.97	8.02
1	9	9	9.46	0.81	6.30
1	10	10	10.02	0.84	6.57

each of these is a solution
for explaining the data

nice visualization of MCMC
samplers

<https://github.com/chi-feng/mcmc-demo>

Visualize the posterior



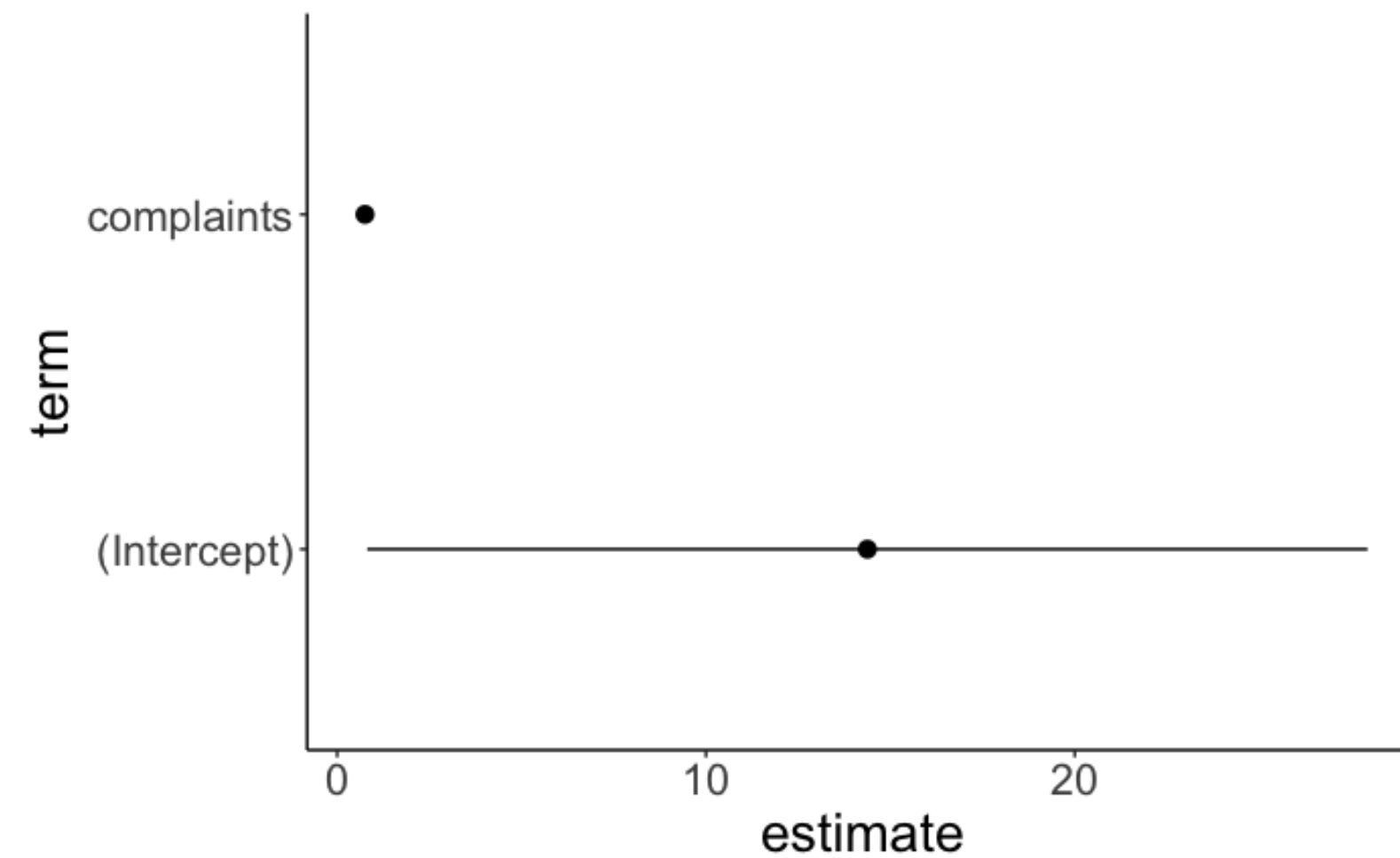
This is the solution of a Bayesian analysis.

A posterior distribution over each parameter in our model.

We can use this to visualize model predictions, and to test hypotheses. 44

Confidence interval

```
1 fit.lm = lm(formula = rating ~ 1 + complaints,
2             data = df.attitude)
```

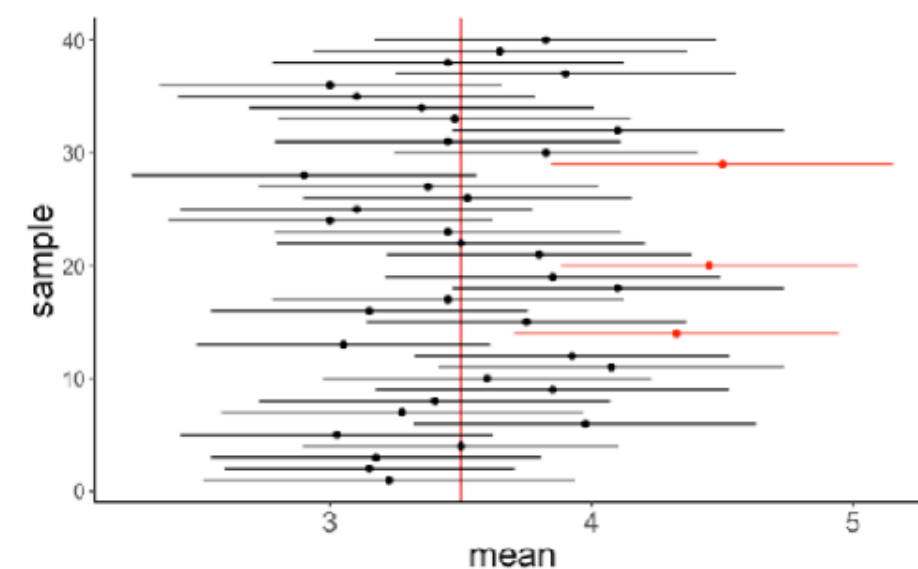


point estimate + confidence interval

Confidence interval

Definition

"If we were to repeat the experiment over and over, then 95 % of the time the confidence intervals contain the true mean."

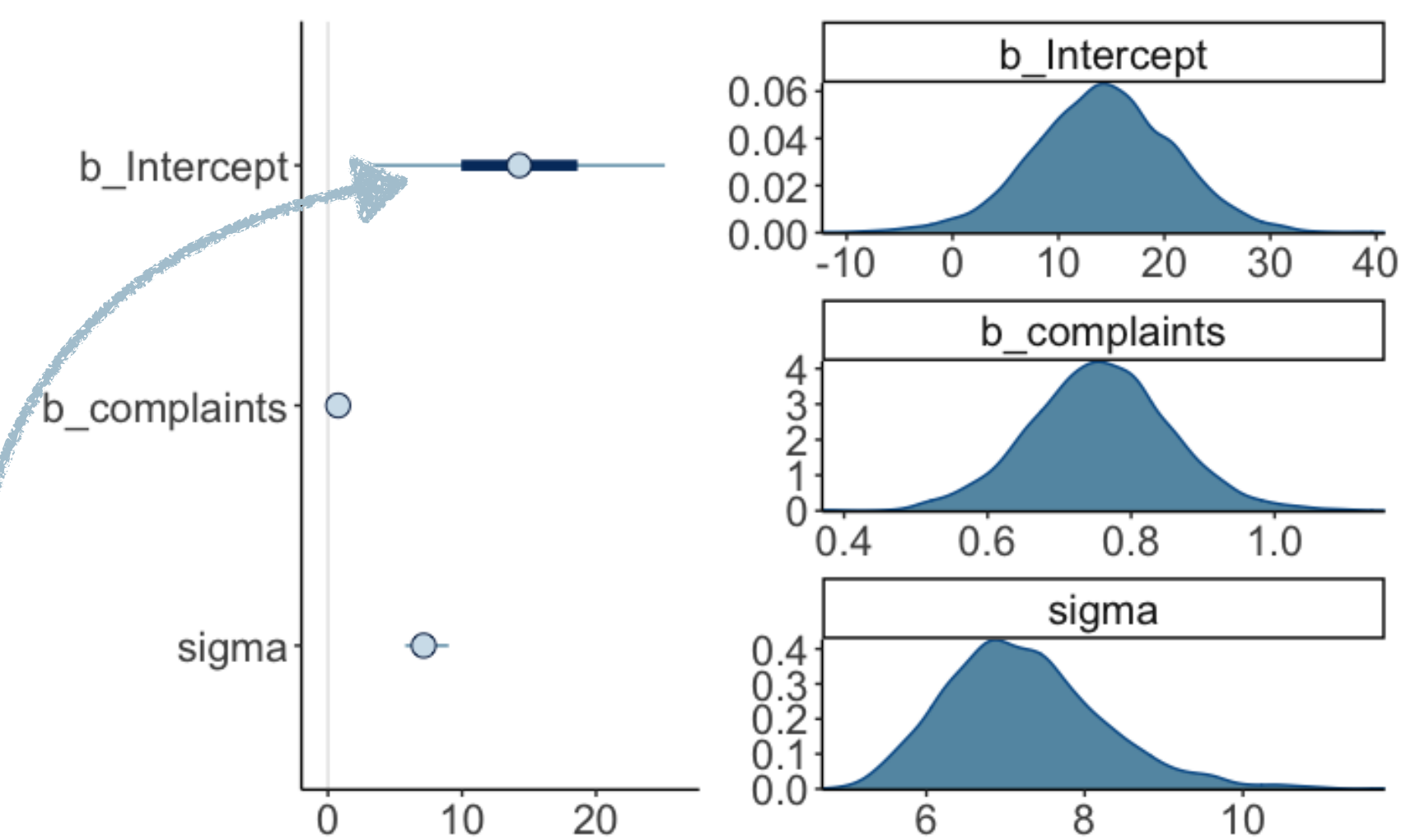


Hoekstra, R., Morey, R. D., Rouder, J. N., & Wagenmakers, E.-J. (2014). Robust Misinterpretation of Confidence Intervals. *Psychonomic Bulletin & Review*, 21(5), 1157-1164.

67

Credible interval

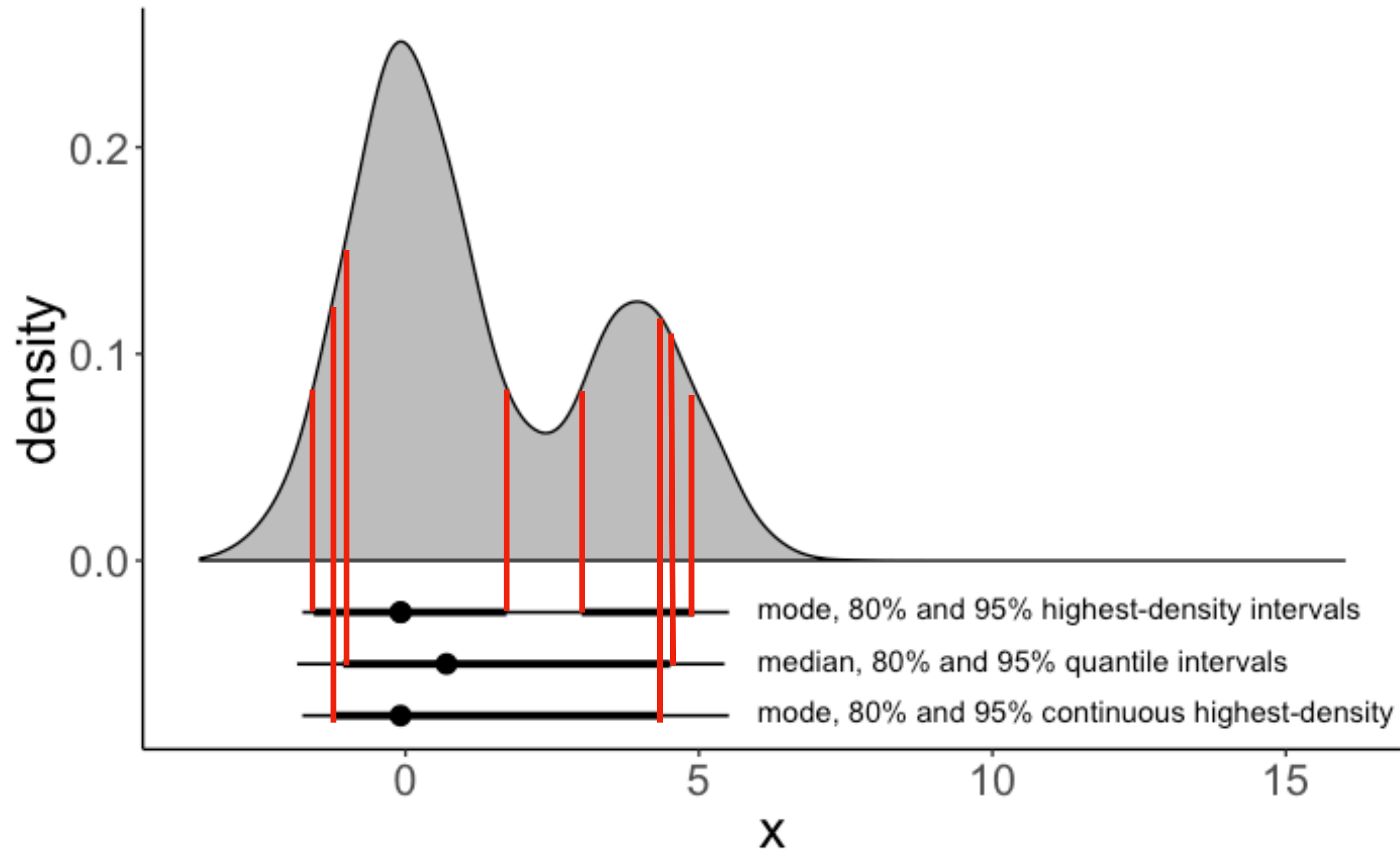
```
1 fit.brm = brm(formula = rating ~ 1 + complaints,
2               data = df.attitude)
```



full posterior distribution over each parameter

with 90% the true parameter lies within this range

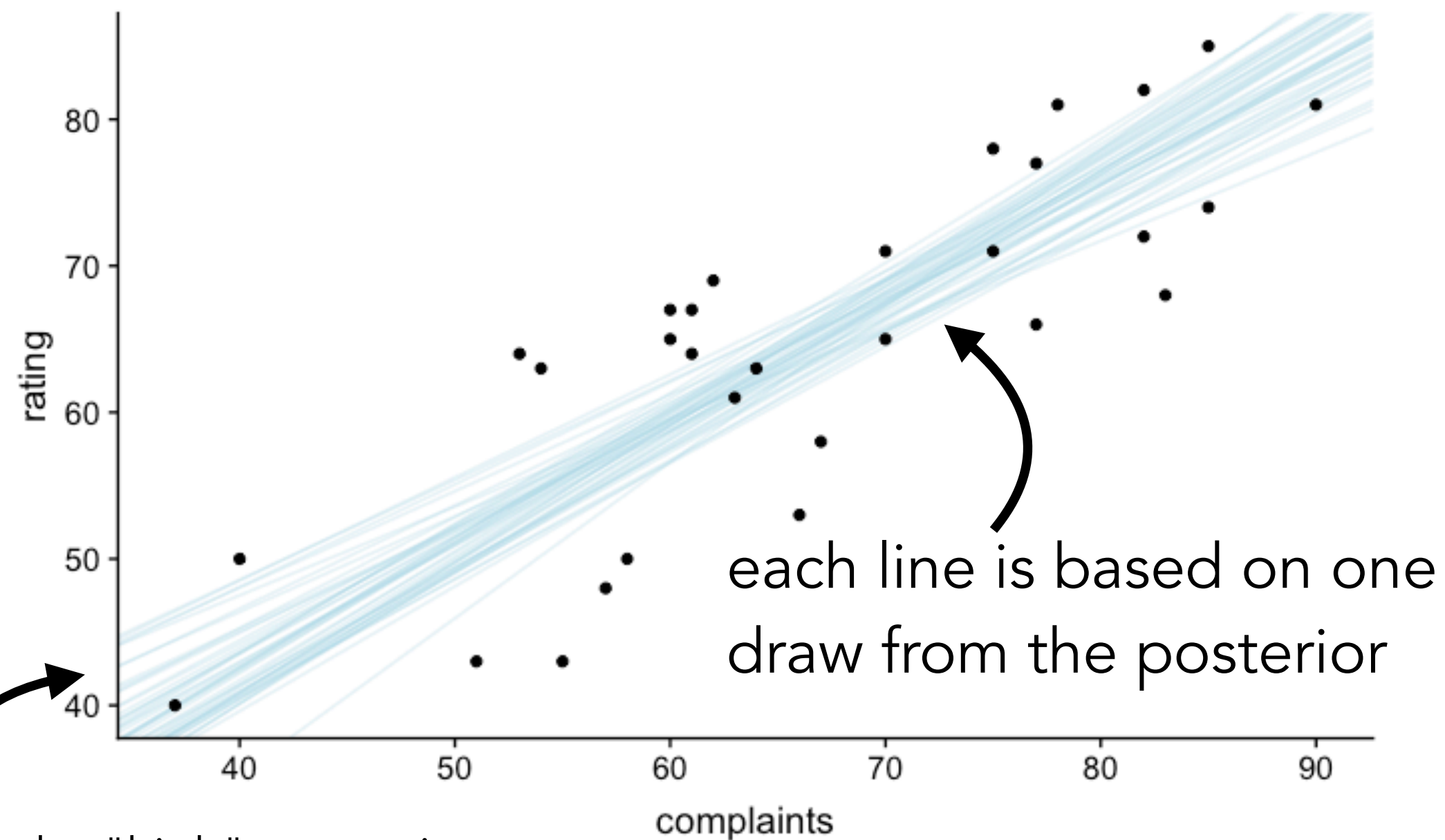
Different kinds of credible intervals



ways of summarizing the posterior distribution

Visualize the model predictions

```
1 ggplot(data = df.attitude,  
2       mapping = aes(x = complaints,  
3                     y = rating)) +  
4   geom_abline(data = df.draws %>%  
5               sample_n(size = 50),  
6               aes(intercept = b0,  
7                 slope = b1),  
8               alpha = 0.3,  
9               color = "lightblue") +  
10  geom_point()
```

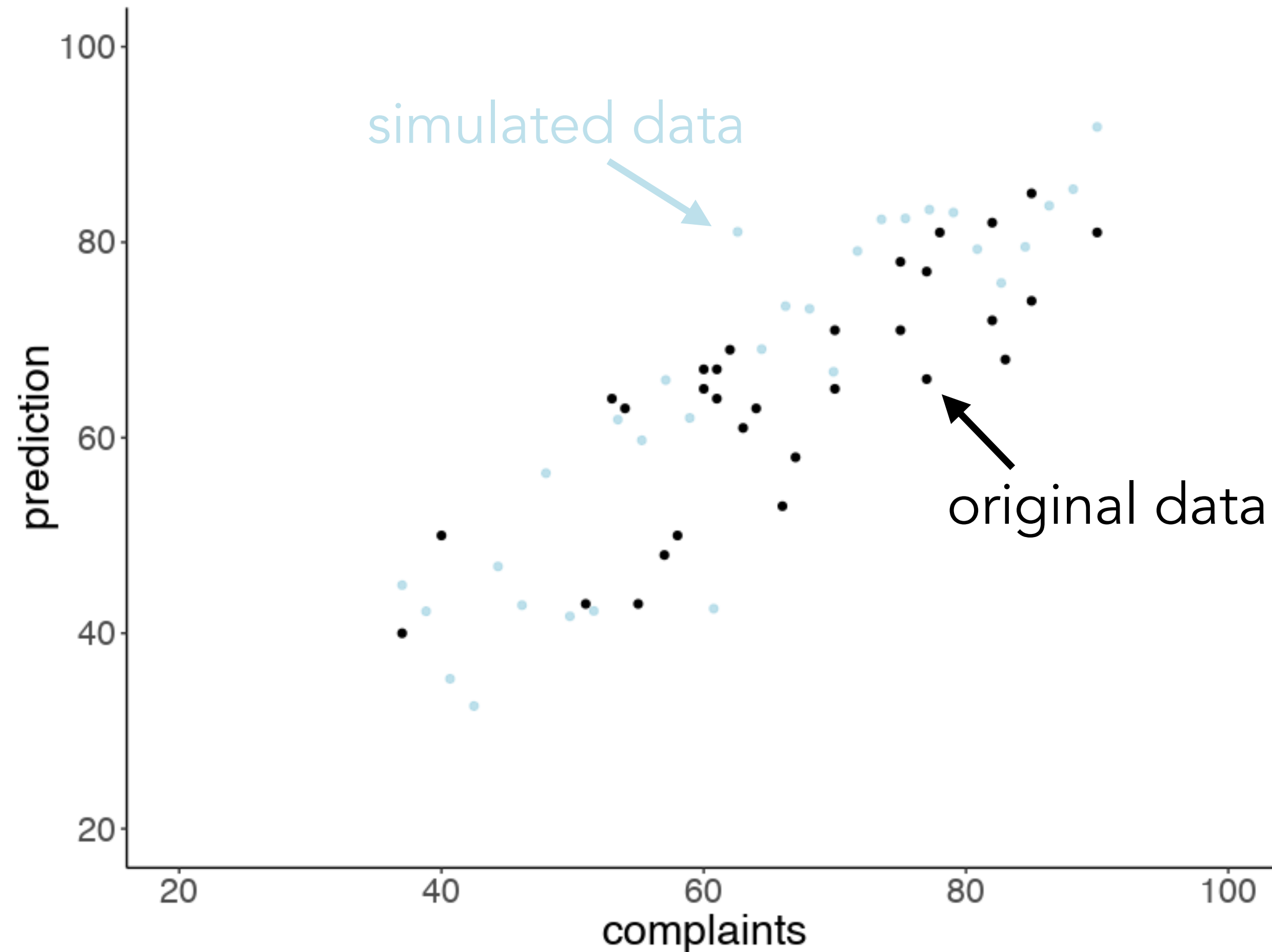


chain	iteration	draw	b0	b1	sd
1	1	1	6.08	0.87	7.60
1	2	2	1.12	0.95	7.66
1	3	3	-1.83	0.99	7.01
1	4	4	-4.23	1.02	6.64
1	5	5	3.26	0.87	7.96
1	6	6	-1.04	0.98	7.67
1	7	7	-0.83	0.97	10.12
1	8	8	-1.41	0.97	8.02
1	9	9	9.46	0.81	6.30
1	10	10	10.02	0.84	6.57

explains the "high" uncertainty
about the intercept

Posterior predictive check

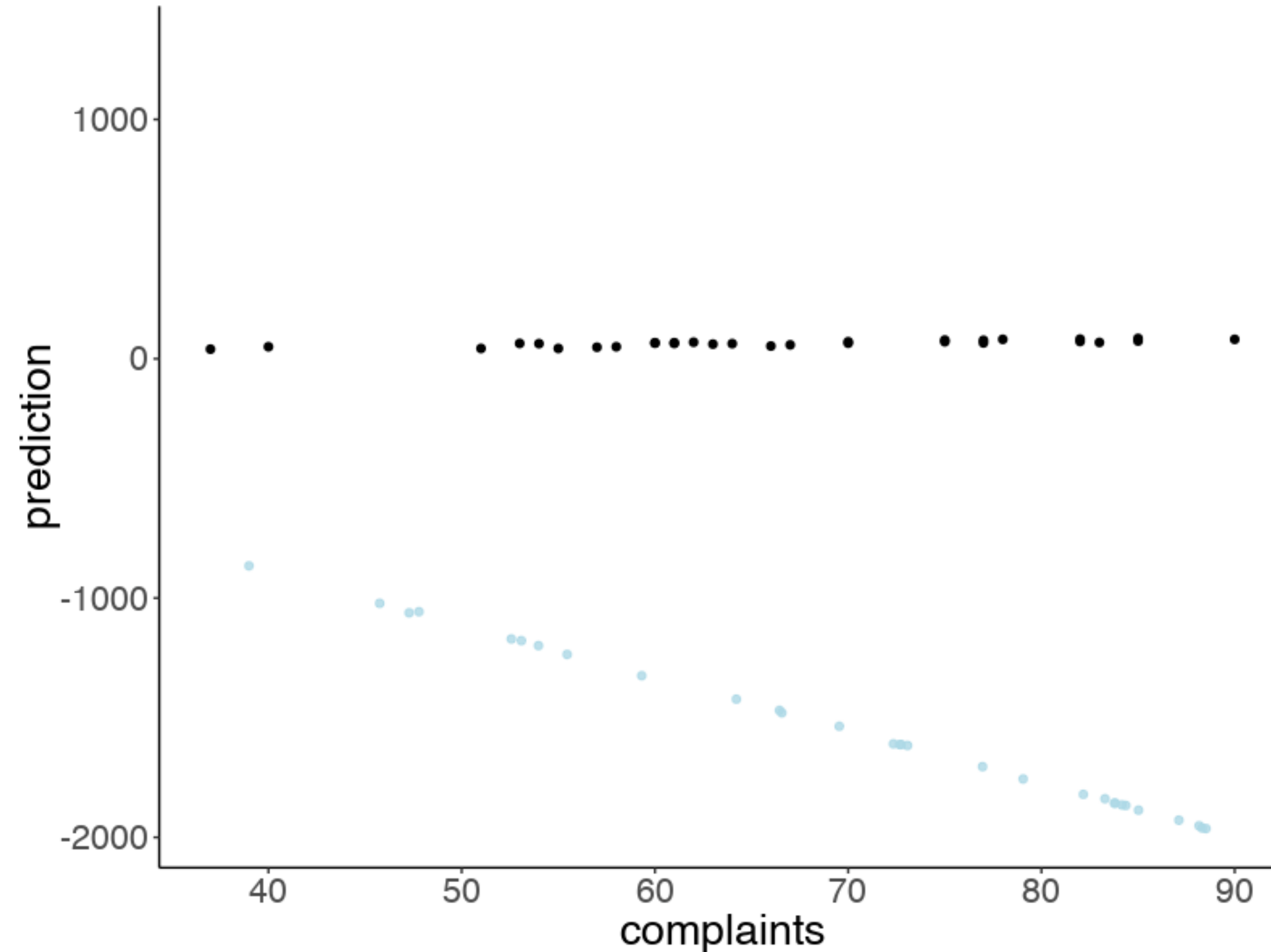
1. sample parameters from the **posterior distribution**
2. generate data using these parameters (using the likelihood function)



Hypothetical outcome plot

Prior predictive check

1. sample parameters from the **prior distribution**
2. generate data using these parameters (using the likelihood function)



Hypothetical outcome plot



Paul Bürkner

Doing Bayesian data analysis with BRMS

Software packages

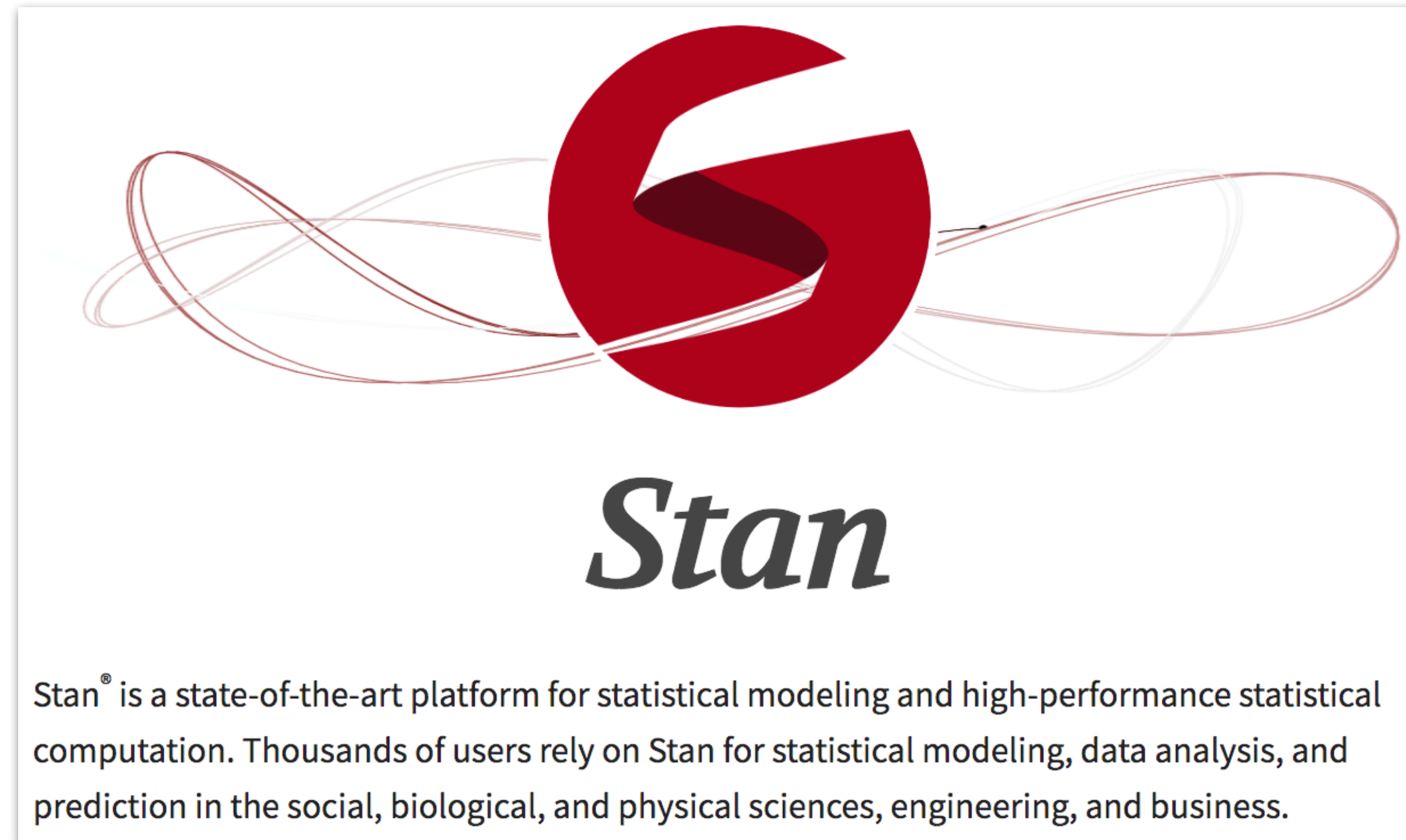


Bayesian regression
modeling with Stan

```
library ("brms")
```

- very powerful package that makes it easy to run Bayesian regression models
- we specify models using the same syntax we've already learned based on **lm()**, **glm()**, and **lmer()**
- `brms` turns this into Stan code and fits the model
- we can then use `tidybayes` to investigate the posterior

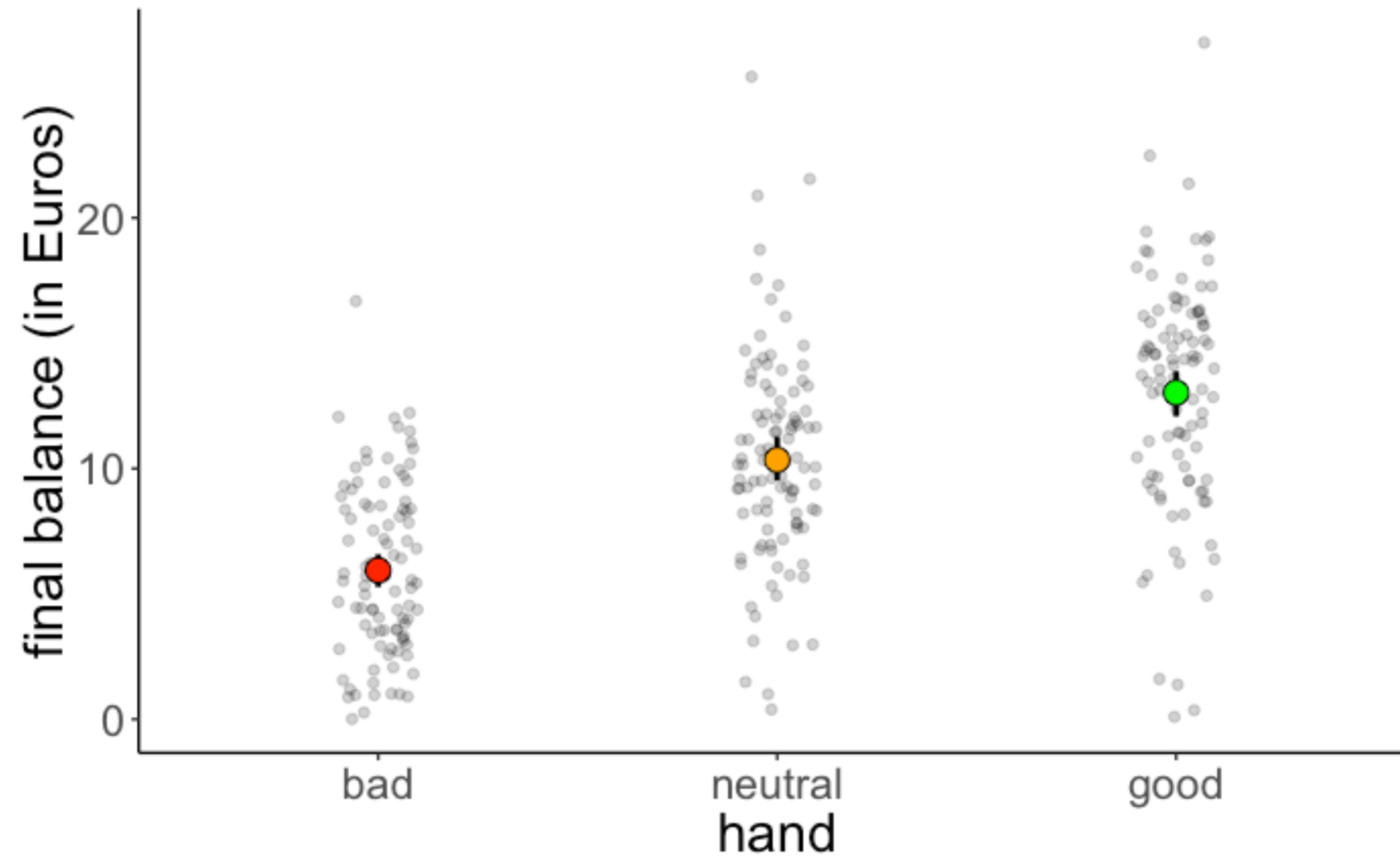
Software packages



<https://mc-stan.org/>

Poker data

Poker data



Using `lm()`

```
1 fit.lm_poker = lm(formula = balance ~ 1 + hand,  
2                  data = df.poker)  
3  
4 fit.lm_poker %>% summary()
```

```
Call:  
lm(formula = balance ~ 1 + hand, data = df.poker)  
  
Residuals:  
      Min       1Q   Median       3Q      Max   
-12.9264  -2.5902  -0.0115   2.6573  15.2834  
  
Coefficients:  
              Estimate Std. Error t value Pr(>|t|)      
(Intercept)   5.9415     0.4111  14.451  < 2e-16 ***  
handneutral   4.4051     0.5815   7.576  4.55e-13 ***  
handgood      7.0849     0.5815  12.185  < 2e-16 ***  
---  
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1  
  
Residual standard error: 4.111 on 297 degrees of freedom  
Multiple R-squared:  0.3377,    Adjusted R-squared:  0.3332  
F-statistic: 75.7 on 2 and 297 DF,  p-value: < 2.2e-16
```

Using brm ()

Good!

```
1 fit.brm_poker = brm(formula = balance ~ 1 + hand,  
2                     data = df.poker)  
3  
4 fit.brm_poker %>% summary()
```

```
Family: gaussian  
Links: mu = identity; sigma = identity  
Formula: balance ~ 1 + hand  
Data: df.poker (Number of observations: 300)  
Samples: 4 chains, each with iter = 2000; warmup = 1000; thin = 1;  
         total post-warmup samples = 4000  
  
Population-Level Effects:  
              Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS  
Intercept      5.93      0.41    5.12    6.72 1.00    2986    2744  
handneutral    4.41      0.58    3.30    5.55 1.00    3497    2903  
handgood       7.10      0.58    5.99    8.29 1.00    3545    2932  
  
Family Specific Parameters:  
              Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS  
sigma         4.12      0.17    3.81    4.46 1.00    3650    2921  
  
Samples were drawn using sampling(NUTS). For each parameter, Eff.Sample  
is a crude measure of effective sample size, and Rhat is the potential  
scale reduction factor on split chains (at convergence, Rhat = 1).
```


Comparison between `lm()` and `brm()`

`lm()`

```
Coefficients:
(Intercept)  5.9415      0.4111    14.451    < 2e-16 ***
handneutral  4.4051      0.5815     7.576    4.55e-13 ***
handgood     7.0849      0.5815    12.185    < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

`brm()`

```
Population-Level Effects:
              Estimate Est.Error 1-95% CI u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept      5.93      0.41    5.12    6.72 1.00    2986    2744
handneutral    4.41      0.58    3.30    5.55 1.00    3497    2903
handgood       7.10      0.58    5.99    8.29 1.00    3545    2932
```

**almost identical
results!**

What about the priors?

```
1 fit.brm_poker = brm(formula = balance ~ 1 + hand,  
2                     data = df.poker)
```

By default, `brms` uses weakly informative priors for the model parameters.

There are quite a few other defaults, let's take a look under the hood ...

"Full" specification of the model

```
1 fit.brm_poker_full = brm(  
2   formula = balance ~ 1 + hand,  
3   family = "gaussian",  
4   data = df.poker,  
5   prior = c(  
6     prior(normal(0, 10), class = "b", coef = "handgood"),  
7     prior(normal(0, 10), class = "b", coef = "handneutral"),  
8     prior(student_t(3, 3, 10), class = "Intercept"),  
9     prior(student_t(3, 0, 10), class = "sigma")  
10  ),  
11  inits = list(  
12    list(Intercept = 0, sigma = 1, handgood = 5, handneutral = 5),  
13    list(Intercept = -5, sigma = 3, handgood = 2, handneutral = 2),  
14    list(Intercept = 2, sigma = 1, handgood = -1, handneutral = 1),  
15    list(Intercept = 1, sigma = 2, handgood = 2, handneutral = -2)  
16  ),  
17  iter = 4000,  
18  warmup = 1000,  
19  chains = 4,  
20  file = "cache/brm_poker_full",  
21  seed = 1  
22 )
```

likelihood

priors

initialization

how many runs in the inference chain

how long for the warmup

how many chains

save the model result

make reproducible

fitting Bayesian models
takes some time, so
storing results is key

Turned into Stan code

```
// generated with brms 2.7.0
functions {
}
data {
  int<lower=1> N; // total number of observations
  vector[N] Y; // response variable
  int<lower=1> K; // number of population-level effects
  matrix[N, K] X; // population-level design matrix
  int prior_only; // should the likelihood be ignored?
}
transformed data {
  int Kc = K - 1;
  matrix[N, K - 1] Xc; // centered version of X
  vector[K - 1] means_X; // column means of X before centering
  for (i in 2:K) {
    means_X[i - 1] = mean(X[, i]);
    Xc[, i - 1] = X[, i] - means_X[i - 1];
  }
}
parameters {
  vector[Kc] b; // population-level effects
  real temp_Intercept; // temporary intercept
  real<lower=0> sigma; // residual SD
}
transformed parameters {
}
model {
  vector[N] mu = temp_Intercept + Xc * b;
  // priors including all constants
  target += normal_lpdf(b[1] | 0, 10);
  target += normal_lpdf(b[2] | 0, 10);
  target += student_t_lpdf(temp_Intercept | 3, 3, 10);
  target += student_t_lpdf(sigma | 3, 0, 10)
    - 1 * student_t_lccdf(0 | 3, 0, 10);
  // likelihood including all constants
  if (!prior_only) {
    target += normal_lpdf(Y | mu, sigma);
  }
}
generated quantities {
  // actual population-level intercept
  real b_Intercept = temp_Intercept - dot_product(means_X, b);
}
```

- probabilistic programming language
- flexible construction of Bayesian models
- ports have been written for R, Python, Julia, ...
- implements a fast inference algorithm

Results

posterior samples

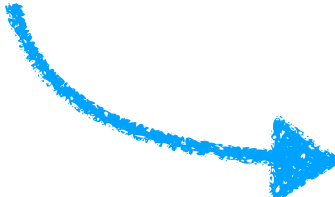
b_Intercept	b_handneutral	b_handgood	sigma
5.97	4.27	7.48	3.94
5.11	5.25	7.40	3.91
7.03	3.78	5.80	4.48
5.72	4.18	7.25	4.00
6.01	4.44	6.15	4.57
5.94	4.69	6.72	4.36
6.39	3.84	6.40	3.92
5.24	5.15	7.69	4.16
6.12	4.51	7.20	4.14
6.43	3.71	6.37	4.13
5.85	5.01	7.32	4.00
6.51	3.58	6.62	3.95
5.85	4.45	7.62	4.17
5.80	5.45	6.36	4.10
5.48	5.51	7.22	3.99

⋮

summary of posterior

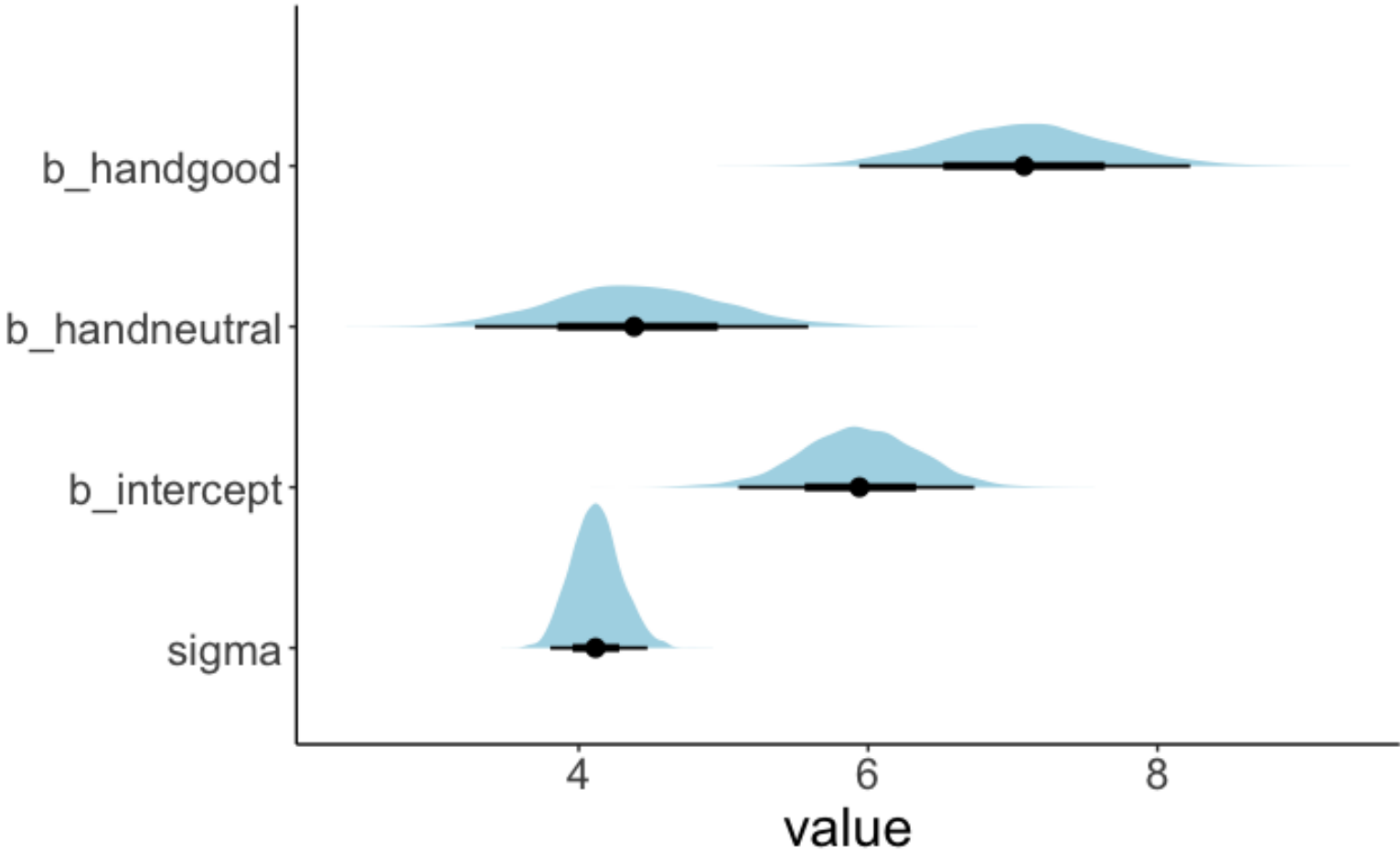
parameter	lower	mode	upper
b_handgood	5.97	7.07	8.27
b_handneutral	3.21	4.43	5.51
b_intercept	5.17	5.95	6.77
sigma	3.81	4.12	4.47

maximum
a posteriori



MAP estimate and 95%
highest density interval

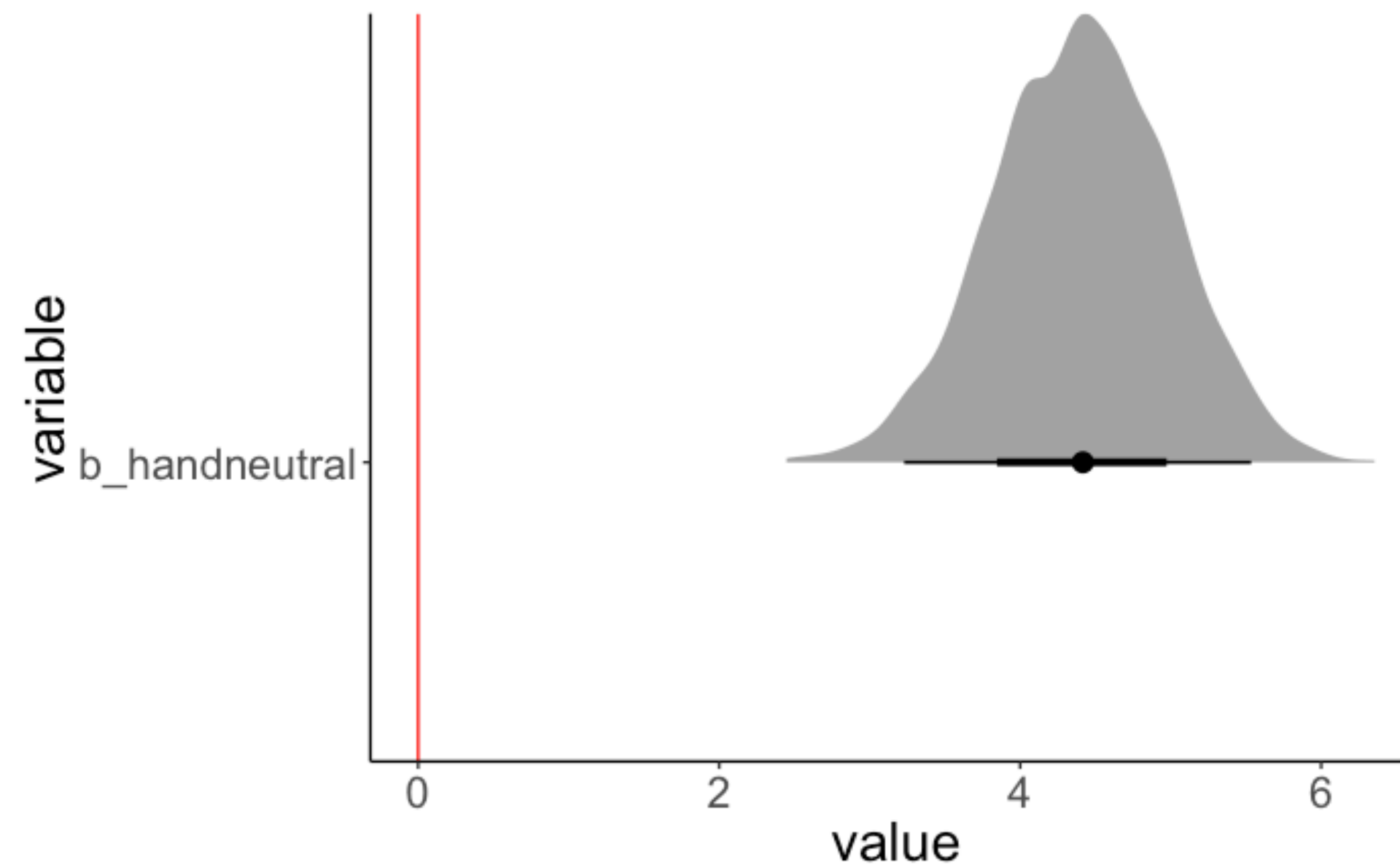
visualization



Testing hypotheses

Asking questions based on the posterior

Do neutral hands earn more money than bad hands?



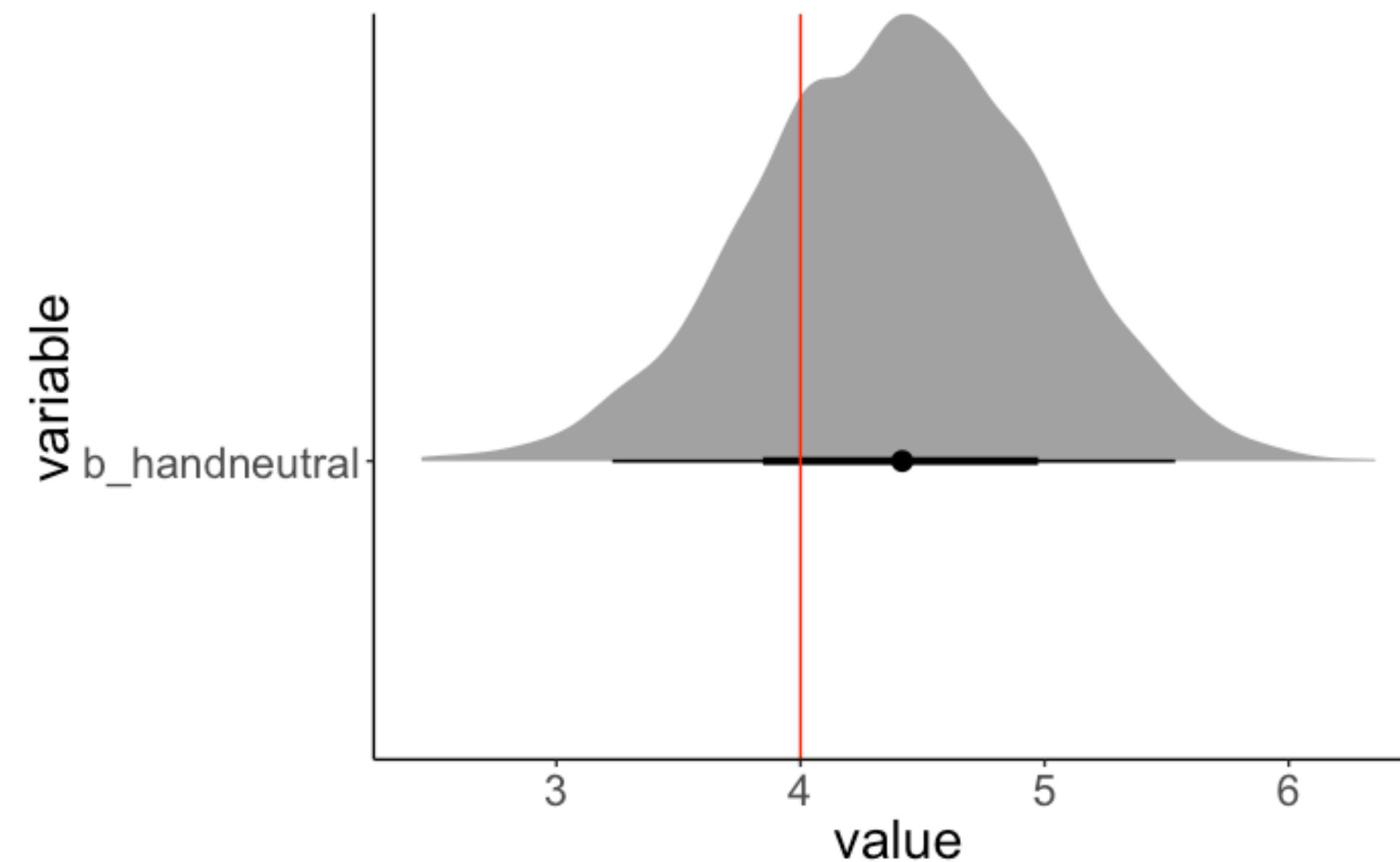
What's the probability that handneutral is less than 0?

```
1 hypothesis(fit.brm,  
2           hypothesis = "handneutral < 0")
```

$p = 0$

Asking questions based on the posterior

Do neutral hands earn much more money than bad hands?



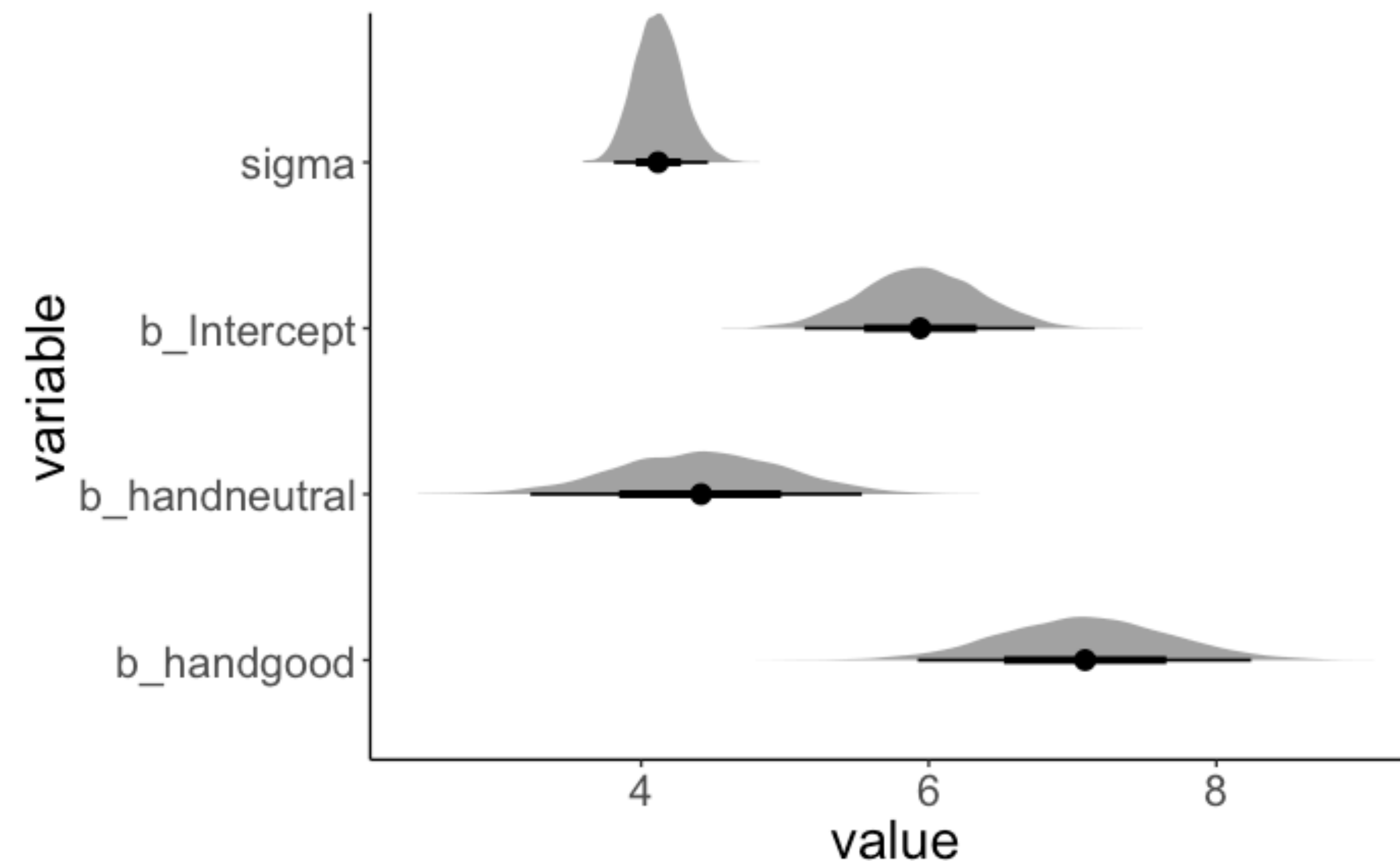
What's the probability that the difference between neutral and bad hands is **more than 4**?

```
1 hypothesis(fit.brm,  
2           hypothesis = "handneutral > 4")
```

$p = 0.75$

Asking questions based on the posterior

Do good hands make twice as much as bad hands?

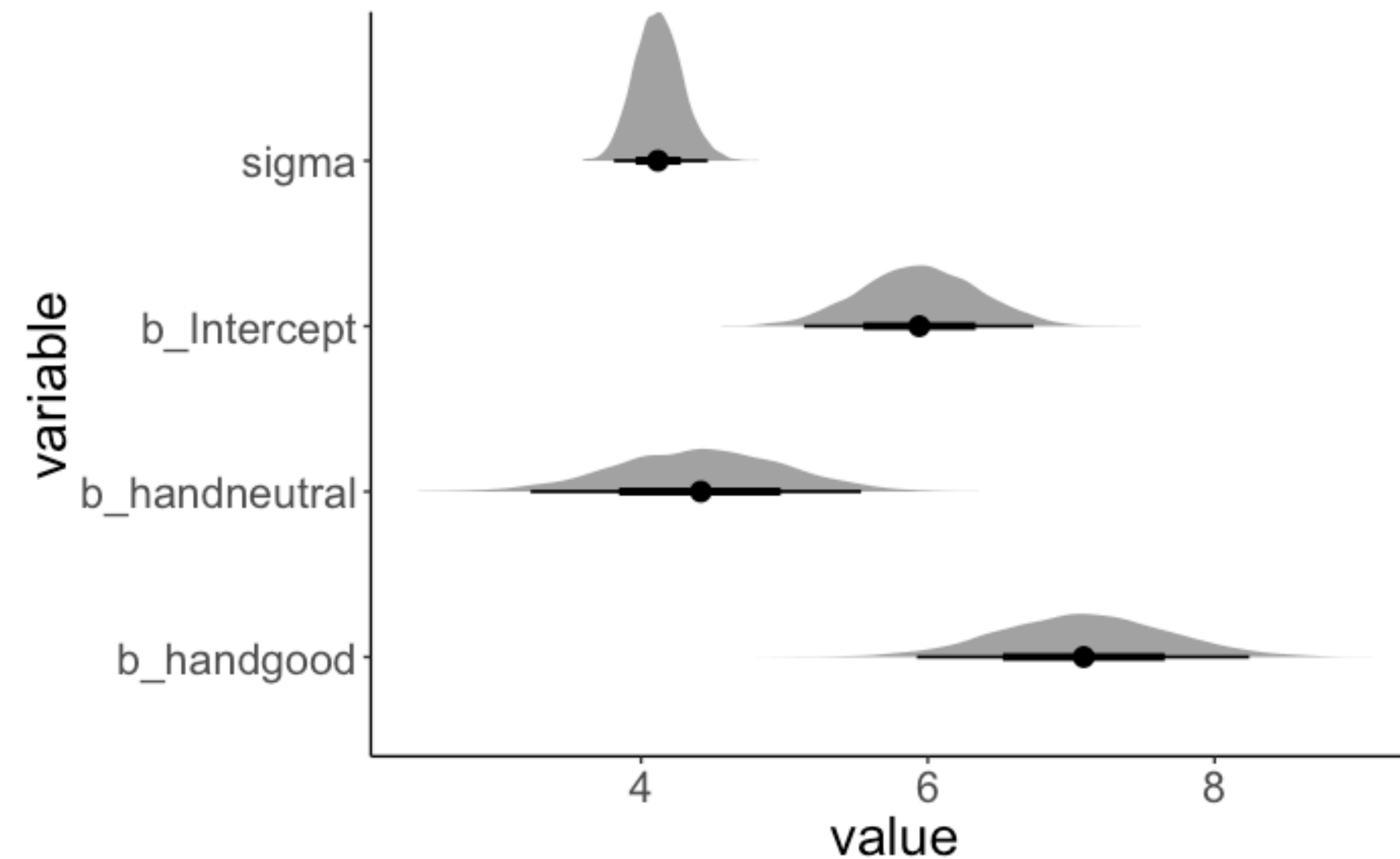


```
1 hypothesis(fit.brm,  
2           hypothesis = "handgood + Intercept > 2 * Intercept")
```

$p = 0.89$

Asking questions based on the posterior

Are neutral hands worse than bad and good hands combined?



```
1 hypothesis(fit.brm,  
2           hypothesis = "Intercept + handneutral < (Intercept + Intercept + handgood) / 2")
```

$p = 0.04$

Testing hypothesis

```
1 df.hypothesis = fit.brm %>%
2   posterior_samples() %>%
3   clean_names() %>%
4   select(starts_with("b_")) %>%
5   mutate(neutral = b_intercept + b_handneutral,
6          bad_good_average = (b_intercept + b_intercept + b_handgood) / 2,
7          hypothesis = neutral < bad_good_average)
```

**samples
from the
posterior**



b_intercept	b_handneutral	b_handgood	neutral	bad_good_average	hypothesis
6.07	4.10	7.20	10.17	9.67	FALSE
6.06	4.44	6.95	10.49	9.53	FALSE
5.88	5.00	6.73	10.87	9.24	FALSE
5.85	4.78	6.18	10.63	8.94	FALSE
5.86	4.46	7.68	10.32	9.70	FALSE

```
1 df.hypothesis %>%
2   summarize(p = sum(hypothesis) / n())
```

$p = 0.04$

Testing hypotheses

Having a posterior distribution allows us to ask questions about the data in a very flexible way!

The "emmeans" package is your friend!

```
1 fit.brm_poker %>%  
2   emmeans(specs = consec ~ hand)
```

estimated
mean for
each group →

```
$emmeans  
hand      emmean lower.HPD upper.HPD  
bad        5.94      5.16      6.78  
neutral    10.34      9.55     11.15  
good       13.02     12.22     13.82
```

```
Point estimate displayed: median  
HPD interval probability: 0.95
```

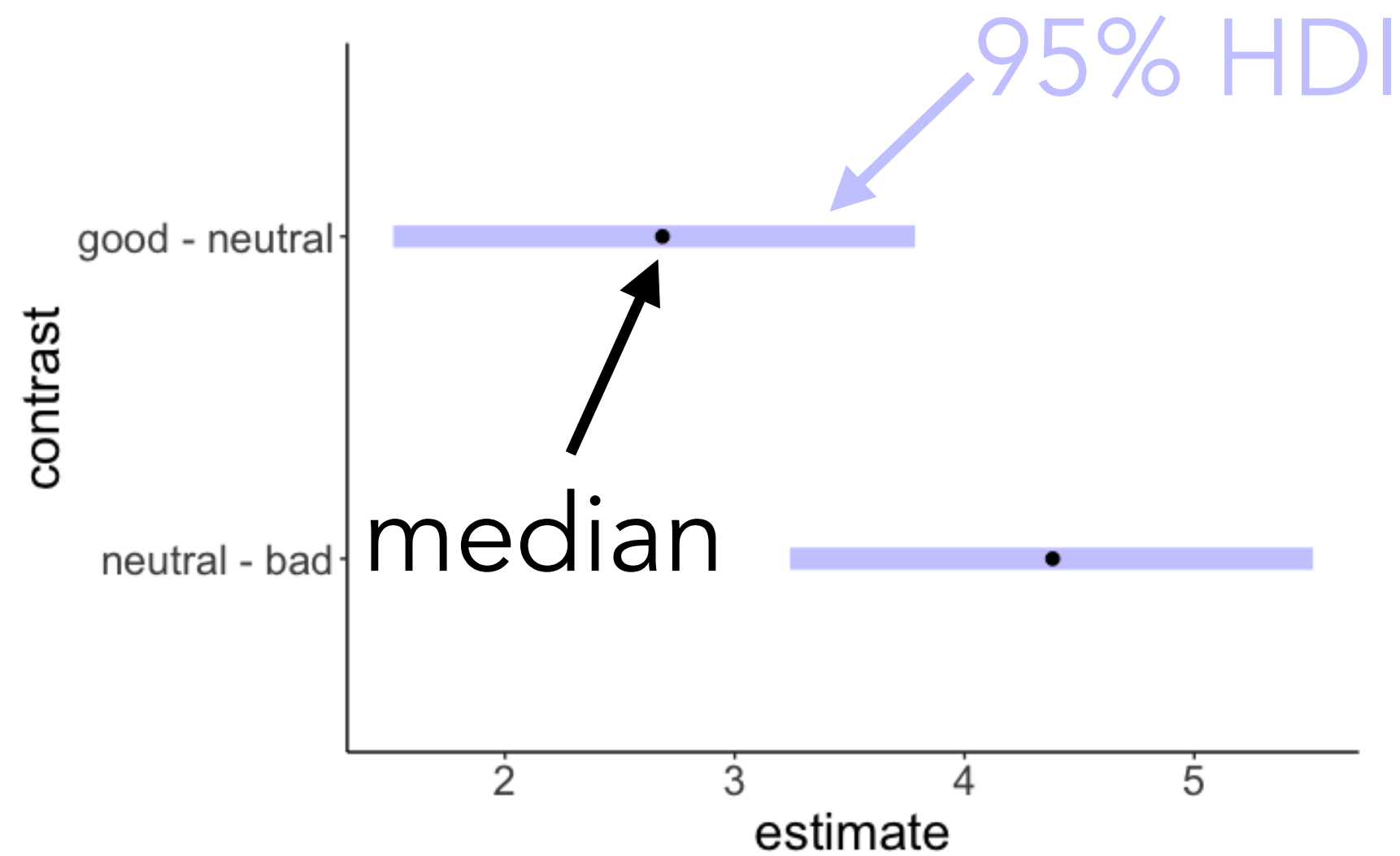
contrasts →

```
$contrasts  
contrast      estimate lower.HPD upper.HPD  
neutral - bad      4.38      3.24      5.52  
good - neutral     2.69      1.51      3.78
```

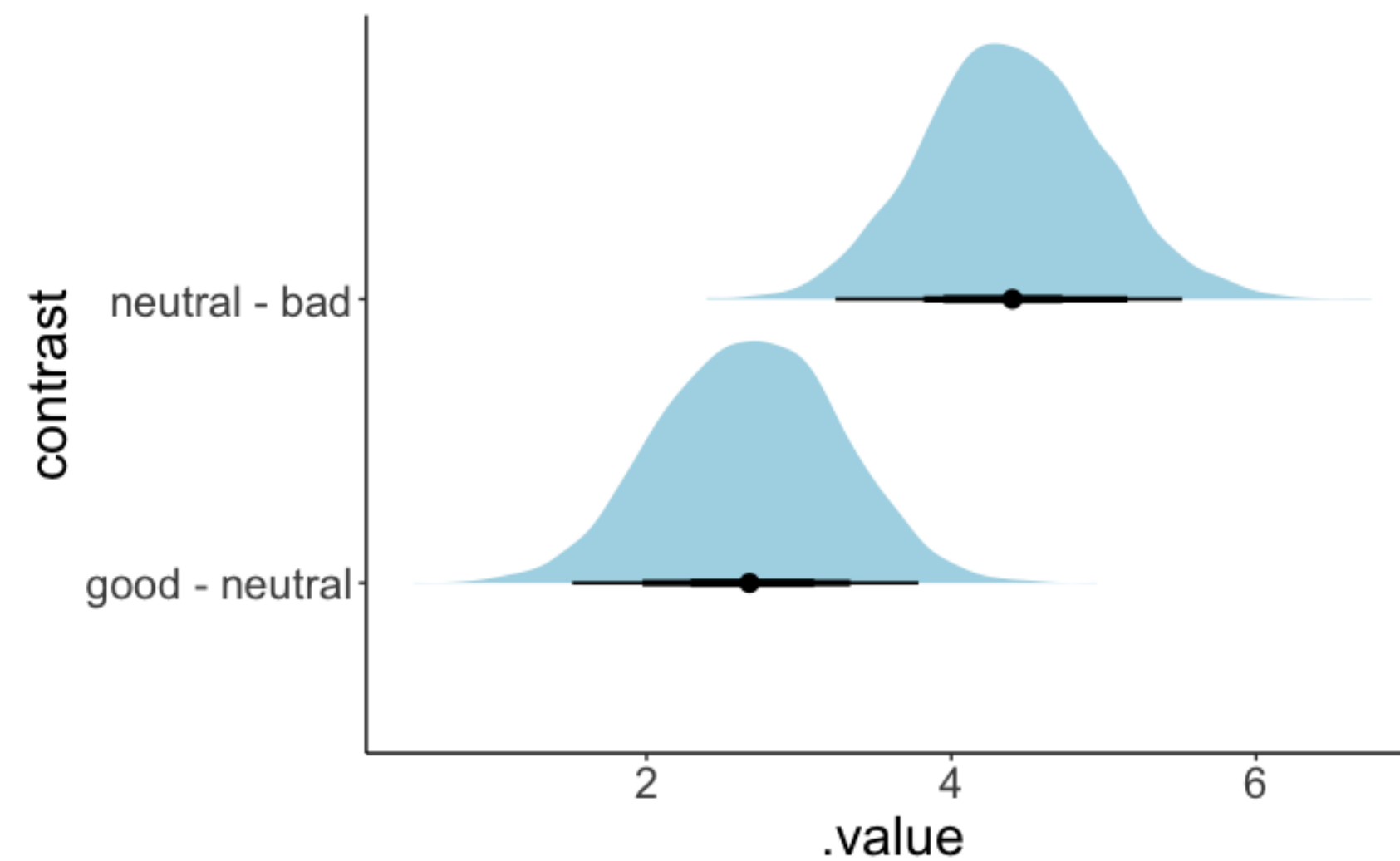
```
Point estimate displayed: median  
HPD interval probability: 0.95
```

Visualizing the contrasts

```
1 fit.brm_poker %>%
2   emmeans(specs = consec ~ hand) %>%
3   pluck("contrasts") %>%
4   plot()
```



```
1 fit.brm_poker %>%
2   emmeans(specs = consec ~ hand) %>%
3   pluck("contrasts") %>%
4   gather_emmeans_draws() %>%
5   ggplot(mapping = aes(y = contrast,
6                         x = .value)) +
7   stat_halfeye(fill = "lightblue",
8               point_interval = mean_hdi,
9               .width = c(0.5, 0.75, 0.95))
```



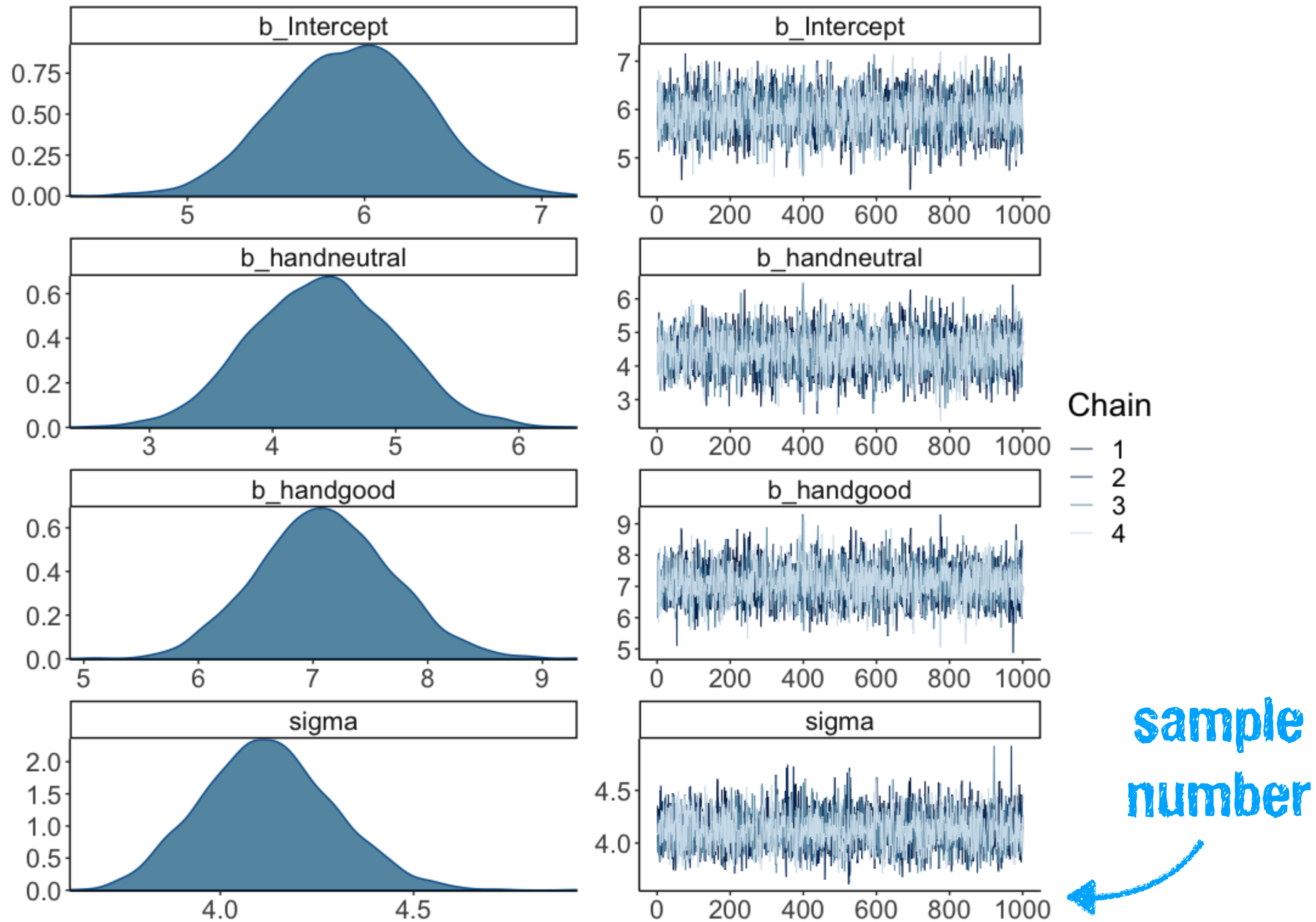
mean, 50% HDI, 75% HDI, 95% HDI

Model evaluation

1. Check whether inference worked

Can we trust the model results?

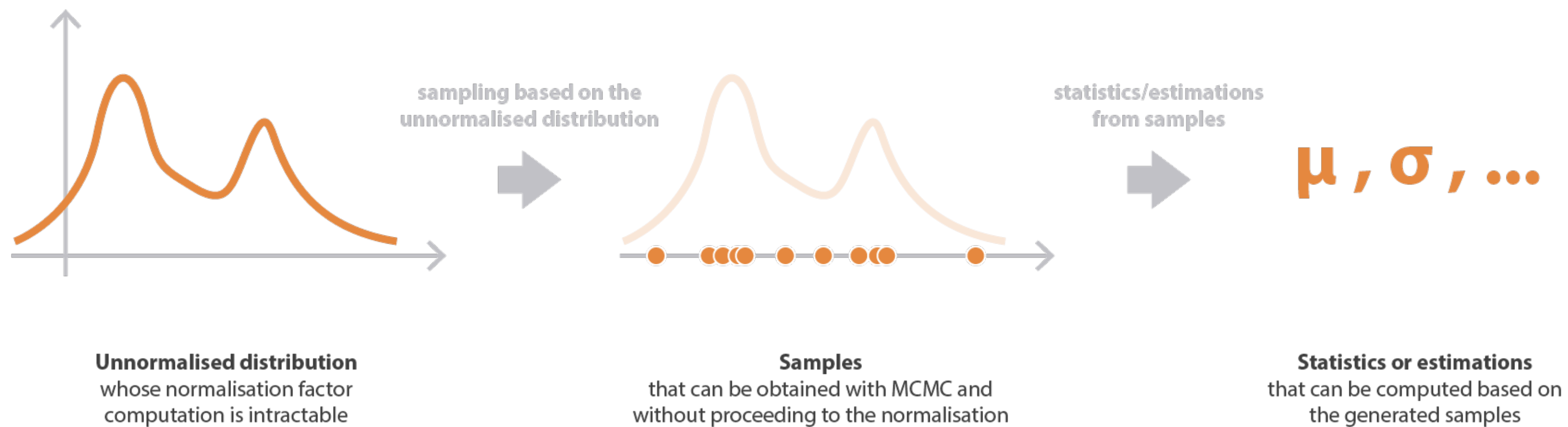
```
plot(fit.brm_poker)
```



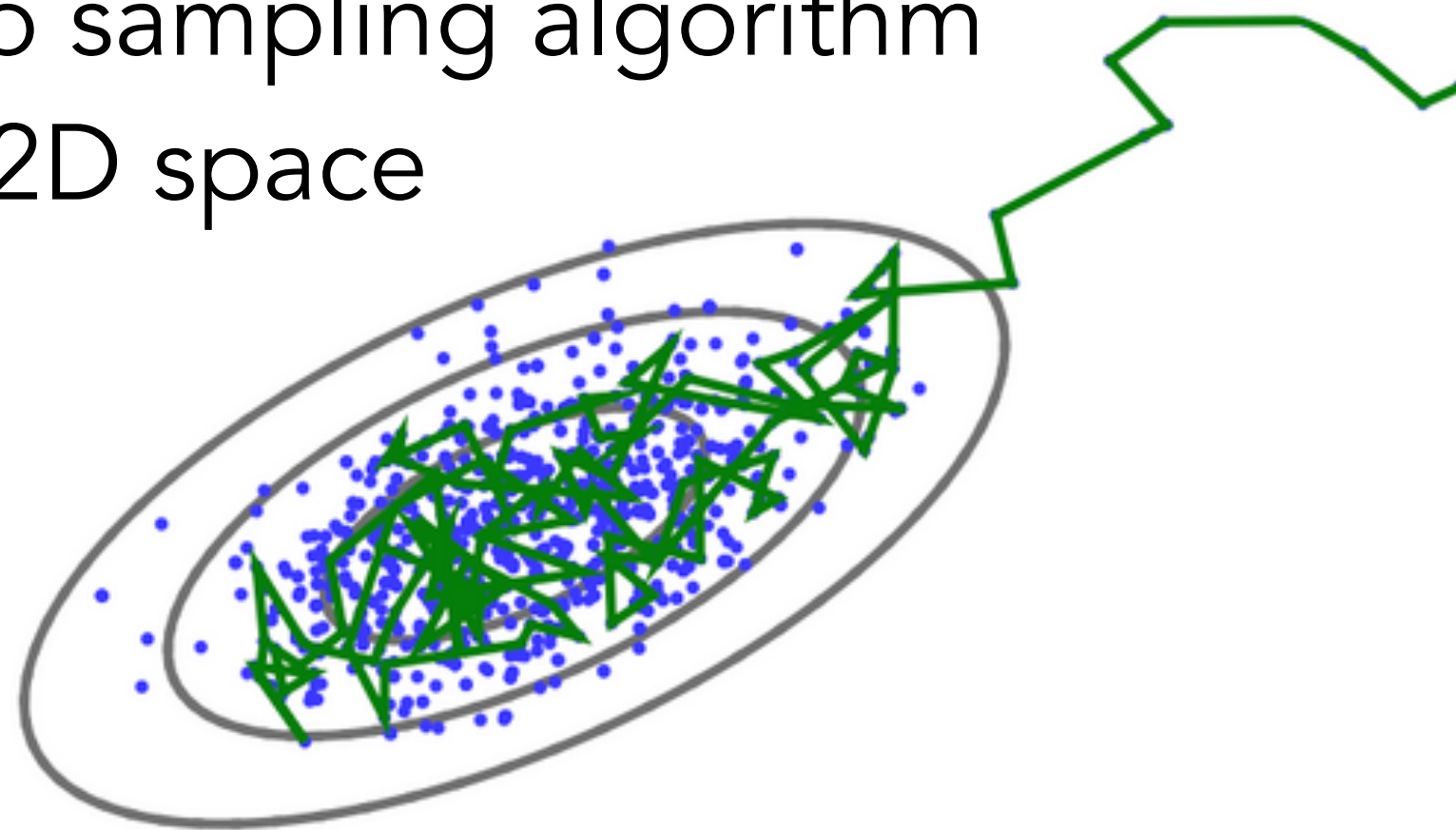
<https://chi-feng.github.io/mcmc-demo/>

Can we trust the model results?

Inference via Markov Chain Monte Carlo (MCMC)



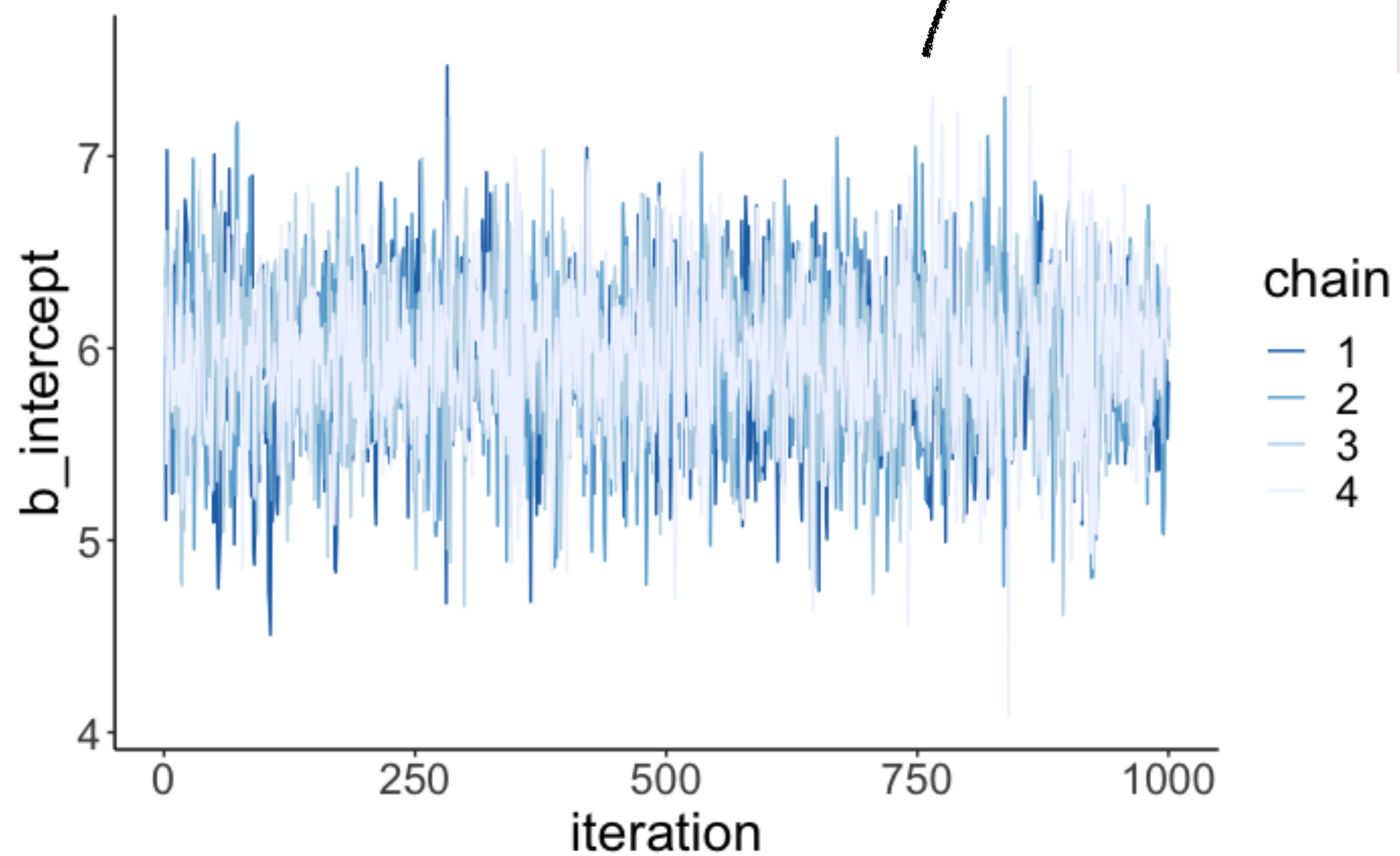
Markov Chain Monte Carlo sampling algorithm in a 2D space



goal: draw **independent** samples from the posterior distribution

Can we trust the model results?

**looks like a fuzzy
caterpillar**

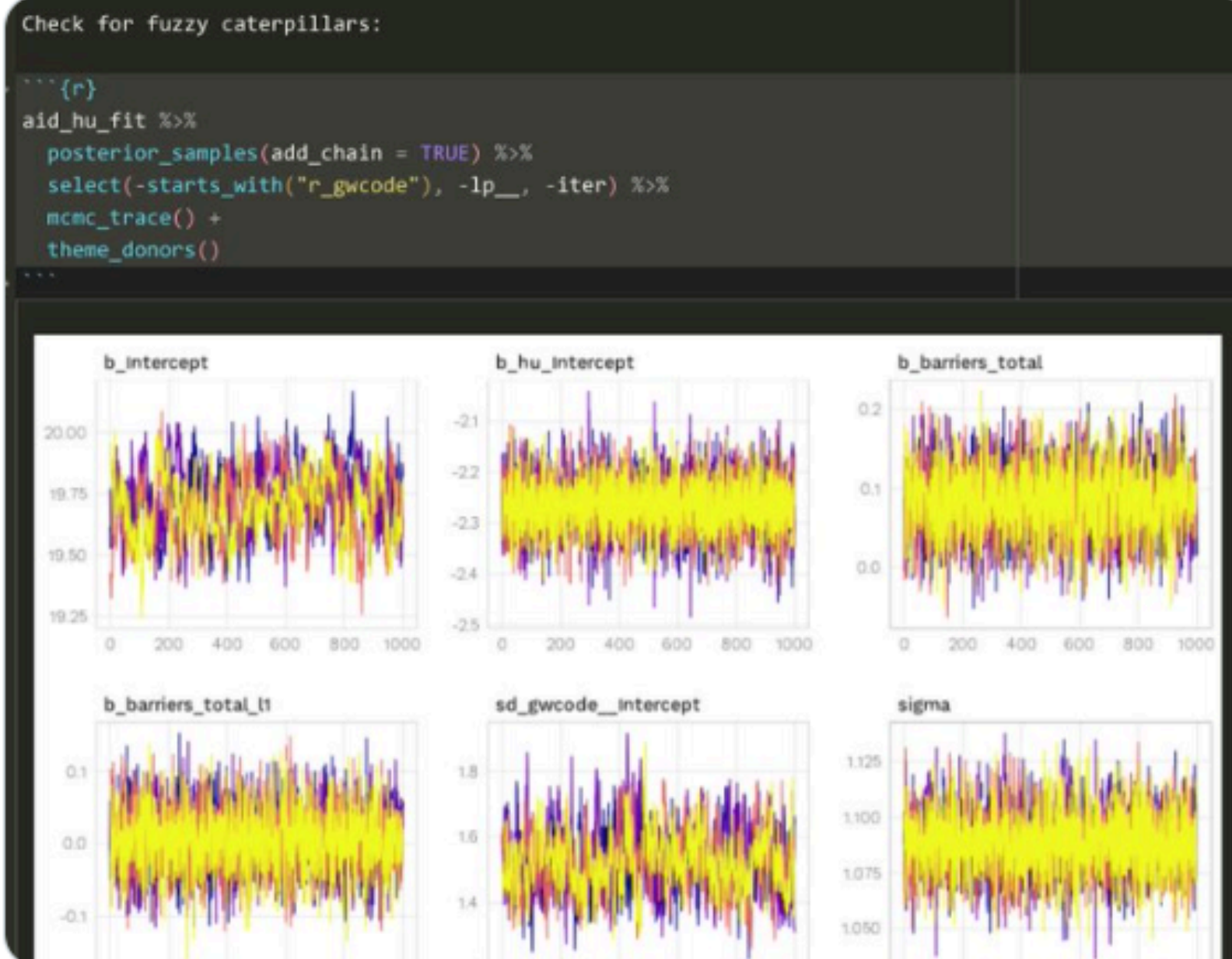


Stats twitter chimes in ...



Andrew Heiss
@andrewheiss

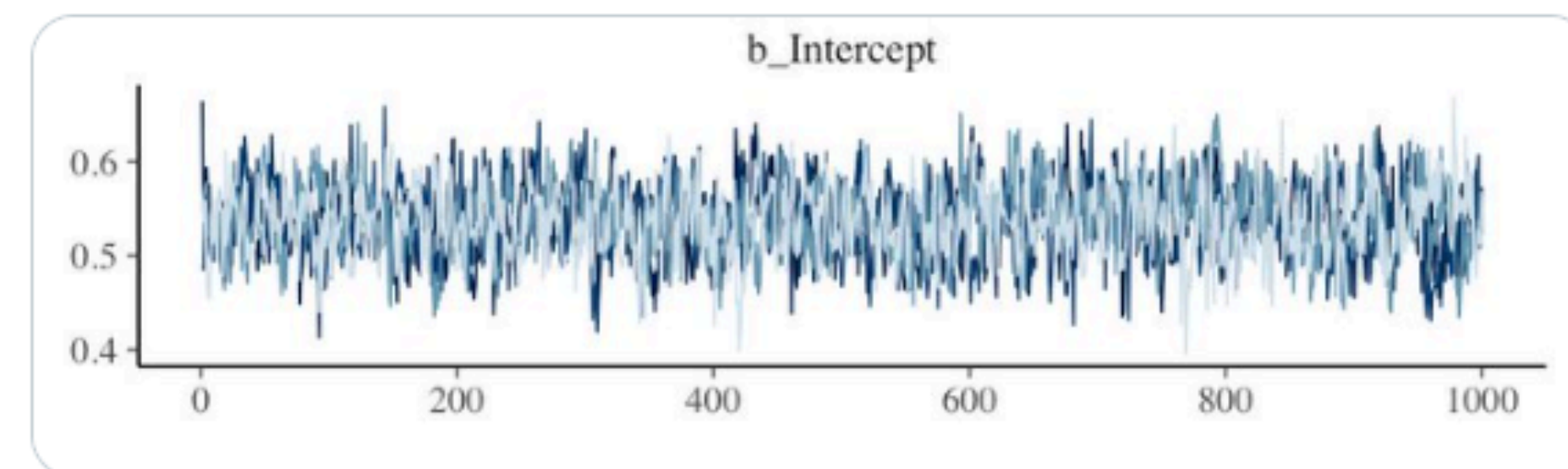
love that "looking for fuzzy caterpillars" is like a legitimate analytical strategy



Chelsea Parlett-Pelleriti
@ChelseaParlett

Do your chains just flow?
Do they sample to and fro?
Do they mix together well?
Is your R-hat small, or no?

Are your trace plots looking killer,
like a fuzzy caterpillar?
Do your chains just flow?



When things don't work out

← only two data points!

```
1 df.data = tibble(y = c(-1, 1))
2
3 fit.brm_wrong = brm(data = df.data,
4                     family = gaussian,
5                     formula = y ~ 1,
6                     prior = c(prior(uniform(-1e10, 1e10), class = Intercept),
7                               prior(uniform(0, 1e10), class = sigma)),
8                     inits = list(list(Intercept = 0, sigma = 1),
9                                   list(Intercept = 0, sigma = 1)),
10                    iter = 4000,
11                    warmup = 1000,
12                    chains = 2,
13                    file = "cache/brm_wrong")
```

← incredibly wide uniform priors

← 100000000000

When things don't work out

```
summary(fit.brm_wrong)
```

```
The model has not converged (some Rhats are > 1.1). Do not analyse the results!
We recommend running more iterations and/or setting stronger priors. There were 1203 divergent
transitions after warmup. Increasing adapt_delta above 0.8 may help.
See http://mc-stan.org/misc/warnings.html#divergent-transitions-after-warmup Family: gaussian
Links: mu = identity; sigma = identity
Formula: y ~ 1
Data: df.data (Number of observations: 2)
Samples: 2 chains, each with iter = 4000; warmup = 1000; thin = 1;
         total post-warmup samples = 6000

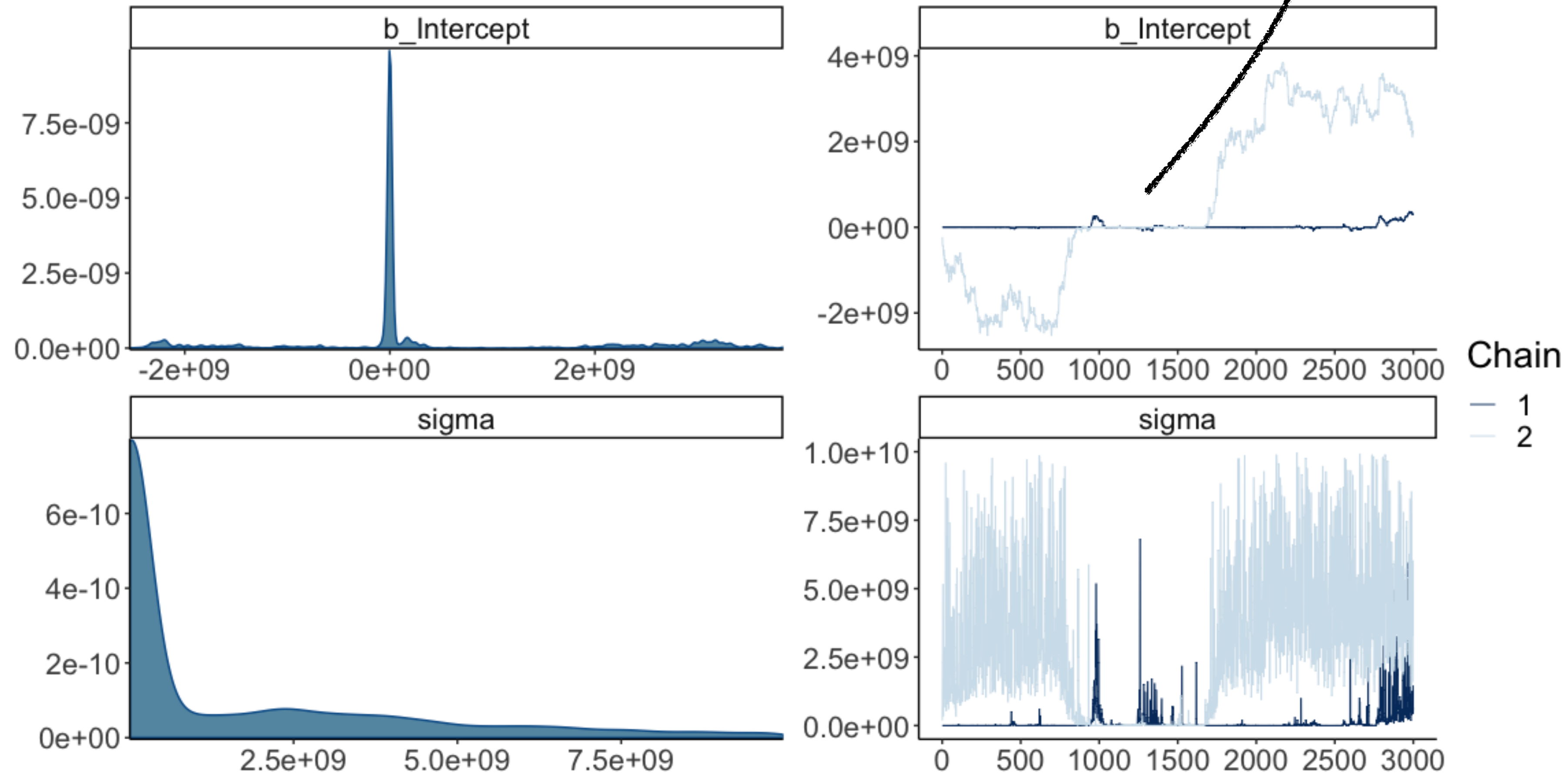
Population Level Effects:
      Estimate      Est.Error    1-95% CI      u-95% CI Rhat Bulk_ESS Tail_ESS
Intercept 357550121.58 1416057299.71 -2244033111.47 3333594132.43 1.78      3      24

Family Specific Parameters:
      Estimate      Est.Error 1-95% CI      u-95% CI Rhat Bulk_ESS Tail_ESS
sigma 1524412740.64 2392424321.98 21668.93 8317582240.06 1.40      4      41

Samples were drawn using sampling(NUTS). For each parameter, Eff.Sample
is a crude measure of effective sample size, and Rhat is the potential
scale reduction factor on split chains (at convergence, Rhat = 1).
```

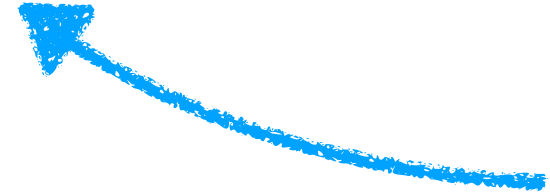
When things don't work out

doesn't look like a
fuzzy caterpillar



Having somewhat informative priors fixes things

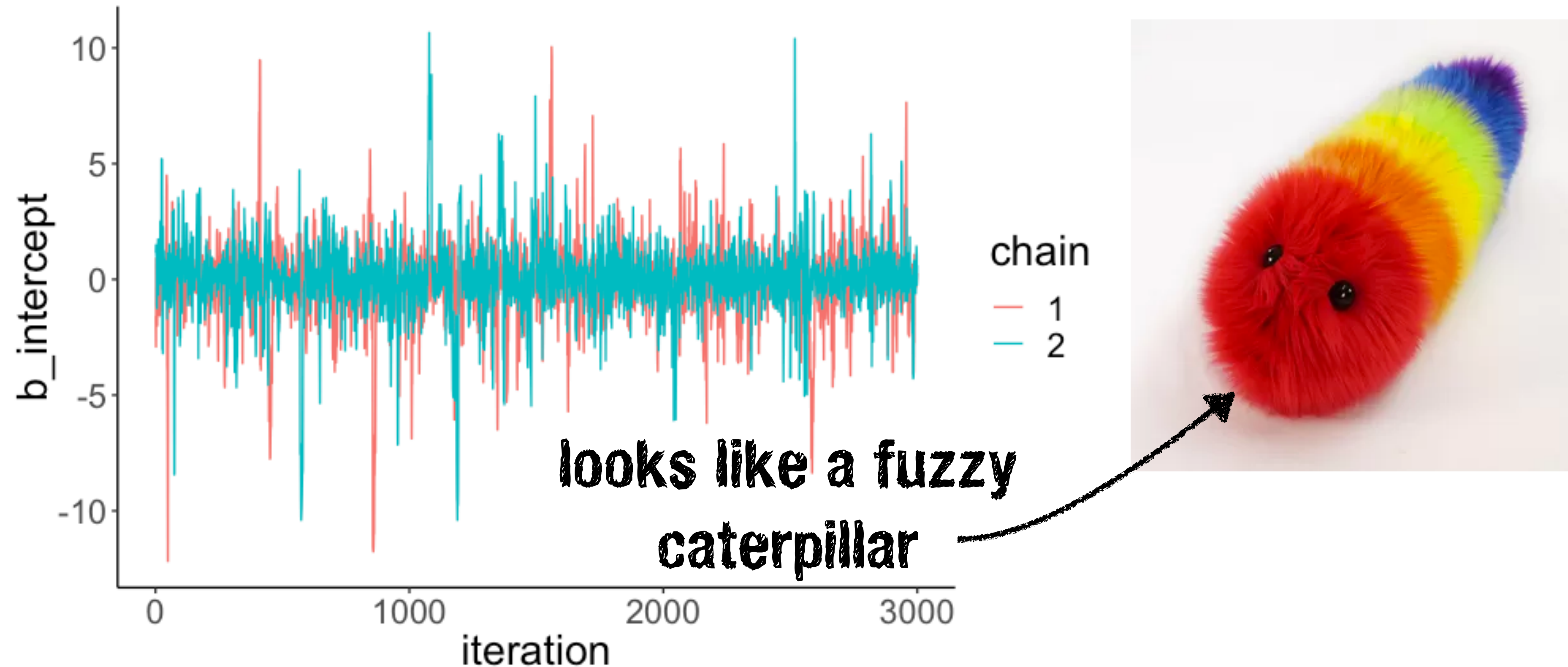
```
1 fit.brm_right = brm(data = df.data,  
2                     family = gaussian,  
3                     formula = y ~ 1,  
4                     prior = c(prior(normal(0, 10), class = Intercept), # more reasonable priors  
5                               prior(cauchy(0, 1), class = sigma)),  
6                     iter = 4000,  
7                     warmup = 1000,  
8                     chains = 2,  
9                     seed = 1,  
10                    file = "cache/brm_right")
```



more reasonable priors

```
Family: gaussian  
Links: mu = identity; sigma = identity  
Formula: y ~ 1  
Data: list(y = c(-1, 1)) (Number of observations: 2)  
Samples: 2 chains, each with iter = 4000; warmup = 1000; thin = 1;  
total post-warmup samples = 6000  
  
Population-Level Effects:  
      Estimate Est.Error 1-95% CI u-95% CI Eff.Sample Rhat  
Intercept    -0.06     1.72   -3.78    3.27     1033 1.00  
  
Family Specific Parameters:  
      Estimate Est.Error 1-95% CI u-95% CI Eff.Sample Rhat  
sigma      2.21     6.99    0.61    6.92     1006 1.00  
  
Samples were drawn using sampling(NUTS). For each parameter, Eff.Sample  
is a crude measure of effective sample size, and Rhat is the potential  
scale reduction factor on split chains (at convergence, Rhat = 1).
```


Having somewhat informative priors fixes things



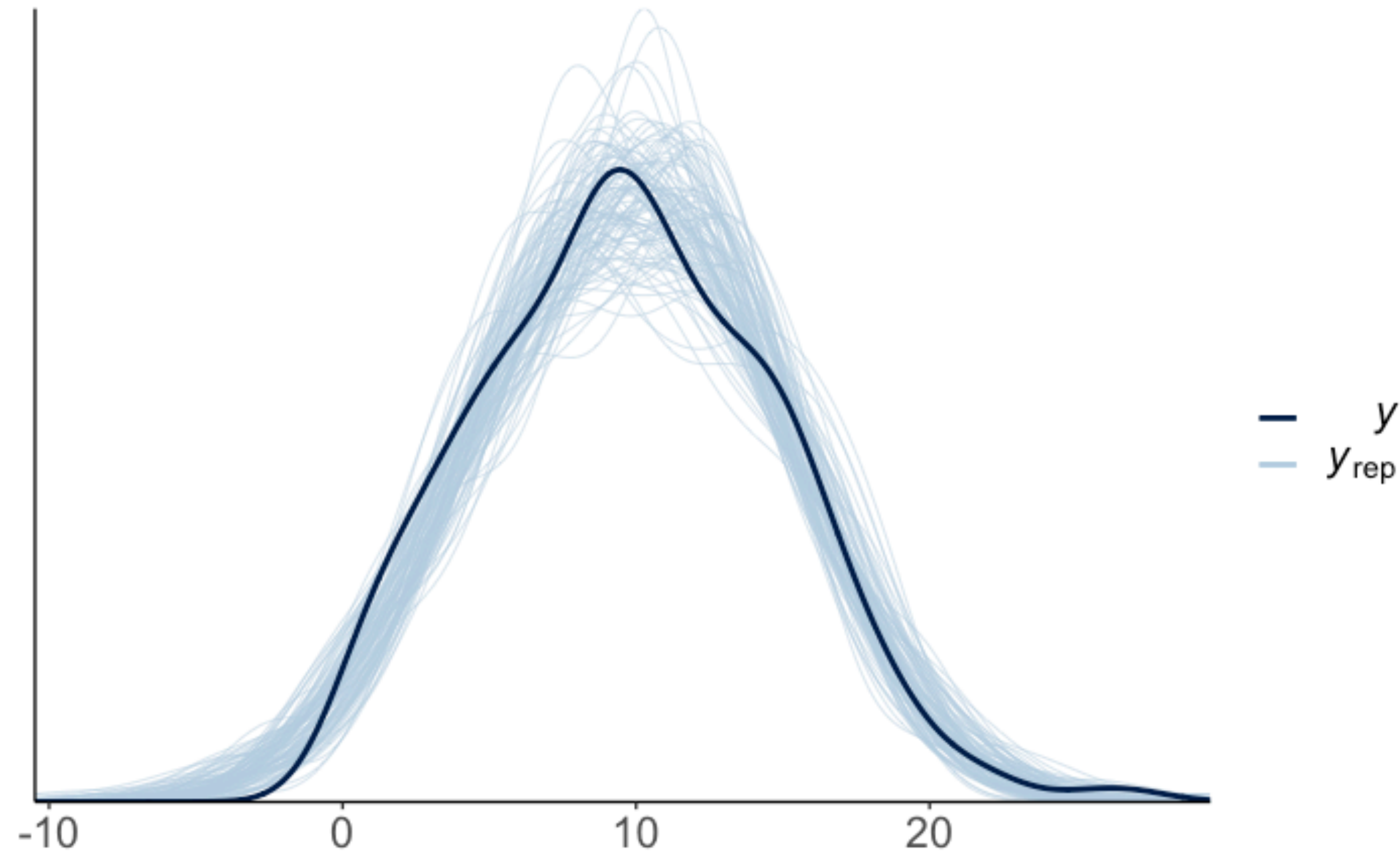
if things go wrong:

- set more informative priors
- run more warm-up samples
- adjust the sampling algorithm as suggested via the `control` argument

2. Visualize model predictions

Posterior predictive check

```
pp_check(fit.brm, nsamples = 100)
```

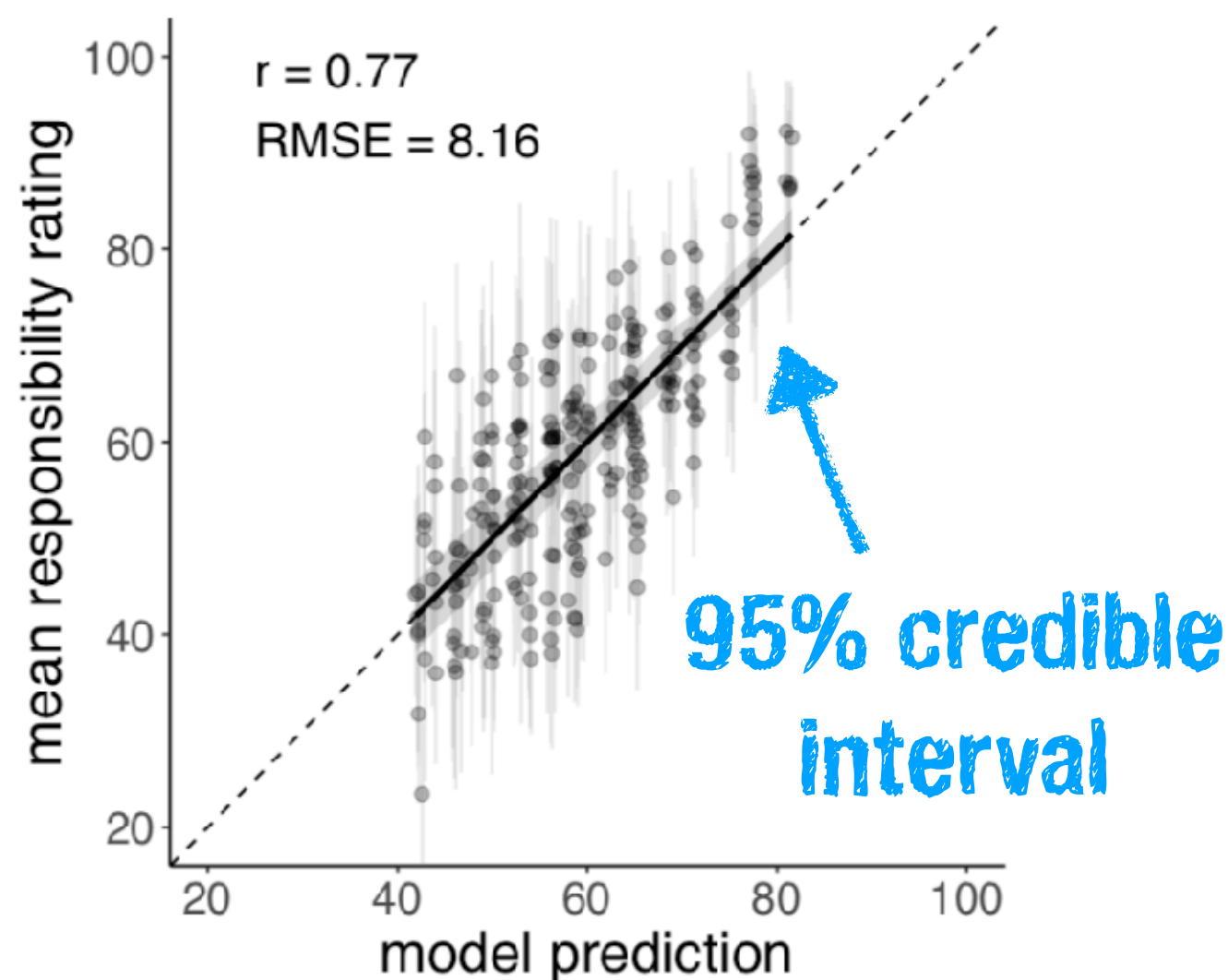


The model accurately captures the distribution of the response variable

Reporting results

Reporting results

Plots



Tables

Table 1
Estimates of the mean, standard error, and 95% HDIs of the different predictors in the Bayesian mixed effects model. Note: `n_causes` = number of causes.

`responsibility ~ 1 + surprise + pivotality + n_causes + (1 + surprise + pivotality + n_causes | participant)`

term	estimate	std.error	lower 95% HDI	upper 95% HDI
intercept	59.94	3.25	54.70	65.22
surprise	21.68	4.57	14.17	29.23
pivotality	13.52	1.82	10.47	16.53
n_causes	-5.72	0.50	-6.55	-4.90

model
formula

parameter
estimates

Text

We computed a Bayesian mixed effects model with random intercepts and slopes to predict participants' responsibility judgments (see Table 1). Figure 6b shows a scatter plot of the model predictions and participants' responsibility judgments for the full set of 170 scenarios (with 250 judgments). Overall, the model predicts participants' responsibility judgments well with $r = .77$ and $RMSE = 8.16$. Table 1 shows the estimates of the different predictors. As can be seen, none of the predictors' 95% HDIs overlap with 0.¹

¹For any statistical claim, we report the mean of the posterior distribution together with the 95% highest-density interval (HDI). All Bayesian models were written in Stan (Carpenter et al., 2017) and accessed with the brms package (Bürkner, 2017) in R (R Core Team, 2019).

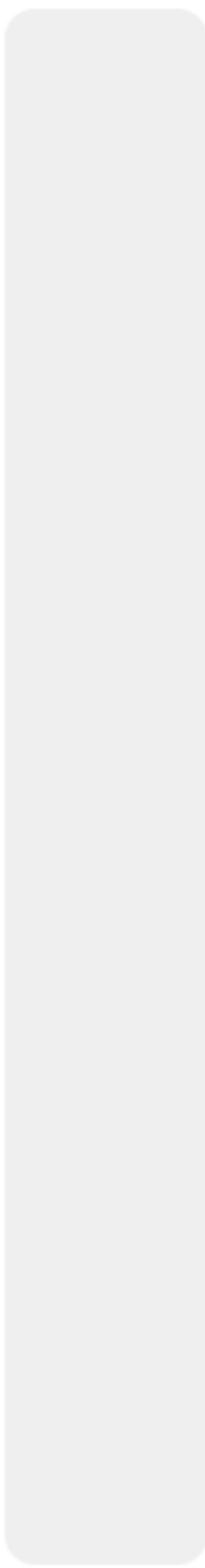
Plan for today

- Quick recap
- Bayesian data analysis
 - Ingredients: likelihood, prior, inference
 - Doing Bayesian data analysis
 - with `greta`
 - with BRMS

Feedback N

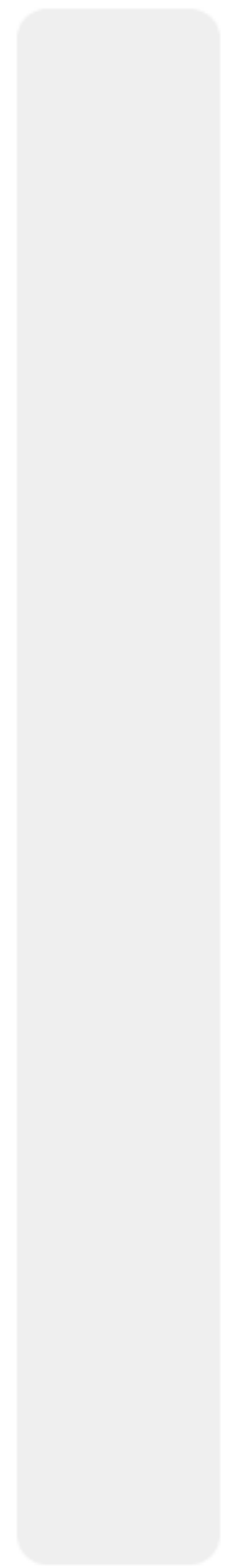


0%



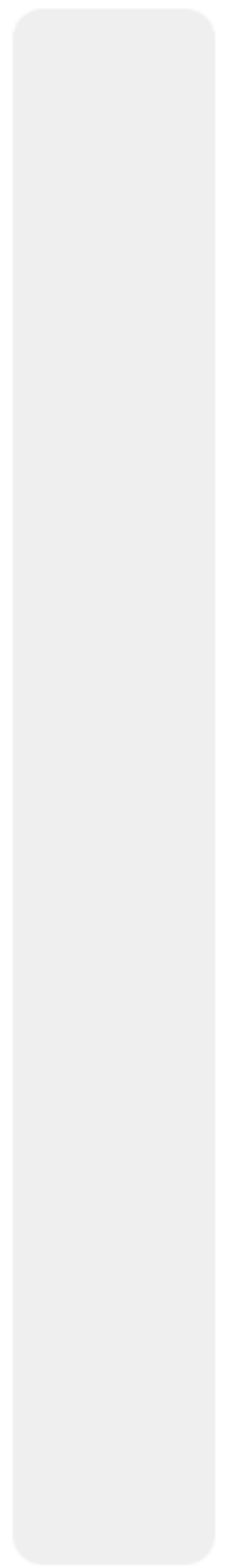
much too slow

0%



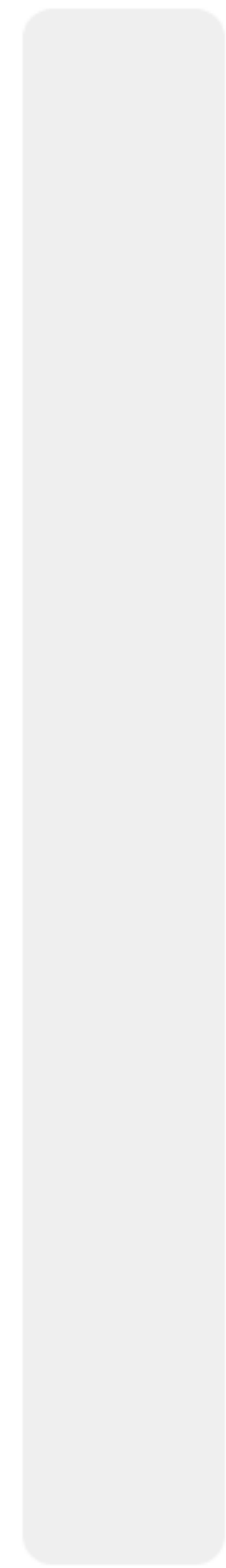
a little too slow

0%



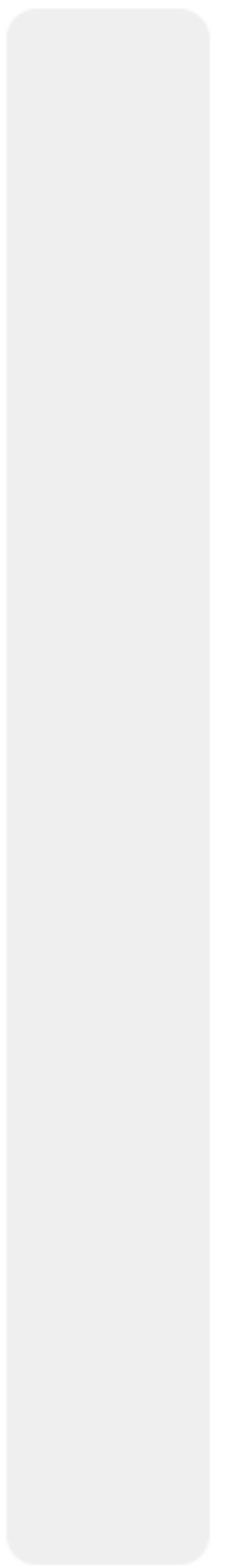
just right

0%



a little too fast

0%



much too fast



Thank you!